

CHAPTER 1

Chapter 1: Fundamentals of Multipoint Networks and Local Area Networks

Medium access protocols form a foundational layer in modern communication systems, particularly within the context of the Internet of Things (IoT). At its core, a communication channel is a system that allows for the remote transfer of data by exploiting a physical transmission medium.

Transmission across these channels generally occurs in one of two modes: point-to-point or multi-access. A point-to-point system consists of a single source node and a single destination node. In this isolated scenario, the link quality is entirely dependent on the transmitted signal and any external interference present along the channel. In contrast, a multi-access system connects two or more source and destination systems. Because multiple devices share the medium, the link quality is directly impacted by the signals transmitted simultaneously from these various nodes.

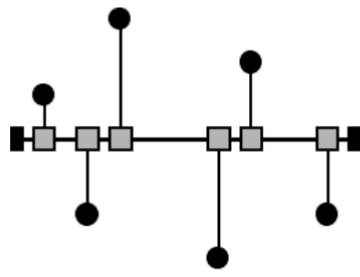
A multipoint network is defined as a communication system connecting independent devices—referred to as stations—that operate within a limited geographical area and utilize a shared physical channel. Due to their restricted geographical extent, multipoint networks are most commonly referred to as Local Area Networks (LAN).

LANs are fundamentally distinguished by two main criteria:

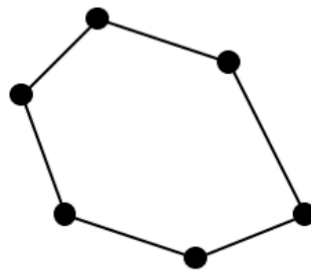
- **Network Topology:** This refers to the physical or logical interconnection mode between the stations.
- **Access Protocol:** This encompasses the specific procedures implemented by each station for transmitting information units to other stations over the shared medium.

LAN Topologies

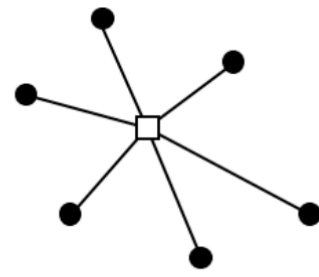
For wired communication media (often called wired multipoint networks or wired LANs), the most common network topologies are the bus, ring, and star configurations.



Bus



Ring



Star

Description: The theoretical schema illustrates a generic LAN represented as a central cloud connecting various independent devices (laptops, desktops, a server, and a printer). Furthermore, structural diagrams define the three main wired topologies: The **Bus** topology features devices connected linearly along a single main communication line with terminating ends. The **Ring** topology connects nodes sequentially in a closed-loop polygon. The **Star** topology links all outlying peripheral nodes to a single central hub or switch.

A unifying characteristic of all wired network topologies—and naturally wireless ones as well—is the "broadcast" nature of the transmission medium, meaning signals sent by one node can potentially be heard by all others. It is important to note that in the case of wireless networks (networks without wiring), it is not possible to identify a rigid physical network topology.

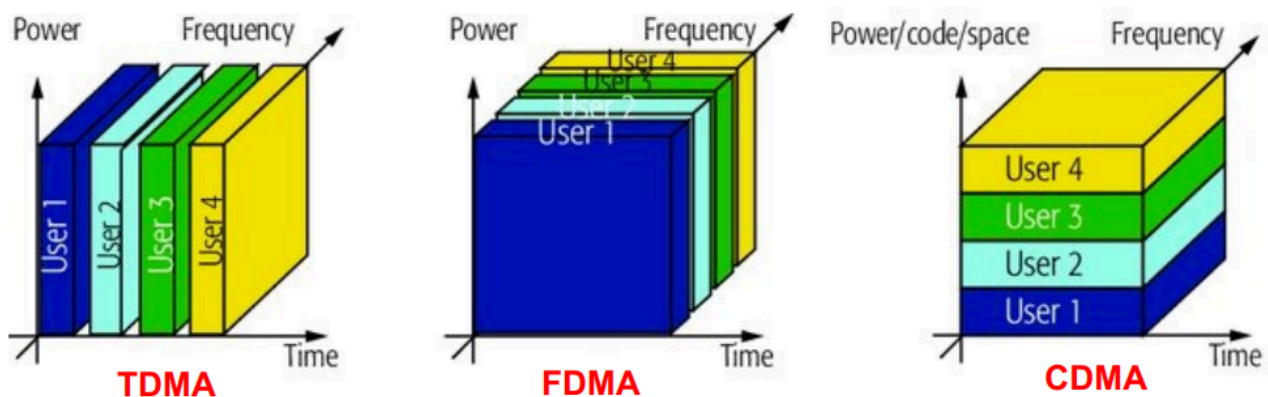
Chapter 2: Classification of Medium Access Protocols

To manage the broadcast nature of the shared medium, multipoint networks rely on specific multiple access techniques. There are several access protocols for LAN media, broadly categorized into static and dynamic allocation methods.

Static Access Techniques (Channelization)

Static access techniques divide the available transmission capacity in the network by pre-assigning it to the different stations, completely regardless of their actual, real-time need for use. Because they carve up the channel into dedicated portions, these are also referred to as "channel assignment" or "channelization" techniques. There are three primary channelization techniques, distinguished by how they divide the channel capacity:

- **Frequency Division Multiple Access (FDMA):** Each station can freely access the transmission medium using a pre-assigned portion of the available frequency capacity.
- **Time Division Multiple Access (TDMA):** The entire frequency capacity of the transmission medium is made available to each station, but only during specific, disjoint time intervals. This method strictly requires a synchronization mechanism to uniquely inform the various stations of the specific time interval assigned to them for transmission.
- **Code Division Multiple Access (CDMA):** This technique makes the entire channel bandwidth available to all stations simultaneously. It is possible to distinguish the transmitted signals upon reception because the various stations use different, unique codes to generate their transmitted signals.



Description: A 3D graphical comparison models TDMA, FDMA, and CDMA across Power, Frequency, and Time axes. In TDMA, time is sliced sequentially among Users 1-4, while each user occupies the full frequency and power spectrum during their slot. In FDMA, the frequency axis is sliced longitudinally among the users, while they transmit continuously over time. In CDMA, all users transmit continuously over the entire time and frequency blocks, but are separated via the Power/code/space axis.

Dynamic Access Techniques

Dynamic access techniques, conversely, take into account the actual, immediate needs of the stations to occupy the shared transmission medium. They assign access to the communication channel based on these real-time requirements. While dynamic techniques are more efficient in their use of bandwidth, they are inherently more complex to design and implement. Dynamic techniques are divided into centralized and distributed approaches:

- **Centralized Access:** A specific network element, known as the base station, performs the function of receiving requests for access to the medium and authorizing the sequence of transmissions. This system typically utilizes two communication channels. The first is a unidirectional channel shared among all stations, used exclusively to request access from the base station. The second is a bidirectional channel used to send information units to all stations; this channel is also utilized by the base station to communicate the sequence of channel access rights.
 - **Distributed Access:** In this model, the stations mutually coordinate access to the transmission medium among themselves. This is achieved using a single bidirectional communication channel where both the exchanged information units and the necessary control information travel together. Notice that, unlike channelization techniques, a portion of the actual channel capacity must be sacrificed to implement the access control procedures.
-

Chapter 3: The Resource Allocation Problem and Collisions

In any access protocol where stations are distributed within a limited geographical area, a coordination action must be provided to avoid conflicts over the shared transmission medium. This resource allocation can occur statically (subdivision into subchannels with individual pre-assignment, like FDMA, TDMA, CDMA) or dynamically (where the medium is a single resource accessed via a control procedure on-demand or pre-assigned).

To understand the core problem, let us consider N independent stations operating on a single channel. The probability that a new packet will be generated in a given time instant ΔT is dictated by the arrival rate of a new packet, mathematically represented as $\lambda \Delta T$.

Once a packet has been generated, the station will become blocked and remains inactive until the packet has been completely transmitted. A critical failure state arises if multiple stations transmit simultaneously; when this happens, a collision of the transmitted packets occurs. Collisions result in mutual interference. For instance, transmissions occurring in the same band will result in consequent interference during reception, destroying the data. Conversely, transmissions separated into different bands (like in FDMA) guarantee the absence of interference in reception.

Chapter 4: Performance Key Performance Indicators (KPIs)

The mathematical performance of a multipoint network can be expressed by evaluating three critical traffic parameters: Throughput, Time delay, and Packet loss probability.

1. Offered Traffic (A_0) The traffic offered to the network is measured in bits per second ($[bit/s]$). It is expressed as the product of the total average arrival rate of Information Units (IUs) and their fixed length. The formula is: $A_0 = N\lambda \cdot L$

Where:

- N is the number of nodes in the network.
- λ is the average arrival rate ($[1/s]$) of IUs to be transmitted.
- L is the IU packet length.

2. Throughput (A_s and S) The throughput ($[bit/s]$) represents the portion of the offered traffic that is successfully transferred through the network and delivered to the respective destinations. It is defined as: $A_s = N\lambda L \cdot (1 - \Pi_p)$

Where Π_p is the probability that a generic IU is not successfully transferred (i.e., lost or corrupted) from source to destination.

In a theoretical scenario where no events lead to the loss of information units (assuming $\Pi_p = 0$), it is possible to express the network throughput normalized to the total network capacity C . This yields the network utilization coefficient, denoted as S or ρ . $S = \frac{N\lambda L}{C}$ The parameter S (or ρ) indicates the specific portion of time during which the network is actively engaged in transmitting information units. Consequently, the relationship $0 \leq \rho \leq 1$ must always hold true. The overall network throughput in $[bit/s]$ can therefore be expressed as $THR = S \cdot C = \rho \cdot C$.

Note on Packet Loss: By assuming an ideal transmission medium free from transmission errors, and assuming that a generic station has a very large buffer (meaning IUs exchanged with the upper layer are never dropped), the probability of data unit loss can be mathematically neglected.

Chapter 5: Detailed Analysis of Static Access Protocols

As established, static access (or channelization) techniques divide the available capacity of the network and pre-assign it to the stations, completely independent

of their real-time demand. Let's deeply examine the three primary architectures: TDMA, FDMA, and CDMA.

5.1 Time Division Multiple Access (TDMA)

In TDMA, a specific time window—known as a slot—is allocated to each station. This slot provides enough time to transmit exactly one information unit (IU). Because the station has access to the full bandwidth of the channel C during its turn, the duration of a single slot s is mathematically defined as:

$$s = \frac{L}{C}$$

The time axis is structured into fixed-duration loops called **frames**, which are further divided into a fixed number of slots N (corresponding to the number of stations). Consequently, the duration of one complete frame is:

$$\text{Frame Duration} = N \cdot \frac{L}{C}$$

[Photo: Bandwidth allocation with TDMA] Description: A linear timeline illustrating the TDMA concept. The timeline is divided into sequential blocks labeled "Frame 1", "Frame 2", etc. Inside each frame, the time is subdivided into smaller identical slots labeled 1, 2, 3, up to N . The total duration of one frame is marked as NL/C .

A real-world example of this is the GSM cellular standard, where a frame lasts exactly 4.62 ms and is divided into 8 slots, making each slot 0.577 ms long .

Throughput and Delay in TDMA: Assuming an infinite buffer size, the network throughput is the sum of the traffic received by all stations. Let $T_t = L/C$ be the transmission delay of a single IU. The throughput S (or network utilization ρ) is expressed as:

$$S = \rho = N \cdot \lambda T_t = N\lambda \frac{L}{C}$$

When a packet is generated, it experiences three distinct components of time delay before it successfully reaches its destination (assuming negligible propagation time) :

1. **Transmission time** (T_t)
2. **Queuing time** (Waiting in the station's buffer)
3. **Slot waiting time** (Waiting for the station's assigned turn in the frame to arrive)

Assuming packet arrivals follow a random Poisson process, the total average time delay D_{TDMA} is:

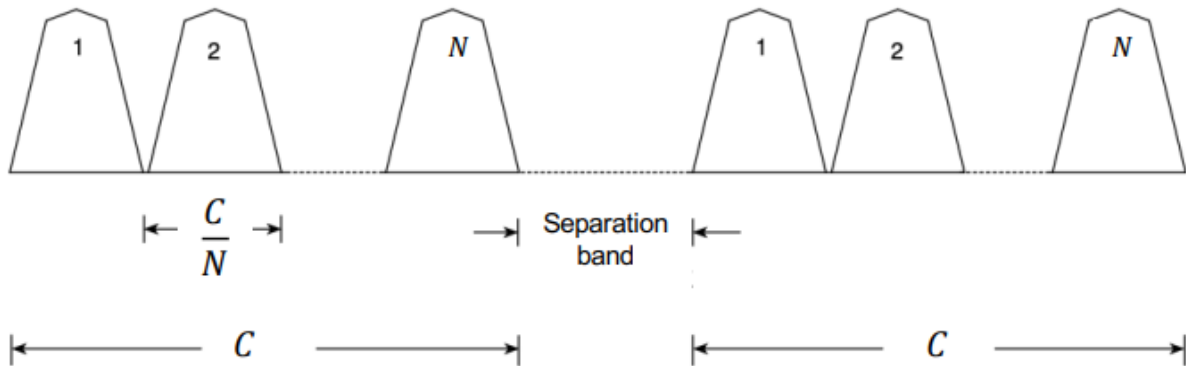
$$D_{TDMA} = \frac{L}{C} + \frac{\rho}{2(1-\rho)} \frac{NL}{C} + \frac{1}{2} \frac{NL}{C}$$

Note on Slot Waiting Time: The last term ($\frac{1}{2} \frac{NL}{C}$) assumes that, on average, a packet arrives at the buffer exactly halfway through the frame duration. For networks with a very large number of stations (N), this slot waiting time becomes the dominant factor, significantly outweighing the actual transmission time.

5.2 Frequency Division Multiple Access (FDMA)

Instead of slicing time, FDMA slices the frequency spectrum. The total available system bandwidth C is divided into N non-overlapping sub-bands. Each station is exclusively assigned one of these sub-bands (C/N) for the duration of its communication.

To prevent signals from bleeding into one another (aliasing effects), "guard bands" or separation bands are placed between the active sub-bands.



Description: A frequency axis graph showing continuous transmission. The total bandwidth C is divided into N trapezoidal shapes representing the individual sub-bands assigned to stations 1 through N . Between each trapezoid is a flat line representing the "Separation band" or guard band designed to prevent frequency overlap.

Duplexing in FDMA:

Bidirectional communication in FDMA can be achieved in two ways:

- **FDD (Frequency Division Duplex):** One contiguous block of sub-bands is dedicated to uplink transmissions, and a completely separate block is dedicated to downlink transmissions.

- **TDD (Time Division Duplex):** This applies a time-sharing approach to a single radio channel. The channel is bidirectional: the first half of a time frame handles uplink, and the second half handles downlink. This is highly advantageous when spectrum is scarce, as it avoids the need to secure two separate frequency bands .

Throughput and Delay in FDMA: Because each station only has access to $1/N$ of the total bandwidth, the physical transmission time for an IU is N times slower than in TDMA:

$$T_{FDMA} = \frac{L}{C/N} = \frac{NL}{C}$$

The throughput S , however, remains identical to TDMA because what FDMA lacks in speed, it makes up for in continuous transmission availability :

$$S = \rho = \lambda T_{FDMA} = \lambda \frac{NL}{C}$$

Because an FDMA station never has to wait for its "turn" (its frequency slice is always open), the slot waiting time component drops to zero. The total overall time delay D_{FDMA} is strictly the transmission time plus the queuing time :

$$D_{FDMA} = \frac{NL}{C} + \frac{\rho}{2(1-\rho)} \frac{NL}{C}$$

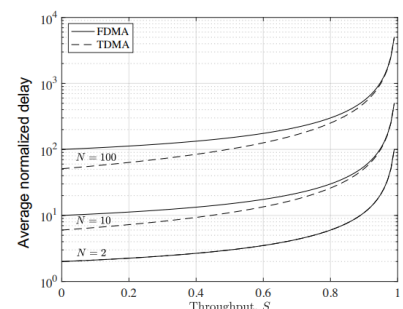
5.3 Performance Comparison: TDMA vs. FDMA

We can directly compare these two protocols by looking at their normalized delay times (relative to the base transmission time L/C):

$$D_{TDMA\ n} = \frac{2(1-\rho)+N}{2(1-\rho)}$$

$$D_{FDMA\ n} = \frac{N(2-\rho)}{2(1-\rho)}$$

Description: A logarithmic plot comparing the Average normalized delay on the Y-axis against Network Throughput S on the X-axis. Three pairs of curves map scenarios for $N = 2$, $N = 10$, and $N = 100$ users. In every scenario, the solid line representing FDMA sits higher on the delay axis than the dashed line representing TDMA. Both lines curve upwards toward infinity as throughput approaches 1.



By charting this relationship, it is evident that delay time increases as throughput grows. At very low traffic volumes, queuing time is negligible . Overall, TDMA demonstrates mathematically better performance (lower delay times) than FDMA; however, this performance comes at the cost of requiring precise clock synchronization among all stations, which adds system complexity .

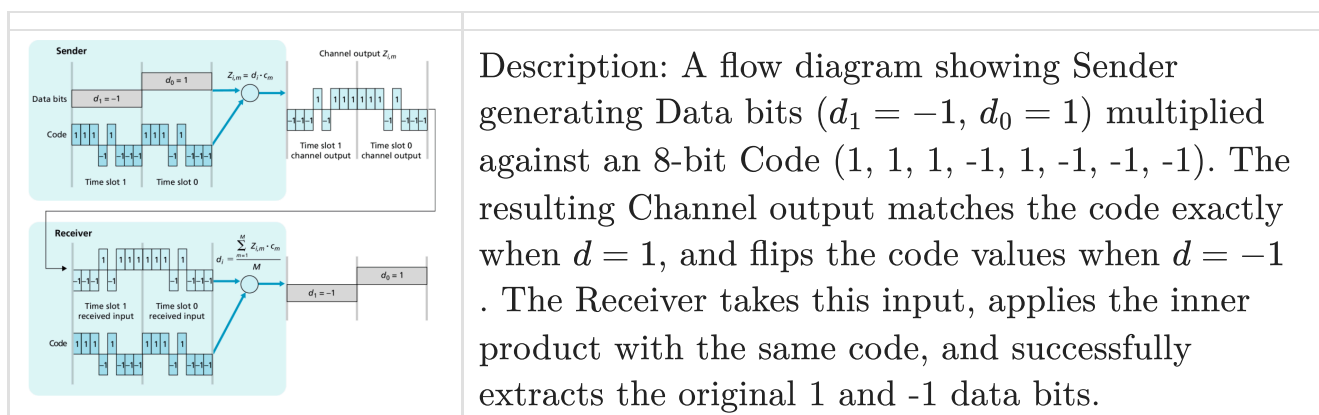
5.4 Code Division Multiple Access (CDMA)

CDMA takes a radically different approach. It assigns the entire channel bandwidth to all stations simultaneously, allowing them to transmit at the exact same time and on the exact same frequency. Interference is minimized, and signals are separated, through the use of unique, mathematically orthogonal codes assigned to each user .

The Encoding and Decoding Process: In CDMA, the sequence of bits a user wants to transmit is multiplied by a highly specific signal (the code). This code changes much faster than the actual data bits. The speed of the code is known as the "chipping rate".

For mathematical computation, a data bit of 0 is represented as -1 , and a bit of 1 remains 1. Each standard data bit time slot is subdivided into M "mini-slots" (e.g., $M = 8$). A user's unique CDMA code consists of a sequence of M values (c_m), each taking a value of either $+1$ or -1 .

- **Encoding:** For the m -th mini-slot, the output of the encoder ($Z_{i,m}$) is the data bit (d_i) multiplied by the code bit (c_m). $Z_{i,m} = d_i \cdot c_m$
- **Decoding (No Interference):** To extract the data, the receiver takes the incoming signal, multiplies it by the exact same unique code, and averages the result over the M mini-slots. $d_i = \frac{1}{M} \sum_{m=1}^M Z_{i,m} \cdot c_m$



Handling Interference: The true power of CDMA lies in managing multiple simultaneous transmissions. When N transmitters broadcast at once, the physical channel simply sums all the signals together.

The receiver receives the composite signal $Z_{i,m}^*$:

$$Z_{i,m}^* = \sum_{s=1}^N Z_{i,m}^{(s)}$$

To decode a specific source's data out of this noise, the receiver applies the target source's unique orthogonal code to the summed composite signal.

$$d_i = \frac{1}{M} \sum_{m=1}^M Z_{i,m}^* \cdot c_m$$

Chapter 6: Practical Applications and Network Mathematics

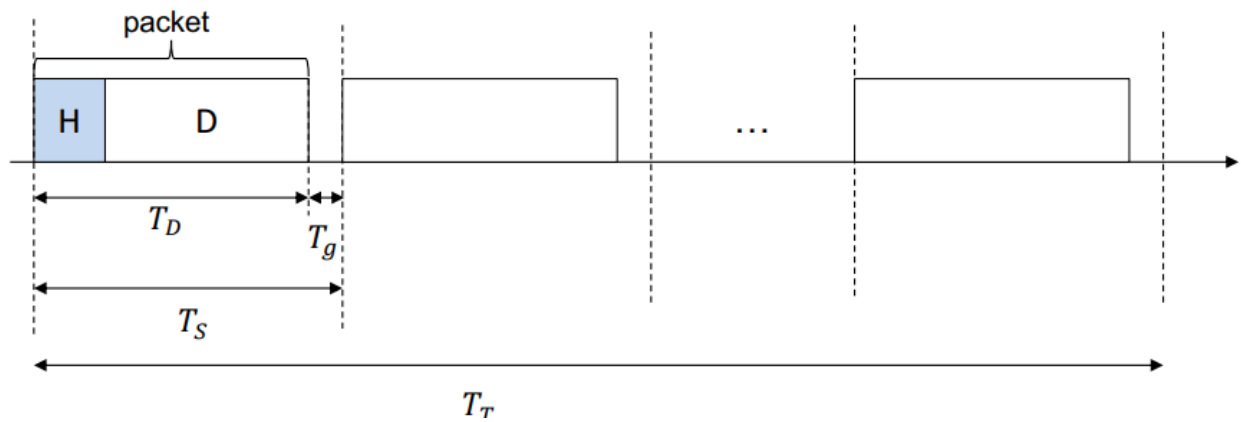
To solidify these theoretical concepts, let us explore three practical network calculations involving static access protocols.

Example 1: Delay and Throughput Comparison Consider a network with $N = 16$ stations. Each station offers traffic at $\lambda = 40$ IU/s. Packet length $L = 1000$ bytes, and network capacity $C = 10$ Mbps.

- **Throughput:** Regardless of whether TDMA or FDMA is used (assuming no loss), the throughput ρ is the same: $\rho = \lambda \frac{NL}{C} = 40 \cdot \frac{16 \cdot (1000 \cdot 8)}{10 \cdot 10^6} = 0.512$ Carried traffic $A_s = 5.12$ Mbps.
- **TDMA Delay:** Plugging into the formula, $D_{TDMA} = 0.8$ ms (transmission) + 6.71 ms (queuing) + 6.4 ms (slot wait) = **13.91 ms**.
- **FDMA Delay:** $D_{FDMA} = 6.71$ ms (queuing) + 12.8 ms (transmission) = **19.51 ms**.
- *(This practically demonstrates the theoretical conclusion that TDMA yields lower delays).*

Example 2: TDMA Framing Mechanics A TDMA system uses $N = 10$ time slots. The frame duration $T_T = 10$ ms. Each packet consists of $D = 180$ bits of data and an $H = 20$ bit header. To prevent overlapping, a guard time of

$T_g = 200\mu s$ is enforced between slots.



- The total slot size $k = D + H = 200$ bits.
- The raw slot time $T_S = T_T / N = 10 \text{ ms} / 10 = 1 \text{ ms}$.
- The actual time dedicated to data transmission $T_D = T_S - T_g = 1 \text{ ms} - 200\mu s = 0.8 \text{ ms}$.
- The **multiplexed transmission rate** W (speed during the active pulse) = $k / T_D = 200 \text{ bits} / 0.8 \text{ ms} = \mathbf{250 \text{ kbps}}$.
- The **net channel rate** V (effective speed of useful data over the whole frame) = $D / T_T = 180 \text{ bits} / 10 \text{ ms} = \mathbf{18 \text{ kbps}}$.

Example 3: Centralized TDMA Capacity A centralized TDMA system features a slot duration $T_S = 10\mu s$ and a guard time $T_G = 2\mu s$. It serves $N = 8$ users with a multiplexed signal rate of $C = 1 \text{ Mbps}$.

1. **Bits per slot (n):** Taking away the guard time, $(10 - 2)\mu s \cdot 1 \text{ Mbps} = \mathbf{8 \text{ bits}}$.
2. **Max input rate per stream (r):** The station sends 8 bits every frame. A frame takes $N \cdot T_S = 8 \cdot 10\mu s$. Therefore, $r = \frac{8 \text{ bits}}{80\mu s} = \mathbf{100 \text{ kbps}}$.

Chapter 7: Dynamic Allocation and Medium Access Control

While static allocation guarantees bandwidth, it is fundamentally inefficient for networks with variable or bursty traffic. To address this, networks employ dynamic allocation, assigning resources on-demand. In dynamic allocation, the regulation of access to the shared medium—including the resolution of inevitable conflicts—is strictly handled by Medium Access Control (MAC) protocols .

Dynamic assignment is categorized into two main philosophies:

1. **Controlled Access:** Stations transmit *only* when they receive explicit permission.

- **Centralized:** A primary base station systematically enables secondary stations to transmit.
- **Distributed:** Control passes from station to station (e.g., token passing). Inactive stations do not occupy resources, resulting in highly efficient use of the transmission medium.

2. **Random Access (Contention Protocols):** Each station accesses the channel entirely independently of its actual availability. The Information Unit (IU) is transmitted the moment it is ready .

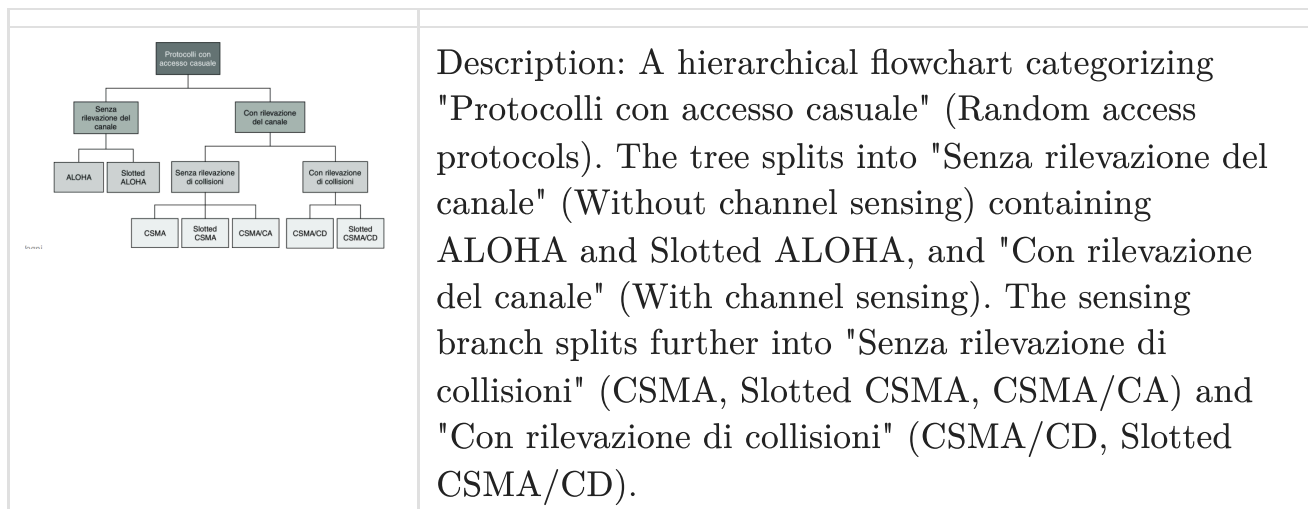
Because there are no pre-assigned transmission capacities, if two or more stations request the resource simultaneously, a contention system occurs—resulting in a collision . Collisions cause mutual interference, rendering the data unusable and forcing a retransmission. Consequently, performance relies heavily on the **vulnerability interval**: the maximum time window during which a station's transmission can collide with another's .

The Contention Protocol Family

Random access protocols are divided into two primary classes based on whether they "listen" to the channel before speaking :

1. **ALOHA:** Protocols that execute transmissions completely blind, without any channel sensing.
2. **CSMA (Carrier Sense Multiple Access):** Protocols that perform channel sensing and only allow transmission when the channel is detected as physically free.
 - **CSMA/CD (Collision Detection):** The channel is actively monitored *during* transmission. If a collision is detected, the transmission is immediately aborted.
 - **CSMA/CA (Collision Avoidance):** Prevents collisions proactively using an explicit request/authorization handshake protocol.

Note on Slotted Protocols: Except for CSMA/CA, every protocol has a "basic" (pure) version and a "slotted" version. In slotted versions, the time axis is divided into equal-duration slots. Stations cannot start transmitting at arbitrary instants; they must wait for the exact beginning of the next slot .



Chapter 8: The ALOHA Protocol

Developed in the early 1970s to interconnect stations across the Hawaiian islands via radio, ALOHA is the simplest random access protocol .

When a station (e.g., Station A) receives an IU from its upper network layer, it transmits it instantly. Because it does not check if the channel is busy, transmission success relies entirely on sheer luck—specifically, the absence of a collision .

The Acknowledgment and Backoff Mechanism: How does Station A know if it succeeded? It waits for an acknowledgment (ACK) message from the destination (Station B). This ACK is assumed to travel over a separate, ideal, collision-free channel.

1. Station A transmits the data.
2. Station A waits for an ACK for a timeout period equal to 2τ (where τ is the maximum end-to-end propagation time between the stations) .
3. If the ACK arrives within 2τ , the transmission is complete.
4. If the timeout expires with no ACK, a collision is assumed. To prevent infinite recurring collisions, stations must not retry simultaneously. Station A initiates a **Backoff Algorithm** .

[Photo: ALOHA Operational Flowchart] Description: A logical loop diagram showing the ALOHA algorithm. Data available -> Transmit data -> Wait for ACK (2τ). A diamond decision box asks "Data acknowledged?". If Yes ->

Transmission complete. If No $\rightarrow$ Calculate max delay $(K - 1)T \rightarrow$ Apply Delay $iT \rightarrow$ Loop back to Transmit data.

Calculating the Rescheduling Time: The station calculates a maximum delay interval $(K - 1)T$, where T is the transmission time of a frame and $K \geq 2$ is an integer. The station then randomly selects one specific delay value iT (where $i = 0, 1, \dots, K - 1$) and waits exactly that long before retransmitting .

Crucially, the value of K is dynamic. As the number of consecutive collisions for a specific packet increases, the system deliberately increases K . Expanding the maximum rescheduling time spreads out the competing stations, effectively reducing the probability of repeated collisions .

The Vulnerability Period: In basic ALOHA, if Station B transmits a frame (duration T), any transmission by Station A that begins up to time T *before* B starts, or up to time T *after* B starts, will overlap and cause a collision. Therefore, the total vulnerability period spans exactly $2T$.

Chapter 9: ALOHA Performance and Mathematical Delay

1. Network Throughput:

To evaluate efficiency, we define G as the offered traffic, which is the average number of total transmission attempts (new packets plus retransmissions) per time unit T .

$$G = \Lambda T$$

Network throughput S is the offered traffic multiplied by the probability of a successful transmission (P_s):

$$S = G \cdot P_s$$

Assuming packet arrivals follow a Poisson distribution (Λ) and there are an infinite number of stations, P_s is simply the probability that absolutely zero other IUs are generated during the vulnerability period $t = 2T$.

$$P_s = P_0(2T) = \frac{(\Lambda \cdot 2T)^0}{0!} e^{-\Lambda \cdot 2T} = e^{-2G}$$

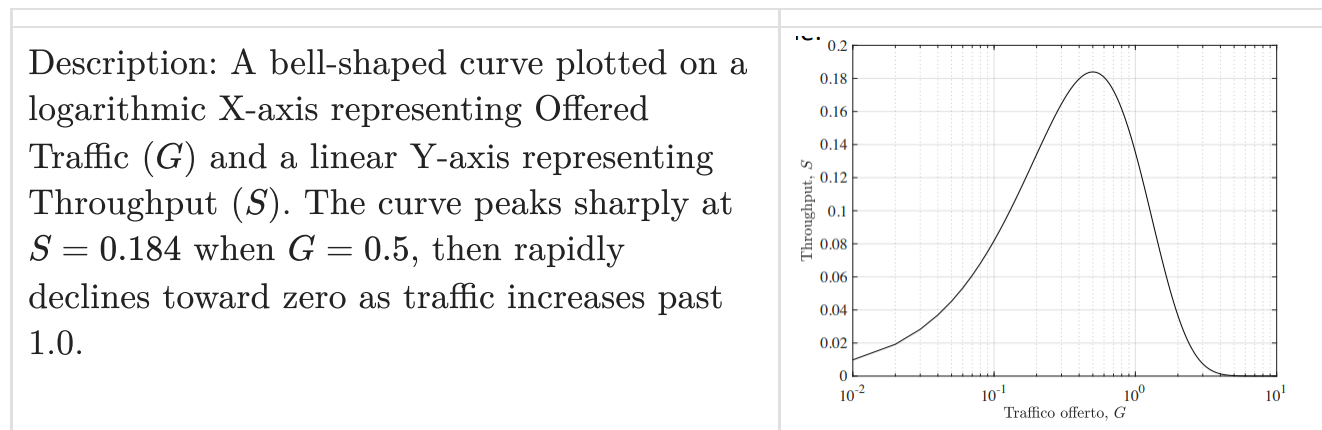
Thus, the pure ALOHA throughput formula is:

$$S = Ge^{-2G}$$

By analyzing this function, we find its peak efficiency. Setting the first derivative to zero ($S'(G) = e^{-2G}(1 - 2G) = 0$), we find that maximum throughput occurs exactly when $G = 0.5$.

$$S_{max} = 0.5e^{-1} = 0.184$$

This reveals a severe limitation: **ALOHA maxes out at only 18.4% of network capacity**. Furthermore, if the offered traffic exceeds $G = 0.5$, the network becomes unstable, and throughput plummets toward zero due to cascading collisions .



2. Time Delay (D): To find the total delay, we must determine how many collisions occur on average. The ratio $G/S = e^{2G}$ represents the total average number of transmission attempts. Therefore, the number of *unsuccessful* attempts is $e^{2G} - 1$.

The average backoff time B chosen from the K available intervals is $B = \frac{K-1}{2}T$.

The total average transmission delay D is the sum of the time wasted on all unsuccessful attempts plus the time taken for the final successful attempt:

$$D = (e^{2G} - 1) \cdot \left(T + 2\tau + \frac{K-1}{2}T\right) + T + \tau$$

To normalize this delay relative to frame transmission time T (where $a = \tau/T$):

$$D_n = \frac{D}{T} = (e^{2G} - 1) \cdot \left(\frac{K+1}{2} + 2a\right) + 1 + a$$

(As K increases, the average backoff time grows, significantly increasing the total delay).

Chapter 10: Practical System Analysis (Exercises)

Scenario 1: Calculating Throughput from Traffic Ratio Consider an ALOHA network where the total channel traffic (attempts) is 1.649 (or $\sqrt{e}$) times higher than the successfully transmitted frames. What is the throughput?

1. We know the ratio of attempts to successes: $G/S = 1.649$

2. Using the ALOHA relationship $G/S = e^{2G}$:

$$e^{2G} = 1.649$$

$$2G = \ln(1.649) = \ln(\sqrt{e}) = 0.5$$

$$G = 0.25$$

3. We calculate the final throughput:

$$S = G \cdot e^{-2G} = 0.25 \cdot e^{-0.5} = \frac{0.25}{1.649} = \mathbf{0.151}$$

Scenario 2: Collision Timeline (2 Mbps vs 1 Mbps)

Three stations (A, B, C) use ALOHA to send a $L = 1000$ byte packet. Assuming zero propagation time ($\tau = 0$).

Station A transmits at $t = 400$ ms.

Station C transmits at $t = 405$ ms.

Station B transmits at $t = 410$ ms.

Case A: Fast Network ($R = 2$ Mbps) Transmission time $T_{UI} = \frac{1000 \cdot 8}{2 \cdot 10^6} = 4$ ms.

- A occupies the channel: 400 to 404 ms.
- C occupies the channel: 405 to 409 ms.
- B occupies the channel: 410 to 414 ms.
- **Result:** No overlap. All packets arrive safely.

Case B: Slow Network ($R = 1$ Mbps) Transmission time doubles:

$$T_{UI} = \frac{1000 \cdot 8}{1 \cdot 10^6} = 8 \text{ ms.}$$

- A attempts: 400 to 408 ms.
- C attempts: 405 to 413 ms.
- **Collision 1:** C's start at 405 interrupts A's transmission. Both fail.
- B attempts: 410 to 418 ms.
- **Collision 2:** B's start at 410 interrupts C's ongoing (and already corrupted) transmission. Both fail.

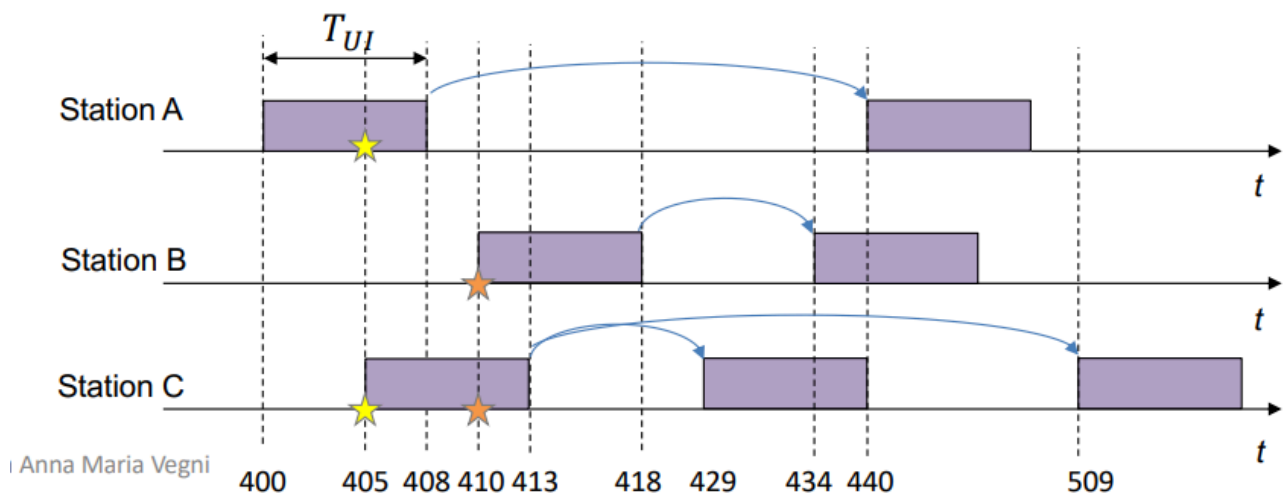
To resolve this, the stations apply a specific backoff formula: $\epsilon = \text{seed} \cdot 2N_c \cdot T_{UI}$, using a provided sequential seed array [2, 1, 1, 3, 2] .

- **Station A Retransmission:** Experiences 1 collision ($N_c = 1$). Takes the first seed (2). $\epsilon = 2 \cdot 2 \cdot 1 \cdot 8 = 32$ ms. New attempt = 408 (end of fail) + 32 = 440 ms. (Success).
- **Station C Retransmission:** Experiences 1 collision ($N_c = 1$). Takes the second seed (1). $\epsilon = 1 \cdot 2 \cdot 1 \cdot 8 = 16$ ms. New attempt = 413 (end of fail) + 16 = 429 ms.
- **Station B Retransmission:** Experiences 1 collision ($N_c = 1$). Takes the third seed (1). $\epsilon = 1 \cdot 2 \cdot 1 \cdot 8 = 16$ ms. New attempt = 418 (end of fail) + 16 = 434 ms.
- **Collision 3:** C is transmitting from 429 to 437. B starts at 434. They collide *again*.
- **Station C Second Retransmission:** Now has 2 consecutive collisions ($N_c = 2$). Takes the next seed (3). $\epsilon = 3 \cdot 2 \cdot 2 \cdot 8 = 96$ ms. New attempt = 413 (original fail) + 96 = 509 ms. (Success).

Station A ★ First collision, $\epsilon = 2 \cdot 2 \cdot 1 \cdot 8 = 32ms$
 Retransmission instant: $408 + 32 = \mathbf{440\ ms}$

Station B ★ First collision, $\epsilon = 1 \cdot 2 \cdot 1 \cdot 8 = 16ms$
 Retransmission instant : $418 + 16 = \mathbf{434\ ms}$

Station C ★ First collision, $\epsilon = 1 \cdot 2 \cdot 1 \cdot 8 = 16ms$
 Retransmission instant : $413 + 16 = \mathbf{429\ ms}$
 ★ Second collision, $\epsilon = 3 \cdot 2 \cdot 2 \cdot 8 = 96ms$
 Retransmission instant : $413 + 48 = \mathbf{509\ ms}$



1. The Slotted-ALOHA Protocol

1.1 Core Concepts and Synchronization

In slotted protocols, the time axis is fundamentally divided into fixed-length time intervals. Each of these intervals is equal to the transmission time of a User Information (UI) packet, denoted as T (time slot). To maintain order on the network, each station is constrained to start transmitting only at the beginning of a time slot. Consequently, a synchronization mechanism is required to indicate to all stations when they are allowed to start transmitting.

Unlike the standard continuous ALOHA protocol, if a UI is generated mid-slot, its transmission is delayed until the next slot start instant. This synchronization mechanism results in a halving of the vulnerability period. For example, if two UIs (A and B) are generated at different instants within the same slot, the two transmissions will fully overlap in the next slot. This reduces the vulnerability period—which is the interval during which the network is unusable due to interference—to the duration of a single UI transmission.

1.2 Protocol Flow and Retransmission

When a station has a new incoming UI available, it waits for the start of the next slot and attempts to transmit in that first available slot. After transmission, an acknowledgment (ACK) is expected within a time equal to $2\tau + \epsilon$, where ϵ is the time required for the occurrence of the next slot start.

If the data unit is acknowledged, the transmission is complete. However, if a collision occurs and the unit is not acknowledged, the station must retransmit the UI in subsequent slots with probability p , until it succeeds.

The Backoff Strategy: To manage collisions, the station attempts retransmission after a number of slots chosen uniformly within an interval $[0, r]$. The system calculates a maximum delay of $(K - 1)T$, applies a delay of iT , and retries.

- **If $r = 0$:** The collision repeats indefinitely, resulting in a throughput of 0.
- **If $r \gg 1$:** If traffic is high, a large value of r is needed to avoid instability.

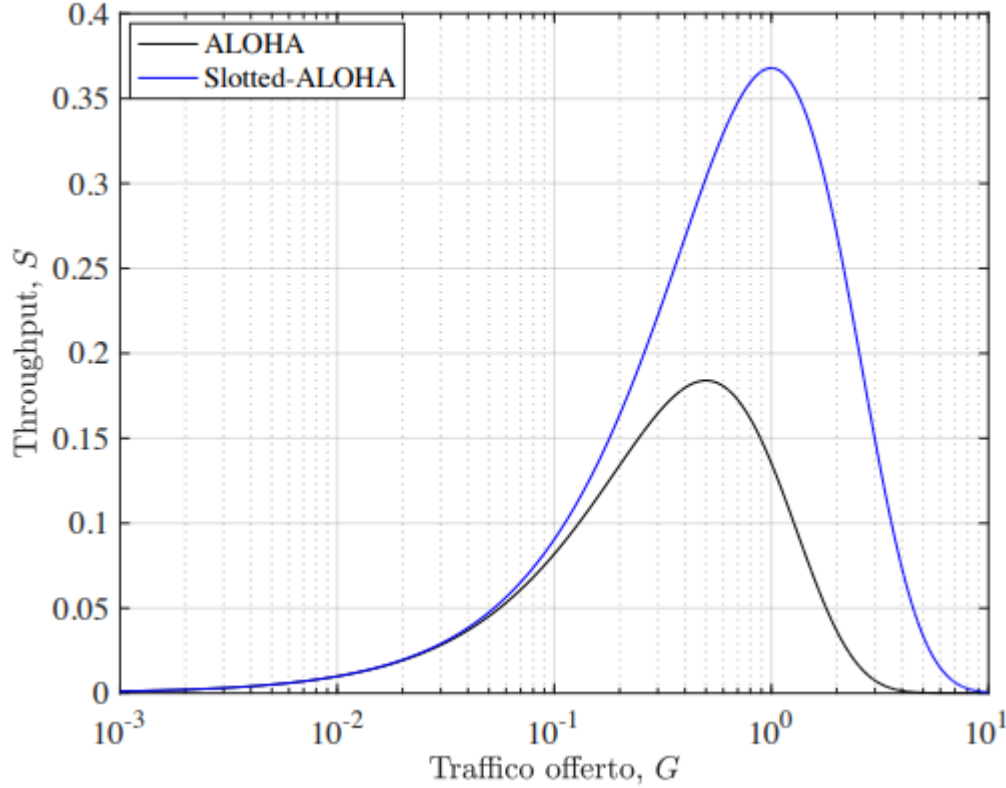
Ultimately, it is preferable to use a small r in a lightly loaded network, while a big r is better in congested network conditions.

2. Throughput in Slotted-ALOHA

2.1 Throughput with Infinite Stations ($N \rightarrow \infty$)

The throughput of the slotted ALOHA protocol can be computed similarly to standard ALOHA, but taking into account the halving of the vulnerability period. Assuming an infinite number of stations, the throughput S is calculated as:

$$S = G \cdot P_s = G \cdot \left[\frac{(\Lambda t)^0}{0!} e^{-\Lambda t} \Big|_{t=T} \right] = G \cdot e^{-\Lambda T} = G \cdot e^{-G}$$



2.2 Throughput with a Finite Number of Stations

In the case of a finite number N of mutually independent stations, a simple model allows the maximum throughput to be evaluated. Assuming that each station offers the same amount of traffic (G/N UIs per slot), the probability P_k that k transmission attempts occur in a generic slot is given by the binomial distribution:

$$P_k = \binom{N}{k} \left(\frac{G}{N} \right)^k \left(1 - \frac{G}{N} \right)^{N-k}$$

Here, G/N is the probability that a station attempts a transmission. This can be conceptually mapped to $P_k = \binom{N}{k} p^k q^{(N-k)}$. The throughput is therefore the probability that there is exactly *one* transmission attempt per slot ($k = 1$):

$$S = P_{k=1} = \binom{N}{1} \left(\frac{G}{N} \right) \left(1 - \frac{G}{N} \right)^{N-1} = G \left(1 - \frac{G}{N} \right)^{N-1}$$

Taking the limit of this formula as the number of stations goes to infinity yields $S = \lim_{N \rightarrow \infty} G \left(1 - \frac{G}{N} \right)^{N-1} = G e^{-G}$.

2.3 Calculating Maximum Throughput

To calculate the maximum throughput S_{max} and the optimal offered traffic, we compute the first derivative of the throughput with respect to the offered traffic (G) and set it equal to zero.

$$\frac{d(Ge^{-G})}{dG} = e^{-G} - Ge^{-G} = e^{-G}(1 - G) = 0 \rightarrow G = 1$$

Substituting $G = 1$ back into the throughput equation yields the absolute maximum throughput:

$$S_{max}|_{G=1} = Ge^{-G}|_{G=1} = \frac{1}{e} = 0.368$$

From this comparison, an improvement in performance of the slotted ALOHA protocol is observed, as it can achieve a maximum throughput that is *twice* that of the continuous ALOHA protocol. The value $G = 1$ represents the maximum load that can be offered to the network under stable conditions.

When analyzing finite stations, applying the same derivative logic to $S = G(1 - \frac{G}{N})^{N-1}$ also yields $G = 1$. Thus, $S_{max}(G = 1) = (1 - \frac{1}{N})^{N-1}$. It can be observed that the maximum throughput increases as the number of stations decreases, becoming equal to 1 for $N = 1$ since no collisions occur. Conversely, an increasing number of stations N represents the worst-case network performance.

3. Delay in Slotted-ALOHA

The average delay in slotted ALOHA is computed considering two factors:

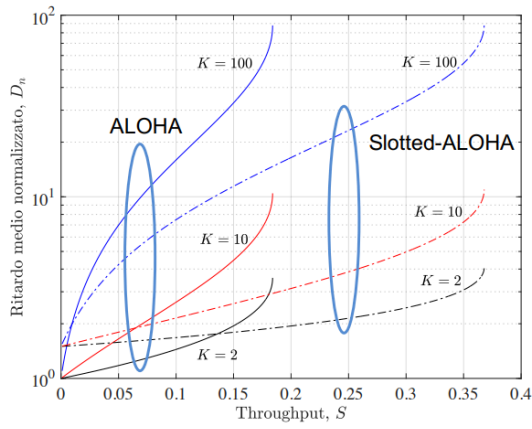
1. The first transmission of a UI is delayed on average by half a slot with respect to the arrival instant.
2. The average number of transmission attempts per UI successfully received at the destination is e^G .

The absolute delay D is:

$$D = (e^G - 1) \left(T + 2\tau + \frac{K-1}{2}T \right) + T + \tau + \frac{T}{2}$$

The normalized delay (D_n) is therefore given by:

$$D_n = \frac{D}{T} = (e^G - 1) \left(\frac{K+1}{2} + 2a \right) + 1.5 + a$$



When comparing the delay times of the two protocols in an ideal scenario ($a = 0$), it can be observed that slotted ALOHA is able to handle twice the traffic compared to unslotted ALOHA.

4. Stability of the ALOHA Protocol

ALOHA protocols are characterized by a throughput behavior such that there exists a specific level of offered traffic that determines the maximum throughput. Higher values of offered traffic lead to instability conditions that cause a collapse of network performance.

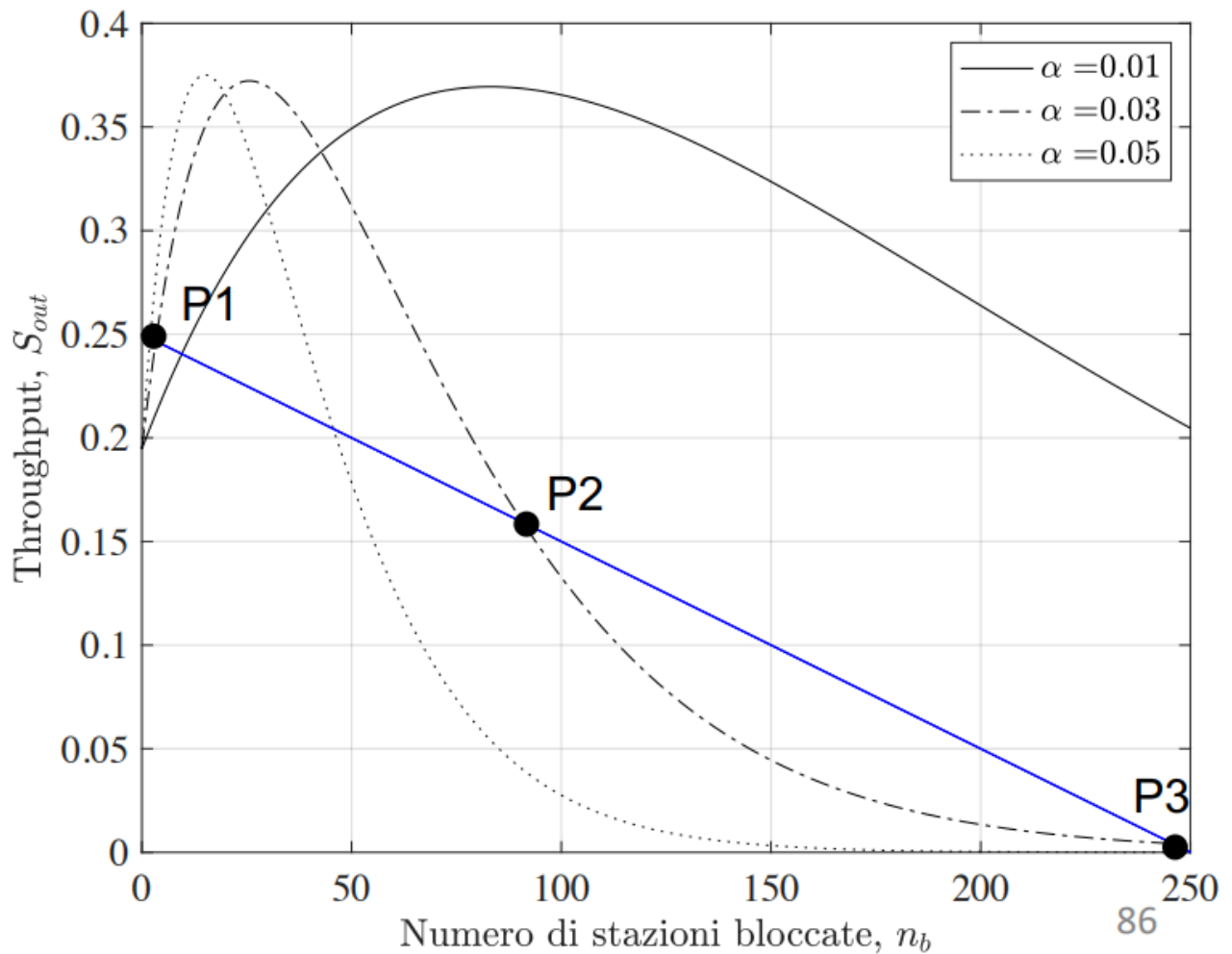
In a generic slot among N stations, there are n_b "blocked stations"—stations attempting to retransmit a UI whose previous transmissions resulted in a collision. The network throughput in state n_b can be expressed as the probability that a transmission in a generic slot is successful. This occurs when *only one* transmission attempt among the n_b blocked stations and the $N - n_b$ unblocked stations is successful.

Denoting p as the probability of arrival of a UI at an unblocked station, and α as the retransmission probability of the blocked stations, the output throughput (S_{out}) is:

$$S_{out} = n_b \alpha (1 - \alpha)^{n_b - 1} (1 - p)^{N - n_b} + (1 - \alpha)^{n_b} p (1 - p)^{N - n_b - 1}$$

To find network equilibrium, we plot the load offered to the network $S_{in} = (N - n_b)p$. Assuming an infinite number of stations N with an infinitesimal offered traffic p such that S_{in} remains constant, the equation simplifies to:

$$S_{out} = n_b \alpha (1 - \alpha)^{n_b - 1} e^{-S_{in}} + (1 - \alpha)^{n_b} S_{in} e^{-S_{in}}$$



The intersections of the S_{in} and S_{out} curves indicate the possible operating points:

- **P1 (Stable):** A fluctuation in S_{in} leads to an increase in S_{out} when S_{in} decreases due to an increase in the number of blocked stations.
- **P2 (Unstable):** A fluctuation in S_{in} leads to a reduction in S_{out} together with a reduction in S_{in} .
- **P3 (Stable, but low performance):** P3 is also a stable operating point, but unlike P1, it has a significantly lower throughput.

For $\alpha = 0.01$, the system is always stable since there is only one operating point. Increasing the value of α in this simplified model corresponds to increasing the parameter K in the real protocol, which determines the maximum retransmission delay. As α increases, a slight increase in throughput is obtained, but instability conditions still persist.

5. Carrier Sense Multiple Access (CSMA) Protocols

5.1 The "Listen Before You Talk" Principle

While ALOHA protocols are simple, they suffer from a major flaw: nodes make the decision to transmit entirely independent of the activity of other nodes on the broadcast channel. A node essentially acts like a rude person who continues to talk over others engaged in a conversation, completely ignoring ongoing transmissions or interference.

To mitigate this, **Carrier Sense Multiple Access (CSMA)** protocols introduce a "listen before you talk" mechanism. Before transmitting, a station "senses" (listens to) the channel.

- **Carrier Sensing:** If the channel is busy, the node waits until it detects that the channel is idle for a specific time interval before initiating transmission.
- **Collision Detection (CSMA/CD):** Used primarily in wired networks, the transmitting node simultaneously listens to the channel. If another node interferes, it halts transmission, waits a random interval, and retries.
- **Collision Avoidance (CSMA/CA):** Used primarily in wireless networks to proactively avoid collisions rather than just detecting them.

5.2 The Critical Role of Propagation Delay

The effectiveness of the Carrier Sense function is fundamentally tied to the network's propagation time (τ) relative to the transmission time (T). We define this relationship as a normalized ratio:

$$a = \frac{\tau}{T}$$

- **When CSMA is Highly Effective ($\tau \ll T$ or $a \ll 1$):** The time it takes for a signal to propagate across the network is tiny compared to the time it takes to transmit the packet. All nodes quickly realize the channel is busy, drastically reducing collisions.
- **When CSMA is Ineffective ($\tau > T$ or $a \geq 1$):** If the propagation delay is high (e.g., satellite networks), a transmission might be completely finished before the signal even reaches all other stations. In such expansive networks, nodes will falsely sense an "idle" channel and transmit, making Carrier Sense practically useless (which is why standard ALOHA doesn't use it).

5.3 The Vulnerability Period & Spacetime Collisions

Even with Carrier Sense, collisions are not entirely eliminated due to the propagation delay (τ), which represents the critical "vulnerability period."

Consider a bus topology network with 5 stations (A, B, C, D, E).

1. Station D begins transmitting a User Information (UI) packet at t_0 .
2. At t_1 , before D's signal has reached it, Station B senses the channel as "idle" and begins its own transmission.
3. At t_2 , D's signal physically reaches B, causing a collision.
4. The corrupted signal propagates outward. Stations A, B, and C receive a corrupted UI.
5. However, Station E might actually receive D's transmission correctly at t_4 , *provided* D finished its transmission at t_3 before any of B's interfering signal could reach E.

This spacetime diagram illustrates that detecting an idle channel does not guarantee another station hasn't already begun transmitting; it only guarantees their signal hasn't reached you *yet*.

6. CSMA Persistence Strategies

When a station has a UI to transmit but senses that the channel is currently busy, it must decide how to behave. This is defined by its "persistence" strategy:

- **Non-Persistent CSMA:** The station defers transmission to a later time, calculating a random delay just like in the ALOHA protocol (using τ as the base delay unit). It does not continuously monitor the channel while waiting. This is highly efficient under heavy traffic loads.
- **1-Persistent CSMA:** The station continuously senses the channel. The moment it detects the channel is idle, it transmits immediately with a probability of 1 (hence the name). While this minimizes idle time, it is disastrous under heavy loads: if multiple stations are waiting for a current transmission to end, they will *all* transmit simultaneously the moment it finishes, guaranteeing a collision.
- **p-Persistent CSMA:** The station continuously senses the channel. When idle, it transmits with a probability of p . If it decides not to transmit (probability $1 - p$), it waits for a time τ and repeats the process.
 - *Note:* At the limits, $p = 0$ degenerates into non-persistent behavior, and $p = 1$ becomes 1-persistent.

6.1 Slotted vs. Unslotted Variations

Just like ALOHA, CSMA can operate with or without a slotted time axis. In **Slotted CSMA**, the duration of a single time slot is equal to the end-to-end propagation time of the network (τ). All stations are synchronized, and the state of the channel is exclusively sensed at the *beginning* of a slot.

- In *Slotted 1-persistent*, the station waits for the first occurrence of an idle slot to transmit.
- In *Slotted non-persistent*, if busy, the station delays by an integer number of slots ($i\tau$) up to a maximum of $(K - 1)\tau$ before checking again.

7. Mathematical Analysis of CSMA Throughput

To evaluate the efficiency of Non-Persistent CSMA, we look at the channel as alternating between two states: **Busy** (signal propagating) and **Idle** (no signal). A "Cycle" is one busy period followed by one idle period.

The throughput (S) is calculated as the ratio of the time spent transmitting successfully without collisions (T_S) to the total cycle time (Busy Time T_O + Idle Time T_L):

$$S = \frac{T_S}{T_O + T_L}$$

(Note: The frequency of arriving UIs is denoted as $\Lambda = G/T$, where G is the offered traffic).

7.1 Unslotted Non-Persistent CSMA

- **Time Success (T_S):** A transmission is successful only if no other UI is generated in the first τ seconds of transmission. Assuming a Poisson distribution of arrivals, the average successful transmission time is the transmission time (T) multiplied by the probability of zero arrivals in τ seconds:

$$T_S = T \cdot Pr\{0 \text{ arrivals in } \tau\} = T \cdot e^{-\Lambda\tau} = Te^{-aG}$$

- **Time Idle (T_L):** The time between the end of a busy period and the next arrival.

$$T_L = \frac{1}{\Lambda} = \frac{T}{G}$$

- **Time Busy (T_O):** Determined by the last UI involved in a collision, which occurs at some time y (where $0 \leq y \leq \tau$) after the first transmission started. Calculating the expected value of y (Y) yields:

$$T_O = Y + T + \tau = T + 2\tau - \frac{T}{G}(1 - e^{-aG})$$

Combining these yields the final unslotted throughput formula:

$$S = \frac{Te^{-aG}}{T + 2\tau - \frac{T}{G}(1 - e^{-aG}) + \frac{T}{G}} = \frac{Ge^{-aG}}{G(1 + 2a) + e^{-aG}}$$

7.2 Slotted Non-Persistent CSMA

In the slotted version, all times are multiples of the base slot τ .

- **Time Success (T_S):**

$$T_S = T \cdot \frac{aGe^{-aG}}{1 - e^{-aG}}$$

- **Time Busy (T_O) & Time Idle (T_L):**

$$T_O = T + \tau$$

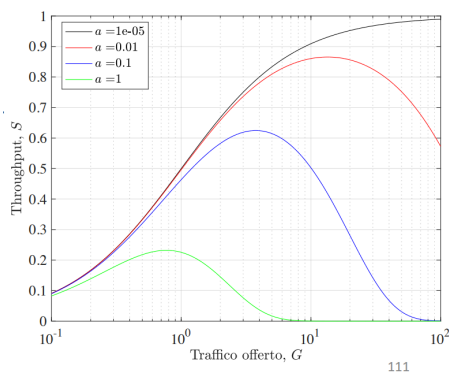
$$T_L = \frac{\tau e^{-aG}}{1 - e^{-aG}}$$

- **Final Slotted Throughput:**

$$S = \frac{aGe^{-aG}}{1 - e^{-aG} + a}$$

7.3 Throughput Comparison and Impact of Load

Analyzing the performance curves reveals critical behavioral differences based on traffic (G) and propagation delay (a):



- **Traffic Load (G):** 1-persistent CSMA offers the best performance when traffic is very low, as packets are transmitted immediately. However, its throughput collapses rapidly to near 0 as G increases due to guaranteed collisions. Conversely, non-persistent CSMA handles heavy loads excellently, capable of reaching a maximum throughput of ~90% (in the slotted version).

- **Propagation Delay (a):** The throughput S drastically increases as a decreases. If $a \rightarrow 0$ (meaning propagation time is effectively zero compared to transmission time), sensing is perfect, instability vanishes, and throughput approaches 1 (100% efficiency) at high traffic. If $a = 1$, the propagation time equals transmission time, crippling the efficiency of the "sensing" mechanism.

8. CSMA/CD (Collision Detection) Protocol

8.1 The "Listen While You Talk" Principle

The Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol is the direct evolution of standard CSMA, historically used primarily in wired Local Area Networks (LANs like Ethernet).

Its fundamental rule adds one critical step: *Listen before you talk, and listen WHILE you talk.*

In standard CSMA, if a collision occurs, the transmitting stations are unaware and continue sending their entire packet, wasting valuable channel time. In CSMA/CD, stations monitor the channel during their own transmission. If a collision is detected, the transmission is aborted immediately. The station then broadcasts a short **jamming signal** (γ) to the entire network. This signal explicitly informs all other stations to discard the corrupted portion of the User Information (UI) they just received.

Like standard CSMA, CSMA/CD has Slotted, Non-slotted, Non-persistent, 1-persistent, and p-persistent variants.

8.2 Spacetime Collision Detection and Efficiency

Let's revisit the spacetime collision example with stations A, B, C, D, and E on a bus topology:

1. Station D begins transmitting at t_0 .
2. Station B falsely senses an idle channel at t_1 and transmits.
3. At t_2 , D's signal reaches B, and a collision occurs.
4. **The CSMA/CD Difference:** Instead of continuing, Station B detects the collision and abruptly halts transmission at t_3 .

By terminating the compromised transmission early, Station B frees up the channel much faster (e.g., at t_6 and t_7 for distant stations D and E) than it would have under standard CSMA.

The duration of a collision interval in CSMA/CD is denoted as $\gamma + 1$ **slots**, where γ is a pre-set parameter (typically 1 or 2 slots) representing the jam signal propagation. Because this collision interval is so much smaller than a full transmission time ($T + 1$), CSMA/CD significantly reduces wasted bandwidth, especially in the 1-persistent version.

8.3 Mathematical Performance of CSMA/CD

The efficiency (η) of CSMA/CD is the ratio between the time required for a successful transmission and the total time spent (including the time wasted on collision intervals).

We define a base time unit of 2τ . Therefore, the transmission time of a UI is $\frac{T}{2\tau}$.

1. Probability of Success (A):

A transmission succeeds if exactly one station receives a UI to transmit in a 2τ interval. Let p be the probability a station receives a UI. Using the binomial distribution ($k = 1$), the success probability A for N stations is:

$$A = \binom{N}{1} p(1-p)^{N-1} = Np(1-p)^{N-1}$$

Setting the derivative to zero yields a maximum success probability when $p = 1/N$:

$$A_{max} = \left(1 - \frac{1}{N}\right)^{N-1}$$

2. Number of Collision Intervals (J):

Assuming collision events are independent, the number of collision intervals before a success follows a geometric distribution. The average number of collision intervals J is:

$$J = \sum_{i=1}^{\infty} iA(1-A)^{i-1} = \frac{1}{A}$$

3. Maximum Throughput (η):

The general throughput equation is:

$$\eta = \frac{T}{T + \tau + 2\tau \cdot J} = \frac{1}{1 + a(1 + 2J)}$$

As the number of stations approaches infinity ($N \rightarrow \infty$), the maximum probability of success A_{max} approaches $1/e$. Consequently, $J = 1/A = e \approx 2.72$. Substituting this gives the absolute maximum throughput formula:

$$\eta = \frac{1}{1 + a(1 + 2e)} \approx \frac{1}{1 + 6.44a}$$

Performance Takeaways:

- If propagation time is 2 to 3 orders of magnitude smaller than transmission time ($a = 0.01$ or $a = 0.001$), network efficiency easily exceeds 95% to 99%.
- As a approaches 1 (propagation time equals transmission time), efficiency plummets below 25%.
- The slotted non-persistent CSMA/CD version yields the best overall performance curves under heavy traffic.

9. CSMA/CA (Collision Avoidance) and WLANs

9.1 The Hidden Station Problem

CSMA/CD is highly effective for wired LANs, but it fails in Wireless LANs (WLANs). In a wireless medium (the ether), signal power drops significantly with distance, making reliable collision *detection* impossible.

This creates the **Hidden Station Problem**:

Imagine three wireless laptops: A, B, and C. Station A is transmitting to Station B. Station C is close to B, but out of range of A. Station C senses the channel,

hears nothing (because it cannot hear A), and begins transmitting to B. C's transmission collides with A's at receiver B. A and C are "hidden" from each other.

9.2 The CSMA/CA Solution

To fix this, WLANs use **Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA)**. Instead of detecting collisions, the protocol attempts to prevent them through an explicit authorization handshake and control frames:

- **RTS (Request to Send):** A sender asks permission to transmit.
- **CTS (Clear to Send):** The receiver grants permission.
- **ACK (Acknowledgment):** Because collisions can still technically happen, explicit ACKs are required for all successful UI data frames.

9.3 Timers: DIFS and SIFS

CSMA/CA utilizes specific interframe spaces (wait times) to prioritize network traffic:

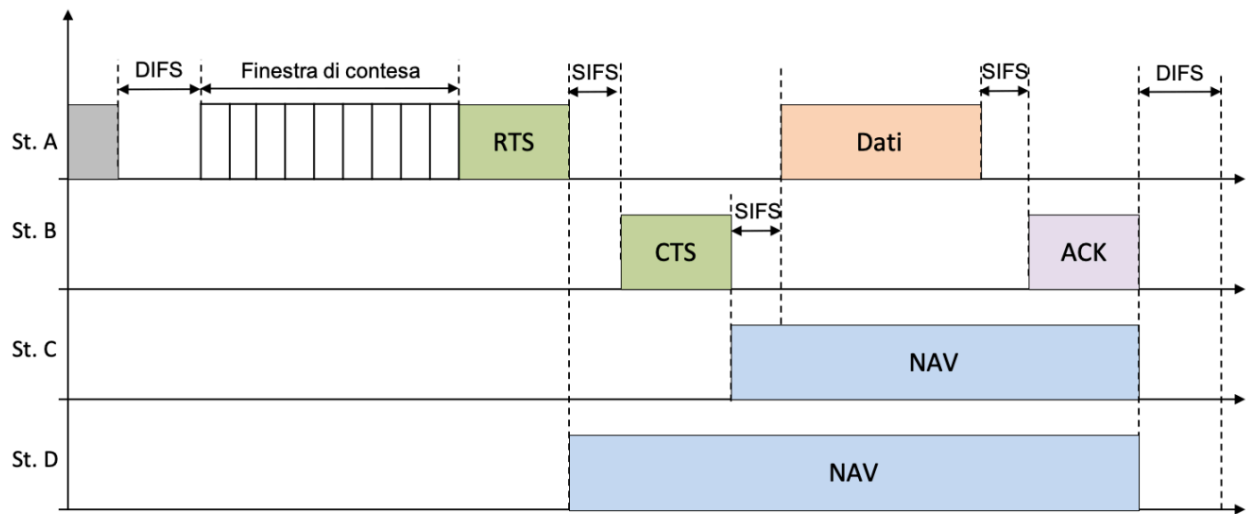
- **DIFS (DCF Interframe Space):** The standard wait time. A station must sense the channel as idle for a full DIFS period before it can attempt to send an RTS or standard data.
- **SIFS (Short Interframe Space):** A shorter wait time used exclusively for high-priority control frames (CTS and ACKs) and subsequent data fragments.

Because $\text{SIFS} < \text{DIFS}$, a station waiting to send an ACK will always seize the channel before a new station waiting a DIFS can send an RTS.

9.4 Network Allocation Vector (NAV)

To solve the hidden station problem, the RTS and CTS frames contain a duration field. This tells everyone listening exactly how long the channel will be occupied (including the data frame, ACKs, and SIFS intervals).

Stations that overhear the RTS or CTS update their internal timer, called the **Network Allocation Vector (NAV)**. For the entire duration of the NAV, these stations remain completely inactive, treating the channel as busy even if they cannot physically hear the radio waves.



9.5 The CSMA/CA Backoff Algorithm

If a station wants to transmit but senses the channel is busy, it applies a backoff algorithm:

1. It waits for the channel to become idle, plus one DIFS interval.
2. It generates a random backoff time, calculated as a random number of slots between 0 and $K - 1$.
3. It begins a slot countdown. **Crucially, the countdown only decreases when the medium is idle.** If another transmission starts, the countdown pauses and only resumes after the channel is idle again for a DIFS.
4. When the countdown hits zero, it transmits the RTS.

Calculating K (The Contention Window):

The maximum window size K grows exponentially with each unsuccessful attempt (i), calculated as:

$$K = 2^{2+i} - 1$$

For the very first attempt ($i = 0$), $K = 2^{2+0} - 1 = 3$ (Wait, the notes explicitly state: "*The initial value is $K = 7$, so that the initial delay is a number of slots between 0 and 7.*" — Note: To yield $K = 7$, the first attempt index in the provided formula must be interpreted as $i = 1$, meaning $K = 2^{2+1} - 1 = 7$). If an RTS collision occurs (e.g., no CTS received within SIFS), or an ACK is not received, i increments, the contention window doubles, and the station tries again.

(Note: Data frame fragmentation is supported; only the first fragment requires a DIFS wait and backoff, while subsequent fragments are sent sequentially separated only by SIFS intervals).

CHAPTER 2

1. Introduction and Network Elements

A wireless network is composed of several fundamental elements. The primary users are **wireless hosts**, which can range from laptops and smartphones to connected IoT devices. These hosts are responsible for running network applications. It is important to note that the term "wireless" does not exclusively imply mobility; these hosts may be highly mobile or entirely stationary.

The second core element is the **base station**. Base stations, such as 802.11 access points (APs) or cellular cell towers, act as vital relays that connect the wireless network to a broader wired network infrastructure. Their primary responsibility is to send, receive, and route packets between the wired network and the wireless hosts located within their specific coverage area.

These elements are connected via a **wireless link**. This link is typically used to connect mobile devices to a base station, though it can also function as a backbone link bridging wider areas. Access to this shared link is strictly coordinated by a multiple access protocol. Wireless links are highly variable and are characterized by different transmission rates, operating distances, and frequency bands.

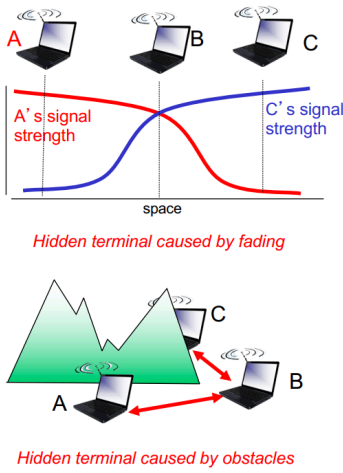
2. Wireless Link Characteristics and Challenges

Wireless networks can be categorized by their environmental range and link capabilities:

- **Indoor (10 to 30 meters):** Common standards include Bluetooth at 2 Mbps, 802.11b at 11 Mbps, 802.11g at 54 Mbps, 802.11n at 600 Mbps, 802.11ac at 3.5 Gbps, and 802.11ax at 14 Gbps.
- **Outdoor (50 to 200 meters):** Standards like 802.11 af and 802.11 ah provide speeds up to 600 Mbps.
- **Midrange Outdoor (200 meters to 4 kilometers):** These distances are often handled by 5G networks operating at up to 10 Gbps.
- **Long-range Outdoor (4 to 15 kilometers):** Broad outdoor links spanning miles generally rely on 4G LTE technology.

Wireless links present significant challenges compared to traditional wired point-to-point connections, making communication inherently more difficult. Three main

issues degrade performance:

 Hidden terminal caused by fading Hidden terminal caused by obstacles	1. Signal Attenuation (Path Loss): Radio signals naturally decrease in strength as they propagate through matter and empty space. 2. Interference: Frequencies like the 2.4 GHz band are shared among numerous everyday sources—including Wi-Fi nodes, cellular devices, and even electric motors—causing environmental interference. 3. Multipath Propagation: Radio signals reflect off the ground and other objects, causing different portions of the signal to arrive at the destination at slightly different times, muddying the overall reception.
--	---

Because of these issues, managing the **Signal-to-Noise Ratio (SNR)** and **Bit Error Rate (BER)** is critical. A larger SNR is highly desirable because it makes it much easier for the receiver to extract the intended signal from background noise. When evaluating a specific physical layer modulation scheme, increasing the transmission power will naturally increase the SNR, which consequently decreases the BER. Conversely, if the SNR is fixed by environmental or hardware constraints, network designers must choose a modulation technique that safely meets the required BER while yielding the highest possible data rate.

For example, higher-order modulations like QAM256 can transmit at 8 Mbps but require a significantly higher SNR to maintain stability compared to simpler, slower modulations like QAM16 (4 Mbps) or BPSK (1 Mbps). Because mobility constantly alters the SNR, wireless systems must dynamically adapt their physical layer, shifting between different modulation techniques and transmission rates on the fly to prevent data loss.

3. Network Taxonomy and Operating Modes

Wireless networks generally operate in one of two fundamental configurations based on the presence or absence of infrastructure:

In **Infrastructure Mode**, a base station (such as an AP) connects mobile devices directly into the wired network. When a mobile device moves out of range of its

current base station, it performs a handoff to transfer its connection to a new base station, maintaining its link to the broader wired infrastructure.

In contrast, **Ad-hoc Mode** functions completely without base stations. Nodes organize themselves autonomously into a decentralized network, independently routing data among themselves. In this setup, a node can only transmit directly to other nodes that fall within its immediate link coverage.

This infrastructure distinction, combined with hop count, creates a four-part taxonomy for wireless networks:

- **Single-hop, Infrastructure:** A host connects directly to a base station (e.g., Wi-Fi or cellular), which then connects to the larger Internet.
 - **Single-hop, No Infrastructure:** Hosts connect directly to each other without a base station or Internet connection, such as in Bluetooth or basic ad-hoc networks.
 - **Multiple-hop, Infrastructure:** A host connects to the broader Internet but must relay its data through several intermediate wireless nodes to reach the base station, characteristic of wireless mesh networks.
 - **Multiple-hop, No Infrastructure:** There is no base station or Internet connection, and nodes must relay messages through each other across vast distances to communicate, as seen in Mobile Ad-hoc Networks (MANETs) and Vehicular Ad-hoc Networks (VANETs).
-

4. The Hidden Terminal Problem

Having multiple wireless senders and receivers operating simultaneously introduces severe complications beyond standard multiple-access coordination. The most notable is the **Hidden Terminal Problem**, which occurs when two nodes are unaware that their simultaneous transmissions are colliding at a mutual receiver.

This problem arises primarily from two causes. The first is **signal fading (attenuation)**. If node A and node C are positioned far apart with receiver node B directly between them, A's signal strength may decay before it reaches C, and C's signal strength may decay before it reaches A. A and C cannot hear each other, meaning they are unaware of each other's presence and might transmit simultaneously, causing undetected interference (a collision) exactly at receiver B.

The second cause is physical **obstacles**. For instance, if nodes A and C are separated by a large mountain but are both trying to communicate with node B located in the valley between them, they will remain "hidden" from one another and accidentally jam B's reception.

5. Summary of IEEE 802.11 Versions

The IEEE 802.11 standard has evolved significantly over the years to combat these physical challenges, offering faster rates and better modulation techniques. Every version of the 802.11 standard fundamentally supports both infrastructure and ad-hoc operating modes, and relies heavily on the CSMA/CA protocol for multiple access coordination.

The capabilities of the major 802.11 standards are as follows:

- **802.11b (1999):** Operates on the 2.4 GHz frequency with a maximum data rate of 11 Mbps and a range of 30 meters.
- **802.11g (2003):** Retains the 2.4 GHz band and 30-meter range but increases the data rate to 54 Mbps.
- **802.11n / WiFi 4 (2009):** Introduces dual-band support (2.4 and 5 GHz), extending the range to 70 meters and pushing data rates to 600 Mbps.
- **802.11ac / WiFi 5 (2013):** Operates exclusively on the 5 GHz band, maintaining a 70-meter range but significantly boosting maximum speeds to 3.47 Gbps.
- **802.11ax / WiFi 6 (2020):** Utilizes both 2.4 and 5 GHz bands over a 70-meter range, achieving massive maximum data rates of 14 Gbps.
- **802.11af (2014):** A specialized standard utilizing unused TV bands (54-790 MHz), allowing for a 1-kilometer range at speeds of 35 to 560 Mbps.
- **802.11ah (2017):** Operates on the lower 900 MHz frequency, delivering 347 Mbps over an expansive 1-kilometer range.

6. Network Structure: BSS and ESS

To organize communication effectively, the IEEE 802.11 standard defines two specific station configurations, classifying networks into basic and extended radio mobile services.

The fundamental building block is the **Basic Service Set (BSS)**, also referred to as a "cell." Within a BSS, wireless stations can communicate with each other

without restriction. When operating in infrastructure mode, the BSS contains a central base station known as an **Access Point (AP)** alongside its connected wireless hosts. This AP acts as a bridge, enabling communication between the mobile stations and other remote network devices. A BSS can operate completely isolated, or it can be connected to a broader system. If the AP is entirely absent, the network defaults to ad-hoc mode, consisting solely of independent hosts.

To provide broader coverage, multiple BSSs can be linked together. This larger configuration is called an **Extended Service Set (ESS)**. An ESS is achieved by interconnecting two or more APs through a **Distribution System (DS)**.

[Photo: Extended Service Set Architecture]

Visualizing this architecture, imagine a central Internet cloud connected to a main switch or router. This central router acts as the Distribution System (DS), which is typically a wired backbone LAN. From this router, physical wired links extend to multiple distinct Access Points (APs), each anchoring its own BSS (e.g., BSS 1, BSS 2, BSS 3). Surrounding each AP is a coverage area (cell) containing various mobile wireless hosts (laptops, smartphones) communicating directly with their respective AP. The DS bridges these distinct cells, creating one cohesive Extended Service Set.

7. Transmission Errors and Fragmentation

The shared nature of the wireless transmission medium demands the intensive use of the electromagnetic spectrum. Consequently, modern networks adopt increasingly higher, previously unused frequencies combined with highly complex modulation techniques. While these advancements increase speed, they also introduce a significant vulnerability: interference with everyday devices.

Because of this, the error rate in wireless transmissions is substantially higher than in wired networks. To mitigate data loss, wireless protocols dictate that data frames exceeding a certain length threshold must be transmitted as a sequence of consecutive, shorter fragments. Losing a shorter frame to transmission errors limits the amount of user information that must be retransmitted, preserving overall network efficiency.

This necessity can be mathematically modeled. Assume a constant bit error probability p during transmission. A frame is only successfully transmitted if *all* of

its bits are correctly received. Therefore, the probability P that a frame of length L (in bits) is discarded due to errors is calculated as:

$$P = 1 - (1 - p)^L$$

Consider a standard frame length $L = 1518$ bytes. If the network experiences a high bit error rate of $p = 10^{-4}$, the transmission failure probability P spikes to 0.703 (meaning over 70% of frames would need retransmission). Conversely, if the channel quality is better, with a lower error rate of $p = 10^{-6}$, the failure probability drops dramatically to $P = 0.012$ (roughly 1% retransmission). Shortening the frame length significantly reduces these failure percentages, heavily optimizing channel usage.

8. Access Protocols: DCF and PCF

To access the shared radio channel, the IEEE 802.11 standard defines two distinct coordination functions:

1. Distributed Coordination Function (DCF):

This is the default channel access procedure and is constantly available to all network stations. DCF defines the basic medium access mode based on the CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance) protocol. Normal traffic relies directly on DCF. Stations engage in "distributed contention" by actively listening to the channel and utilizing mechanisms like DIFS, backoff timers, and RTS/CTS exchanges to avoid collisions. Because it is decentralized, DCF is ideal for ad-hoc networks and standard best-effort traffic.

2. Point Coordination Function (PCF):

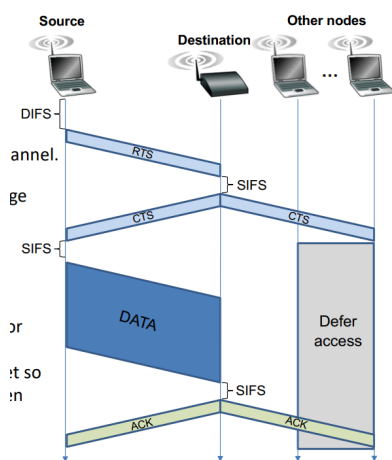
PCF is an additional, optional procedure used exclusively by an Access Point to centrally coordinate access to the transmission medium, entirely preventing possible collisions. PCF is tailored to support real-time traffic. It relies on a "polling" system centrally managed by a Point Coordinator located in the AP, providing a strict contention-free channel access service. Crucially, PCF is built on top of DCF and exploits its foundational functionalities to provide this organized access.

Both DCF and PCF can operate simultaneously within the exact same BSS, providing alternating contention periods (for DCF) and contention-free periods (for PCF). Regardless of the mode, network time is divided into intervals called

slots. A slot is the system's fundamental time unit, and its duration is dictated by the physical layer implementation (for instance, in IEEE 802.11b, a slot is 20µs). Furthermore, stations maintain synchronization through the periodic transmission of **beacon frames**. In infrastructure mode, these are broadcast by the AP; in ad-hoc mode, the stations coordinate these beacons amongst themselves.

9. CSMA/CA and Collision Avoidance (RTS/CTS)

Because DCF relies on CSMA/CA, a strict procedure is followed to prevent the "hidden terminal" collisions mentioned earlier.



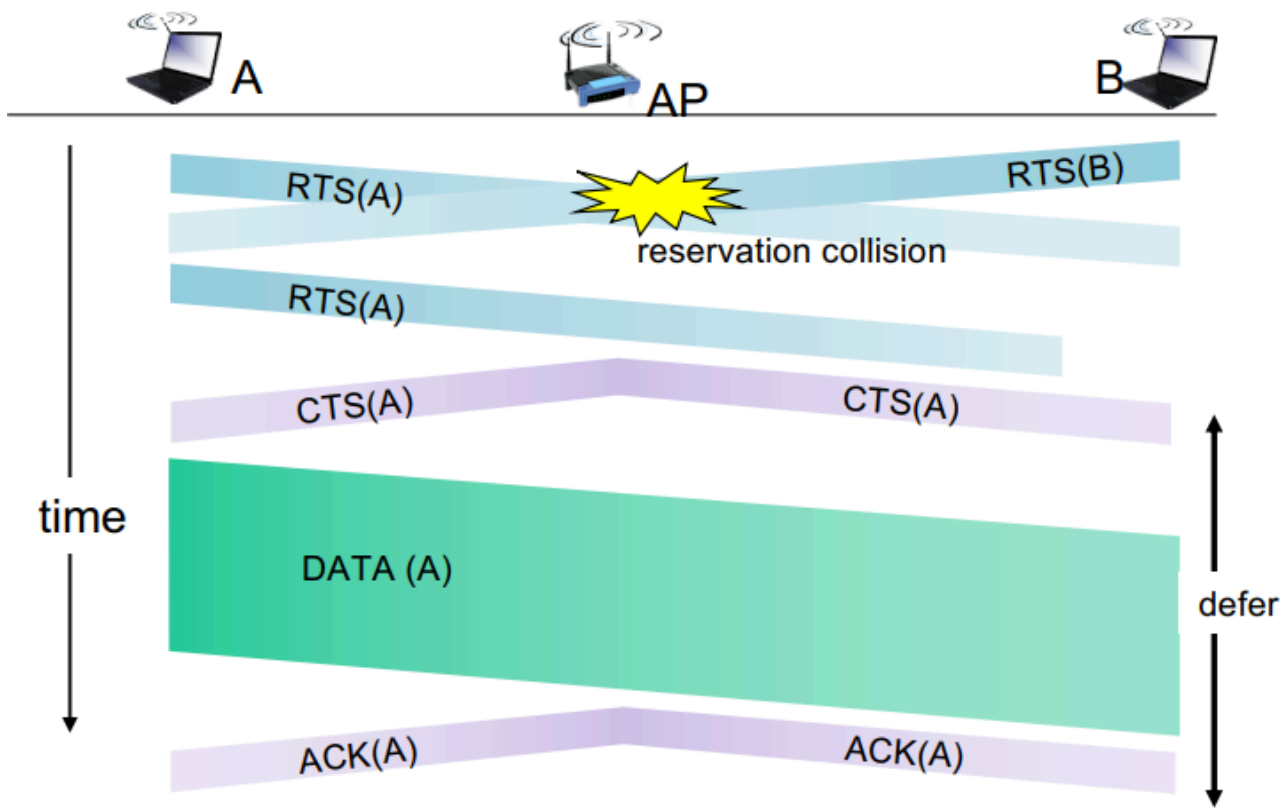
This RTS frame contains a crucial piece of information: the **Network Allocation Vector (NAV)**, which explicitly states the expected time interval required to complete the entire data transmission.

Upon successfully receiving the RTS without collision, Station B replies with a broadcast **Clear To Send (CTS)** frame. This CTS frame repeats the exact same NAV time interval announced by the sender. Once

Station A successfully receives this CTS frame, it finally proceeds to transmit its actual **DATA** frame. Once Station B receives the DATA frame without errors, it immediately returns an **ACK (Acknowledgment)** frame to Station A to confirm successful delivery.

The Power of the NAV: Both the RTS and CTS frames are broadcast to the entire network. This is how the protocol solves the hidden terminal problem:

- If a third node hears the **CTS frame**, it knows it is physically close to the receiver (Station B). Therefore, it defers all access and remains silent for the entire NAV duration specified to protect the receiver from interference.
- If a third node hears the **RTS frame** but *does not* hear the CTS frame, it deduces that it is close to the sender but too far from the receiver to cause interference. Thus, it is theoretically free to transmit without disrupting Station B.



Nodes in 802.11 do not detect collisions directly like wired Ethernet does. Instead, they infer collisions. If two nodes broadcast an RTS frame simultaneously, those frames collide. The sending nodes will simply wait; if they do not receive a CTS response, they assume an RTS collision occurred and will trigger a backoff algorithm before trying again. The RTS/CTS exchange introduces a slight delay and consumes minor channel resources, so it is strictly reserved for reserving the channel for *long* data frames. An internal RTS threshold ensures this handshake is skipped for tiny packets.

10. IEEE 802.11 Timing Intervals (IFS)

The standard cleverly prioritizes access to the transmission medium by differentiating the waiting times before a station is allowed to speak. These minimum mandatory waiting intervals between consecutive frame transmissions are called **Interframe Spaces (IFS)**.

1. **Short Interframe Space (SIFS):** The shortest mandatory waiting time. Because it is the shortest, it grants the absolute highest priority. It is used exclusively to prevent collisions during active, reserved conversations. Stations wait only a SIFS before transmitting high-priority control frames like CTS,

ACK, responses to AP PCF queries, the first permitted data frame in a sequence, and any subsequent data fragments in a fragmented transmission.

2. **PCF Interframe Space (PIFS):** An intermediate time interval. Because it is shorter than standard traffic waiting times but longer than SIFS, it allows the AP operating in PCF mode to seize control of the channel and enforce a contention-free polling period before standard devices can attempt to transmit.
3. **DCF Interframe Space (DIFS):** The longest interval. This is the standard waiting time used by all regular stations operating in DCF mode. A station must wait a full DIFS before attempting to transmit the first fragment of a standard data frame or management frame.

If a station waits a full DIFS and finds the channel is still busy, it enters a **contention window**. This window is a randomized backoff delay time defined as a multiple of the base "slot time" (τ). If τ represents the propagation time between the two most distant stations in the network, the minimum durations scale accordingly: SIFS is exactly τ , PIFS is 2τ , and DIFS is 3τ .

11. Network Association and Frequency Channels

Due to the inherently mobile nature of wireless terminals, a device must formally join a specific Basic Service Set (BSS) through an association procedure before it can send or receive any data. When an Access Point (AP) is installed, it is assigned a Service Set Identifier (SSID) and a specific operating channel number.

The classic IEEE 802.11 standard operates in the 2.4 GHz to 2.485 GHz frequency range, occupying an 85 MHz band. Within this specific band, the spectrum is divided into 11 partially overlapping channels. A critical rule of network design is that a pair of channels is only considered "non-overlapping" if they are separated by 4 or more channels. Consequently, the only set of three completely non-overlapping channels available in the 2.4 GHz band is the combination of channels 1, 6, and 11.

12. Finding a Network: Passive vs. Active Scanning

To associate with an AP, a station must first discover it. This is achieved through one of two scanning methods:

Passive Scanning: In this method, the AP takes the active role by periodically broadcasting a **beacon frame**, which contains its SSID and MAC address. A station wishing to access the network passively scans all 11 channels, listening for these beacons. [Photo: Passive Scanning Sequence] The visual sequence of passive scanning is straightforward: First, AP 1 and AP 2 broadcast their beacon frames. Second, the mobile host (H1) receives these beacons and selects an AP—typically the one with the highest received signal strength. Third, H1 sends an Association Request frame to the chosen AP. Finally, the selected AP replies with an Association Response frame, completing the connection.

Active Scanning: In contrast, active scanning places the burden of discovery on the wireless device itself. The device broadcasts a **probe request** packet to all APs within its transmission range. [Photo: Active Scanning Sequence] The sequence shifts: First, the host (H1) broadcasts a Probe Request frame. Second, any AP that receives this probe (e.g., AP 1 and AP 2) replies directly to H1 with a Probe Response frame. Third, H1 evaluates the responses, chooses an AP, and sends a targeted Association Request frame. Fourth, the chosen AP finalizes the link with an Association Response frame.

Once successfully associated via either method, the station typically sends a DHCP discovery message to the network to obtain a valid IP address.

13. The IEEE 802.11 Frame Format

The 802.11 protocol defines three primary types of frames:

1. **Management frames:** Used to perform administrative functions like authentication, station association with the AP, and network synchronization.
2. **Control frames:** Used to coordinate the transfer of data and prevent collisions, which include the previously discussed RTS, CTS, and ACK frames.
3. **Data frames:** The frames that actually carry the user data.

The MAC frame itself is structured sequentially with the following fields and byte lengths:

- **Frame Control (2 bytes):** Contains flags and type indicators.
- **Duration (2 bytes):** Indicates the reserved transmission time (the NAV).
- **Address 1 (6 bytes), Address 2 (6 bytes), Address 3 (6 bytes):** MAC addresses used for routing the frame.

- **Sequence Control (2 bytes):** Used for reliable data transfer and fragmentation.
 - **Address 4 (6 bytes):** A fourth MAC address used only in specific ad-hoc bridging scenarios.
 - **Payload (0-2312 bytes):** Contains the data to be transmitted. While it can technically reach 2312 bytes, it is typically a standard 1500-byte IP datagram or an ARP packet (for CTS/RTS/ACK frames).
 - **CRC (4 bytes / 32 bits):** A Cyclic Redundancy Check used by the receiver to detect any bit errors that occurred during wireless transmission.
-

14. Deep Dive: Frame Control and Sequence Fields

The 2-byte **Frame Control** field acts as the instruction manual for how the rest of the frame should be read. It is divided into several specific bit-flags:

- **Protocol version (2 bits):** Specifies the standard version.
- **Type (2 bits):** Identifies the frame category: **00** for management, **01** for control, and **10** for data.
- **Subtype (4 bits):** Further specifies the exact frame (e.g., distinguishing an RTS control frame from an ACK control frame).
- **To AP (1 bit) & From AP (1 bit):** These two bits are critical; their combination determines exactly how the four address fields are interpreted.
- **More frag (1 bit):** Signals to the destination that more fragments of this specific frame are queued to arrive.
- **Retry (1 bit):** Indicates that this frame is a retransmission of a previously lost frame.
- **Power mgt (1 bit):** Indicates if the transmitting station is active or entering power-saving mode.
- **More data (1 bit):** Used by the AP to inform a sleeping station that additional frames addressed to it are currently buffered and waiting.
- **WEP (1 bit):** Indicates if the frame payload has been encrypted using the Wired Equivalent Privacy (1997) cryptographic algorithm.
- **Rsvd (1 bit):** Reserved for future protocol use.

The **Sequence Control** field (2 bytes) is essential because wireless ACKs can easily be lost. Without sequence numbers, a station resending a frame might cause

the receiver to process a duplicate packet. The first 4 bits indicate the fragment number, while the remaining 12 bits provide a sequence number (from 0 to 4095) identifying the specific frame exchanged between the stations.

Additionally, the **Duration/ID** field (2 bytes) usually holds the NAV value expressed in microseconds. However, if the network is operating under the PCF (polling) mode, this field changes purpose to indicate the identifier of the source station responding to the AP's poll.

15. The Four Address Fields and Internetworking

Because a wireless host must often communicate through an AP to reach the wider Internet, a standard source/destination address pair is insufficient. The 802.11 frame utilizes up to four 48-bit MAC addresses depending on the configuration of the **To AP** and **From AP** bits in the Frame Control field. Address 1 *always* identifies the physical receiver of the radio transmission, and Address 2 *always* identifies the physical transmitter.

The Address Matrix:

- **Case A (To AP=0, From AP=0): Within the BSS.** Two mobile stations are communicating directly. Address 1 is the Destination, Address 2 is the Source, and Address 3 holds the BSS identifier.
- **Case B (To AP=0, From AP=1): Entering the BSS.** A frame arriving from the wired Distribution System. Address 1 is the Destination host, Address 2 is the Transmitting AP, and Address 3 is the original Source address.
- **Case C (To AP=1, From AP=0): Leaving the BSS.** A host sending data out to the wired network. Address 1 is the Receiving AP, Address 2 is the Source host, and Address 3 is the final Destination address.
- **Case D (To AP=1, From AP=1): From BSS to BSS (Ad-Hoc/Mesh).** Address 1 is the Receiving AP, Address 2 is the Transmitting AP, Address 3 is the Destination, and Address 4 is the Source.

Internetworking Example: Consider a router (R1) connected to the Internet, hardwired to an AP, which is wirelessly serving a host (H1). The AP is strictly a physical/data-link layer device; it does not understand or process IP packets. When H1 wants to send a datagram to R1, H1 constructs an 802.11 frame (leaving the BSS - Case C). It sets Address 1 to the AP's MAC, Address 2 to its own MAC, and Address 3 to the Router's MAC. When the AP receives this, it strips

away the wireless headers and converts it into a standard 802.3 Ethernet frame to send over the wire to the router, setting the Ethernet Source to H1's MAC and the Destination to R1's MAC. *Address 3 is crucial here: it is the only way the AP knows the final intended destination MAC address on the wired network.*

16. Layer 2 Mobility Across BSSs

When a wireless host (H1) physically moves out of range of its current AP (BSS 1) and into the range of a new AP (BSS 2), a handoff occurs.

If these two APs are interconnected by a standard Layer 2 Switch, both APs and all their connected stations belong to the exact same IP subnet. Because the subnet hasn't changed, H1 can seamlessly maintain its current IP address and keep all of its active TCP connections alive without interruption.

However, if the device interconnecting the two APs was a Router (a Layer 3 device), BSS 1 and BSS 2 would be on completely different IP subnets. Moving between them would force H1 to request a brand new IP address, which would instantly sever and reset any active TCP connections.

17. The Physical Layer (PHY) Architecture

While the MAC layer handles addressing and medium access, the IEEE 802.11 physical layer (PHY) is responsible for the actual characteristics of the transmission medium and how it is utilized. The frequency bands initially designated for IEEE 802.11 fall within the unlicensed ISM (Industrial, Scientific, and Medical) spectrum, specifically the 2.4 GHz and 5.8 GHz bands.

To successfully transmit the MAC-layer frame over the air, the PHY layer encapsulates it within an additional information unit.

[Photo: IEEE 802.11 Physical Layer Encapsulation] This diagram illustrates the encapsulation process. The MAC header, MAC payload, and Frame Check Sequence (FCS) are grouped together to form the "PHY payload." Before this payload, the physical layer prepends its own specific information: a PHY preamble followed by a PHY header. Together, these elements make up the complete transmitted radio frame.

The prepended components serve critical physical functions:

- **PHY Preamble:** This alerts the network that a transmission is starting and performs crucial receiver synchronization.
- **PHY Header:** This field, which is always 6 bytes long, provides error-control functionality and explicitly specifies the parameters used for the current transmission, such as the payload length and the specific transmission frequency.

Because modern networks dynamically adapt to changing interference and signal quality, the PHY header must allow devices to seamlessly switch between different transmission techniques on the fly.

18. Evolution of the IEEE 802.11 Standards

The 802.11 standard has seen continuous upgrades to modulation schemes, channel bandwidths, and antenna technologies to deliver faster, more reliable connections.

IEEE 802.11b

This was the first standard to be widely accepted and commercialized. It operates entirely within the 2.4 GHz ISM band using DSSS (Direct Sequence Spread Spectrum) modulation. It defines four specific data rates: 1, 2, 5.5, and 11 Mbit/s, dynamically adapting the rate based on the detected channel quality. To achieve these rates over standard 20 MHz radio channels, it uses different modulation schemes: BPSK (Binary Phase Shift Keying) and QPSK (Quadrature Phase Shift Keying) for the lower speeds, and CCK (Complementary Code Keying) for the higher speeds. To optimize transmission efficiency based on the modulation used, the 802.11b preamble length is dynamic: it uses a 144-bit preamble when transmitting with DSSS, and a shorter 72-bit preamble when utilizing CCK.

IEEE 802.11a

(Note: Identified in some legacy documentation texts with typographic errors as "802.119") This standard shifted to a different ISM band, operating around 5 GHz. It introduced a highly efficient multicarrier transmission technique called OFDM (Orthogonal Frequency Division Multiplexing) over 20 MHz radio channels, incorporating Forward Error Correction (FEC). It defines several channel data rates starting from 6 Mbit/s and reaching up to 54 Mbit/s. Speeds up to 18 Mbit/s are achieved using PSK (Phase Shift Keying), while higher speeds rely on QAM (Quadrature Amplitude Modulation). Because it uses the higher 5 GHz

frequency, its coverage range is physically smaller than that of the 2.4 GHz 802.11b standard.

IEEE 802.11g

The 802.11g standard brought the benefits of OFDM transmission back to the 2.4 GHz ISM band. Operating on the same frequency made it backward compatible with older 802.11b devices. Like 802.11a, it uses 20 MHz radio channels and reaches up to 54 Mbit/s, utilizing PSK (BPSK, QPSK) and QAM (16QAM, 64QAM) modulation schemes.

IEEE 802.11n (Wi-Fi 4)

This standard represented a massive leap forward by introducing **MIMO (Multiple-Input Multiple-Output)** technology. MIMO uses multiple antennas to transmit up to four parallel data streams simultaneously in the downlink direction. Furthermore, it doubled the radio-channel bandwidth from 20 MHz to 40 MHz, vastly increasing both transmission and reception capacity. It operates in both the 2.4 GHz and 5 GHz ISM bands, uses the same modulation schemes as 802.11g, and pushes maximum data rates up to 600 Mbit/s.

IEEE 802.11ac (Wi-Fi 5)

Operating exclusively in the 5 GHz ISM band, 802.11ac adopted even wider radio channels (up to 80 MHz and 160 MHz) and doubled the maximum number of parallel downlink streams to eight. It optimized the frame formats and introduced highly sophisticated 256QAM modulation. While its theoretical network capacity extends into several Gbit/s, real-world implementations typically achieve data rates in the high hundreds of Mbit/s.

IEEE 802.11ax (Wi-Fi 6)

Unlike 802.11ac, the newer 802.11ax standard brings next-generation performance back to both the 2.4 GHz and 5 GHz ISM bands. Looking ahead, it is also designed to utilize an upcoming unlicensed band around 6 GHz, which offers a massive maximum bandwidth of 1200 MHz. A major breakthrough in 802.11ax is that it applies MIMO technology to the **uplink channel** (from the mobile terminal back to the AP), not just the downlink. Utilizing an incredibly dense 1024QAM modulation scheme, 802.11ax provides significantly higher effective data rates and network efficiency than any of its predecessors.

CHAPTER 3

Introduction to Wireless Personal Area Networks (WPANs)

Wireless Personal Area Networks enable communication between devices located in very close proximity to each other, typically within a radio range of 10 to 100 centimeters. The application domains for these networks are vast and diverse:

- **Home Environment:** WPANs are utilized for the remote control of motorized devices, electronic appliances, and lighting systems.
- **Industrial Environment:** They facilitate communication among equipment of varying sizes and complexities, ranging from massive industrial robots to miniaturized sensors.
- **Personal Environment:** Individuals can wear numerous devices—such as watches, bracelets, headphones, and medical monitors (e.g., blood pressure or heart rate monitors)—that communicate by exchanging information either periodically (like sending temperature data) or continuously (like audio streaming).

Bluetooth Technology: History and Evolution

The most widely used technology for developing and deploying WPANs today is Bluetooth. The technology was named after the 10th-century Danish king Harald Blåtand (Harald Bluetooth). Known for his fondness for blueberries, which earned him his moniker, King Harald successfully unified Norway and Denmark. This demonstrated a strong ability to bring things together, a trait mirrored by Bluetooth technology's ability to connect devices. In 1994, the telecommunications company Ericsson adopted the king's nickname for their new family of protocols.

By 2006, mobile phones universally began incorporating Bluetooth interfaces, allowing people to share files and multimedia content directly through this connection. The year 2015 marked the beginning of the Internet of Things (IoT) era, which drove the widespread integration and use of Bluetooth in smartphones. Subsequently, in 2019, the Bluetooth Low Energy (BLE) Mesh standard was released to further support complex network structures.

Generations of Bluetooth

Bluetooth solutions generally enable the transfer of high-quality data and audio between nearby devices using low-cost hardware. The standard has undergone significant evolutionary steps:

- **Bluetooth 1.1 (Basic Rate - BR):** Introduced in 2002 under the IEEE 802.15.1 standard, this initial specification defined transmissions up to 1 Mbit/s with a functional range of up to 10 meters.
- **Bluetooth 2.0 (Extended Data Rate - EDR):** Released in 2004, EDR introduced data transmissions at 2 Mbit/s and 3 Mbit/s by utilizing two different modulation schemes: $\pi/4$ DQPSK and 8DPSK, respectively.
- **Bluetooth 3.0:** Introduced in 2009, this version increased the data rate and acted as a support to IEEE 802.11, reaching speeds up to 24 Mbps.
- **Bluetooth 4.0 (Low Energy - LE / Bluetooth Smart):** The most significant change occurred in 2010. BLE provides a low-energy feature specifically designed for IoT devices and low-power applications. It offers lower data rates (up to 1 Mbps using GFSK modulation). BLE is intended for applications requiring periodic transmissions of small amounts of data (e.g., fitness trackers) rather than continuous data streams (e.g., wireless earbuds).
- **Bluetooth 5.0:** Released in 2016, it improved BLE performance in both data rate and range, offering four distinct data rates: 2 Mbps, 1 Mbps, 500 kbps, and 125 kbps.
- **Bluetooth 5.1:** Released in 2019.

Network Topology: Piconets and Scatternets

The foundational structural unit of a Classic Bluetooth network is the **piconet**, which consists of up to 8 "active" devices. Within a piconet, one device assumes the role of the **master**, while the remaining devices act as **slaves**.

- Information exchange occurs exclusively between the master and its slaves; slave devices cannot communicate directly with one another.
- A piconet can technically include up to 255 slave devices, but a maximum of 7 can be in an "active" state at any given time. The remaining devices are kept in a "parked" state.
- The master dynamically manages these states based on the need to transmit or receive information from specific devices.
- To conserve energy while active, devices can use a "sniff" (intermittent) mode, waking up to listen to the channel only during predetermined time slots at fixed intervals.

Individual piconets can be interconnected to form a broader topology known as a **scatternet**. This occurs when certain slave devices act as bridges, communicating simultaneously in more than one piconet (i.e., a slave communicates with two

different masters). Within scatternets, slave devices can maintain either active or parked states across their respective piconets.

BLE Network Topology

In Bluetooth Low Energy (BLE), the communication paradigm shifts slightly. Slave devices communicate with the master over a completely separate physical channel. Unlike Classic Bluetooth piconets where slave nodes passively listen for master connection requests, a BLE slave node actively initiates the connection, allowing it to strictly control its own energy consumption. The BLE master node, typically free from severe power constraints, listens for multiple advertisement frames on the "BLE Advertising PHY Channel" and initiates the connection after receiving one. Once connected, data flows over the "BLE Piconet PHY Channel" in a Star-Bus topology.

Bluetooth Device Architecture

The protocol stack of a Bluetooth device fundamentally differs from the standard TCP/IP model, divided into hardware/firmware and software components.

The Bluetooth Controller (Hardware/Firmware)

The Bluetooth controller is the physical chip responsible for physical links, packet management, and frequency hopping. It comprises three main modules:

1. **Radio Tx/Rx:** Generates and receives the radio-frequency (RF) signals that carry data.
2. **Link Controller:** Responsible for the coding and decoding of information units (IUs) transmitted over the radio channel.
3. **Link Manager:** Creates, edits, and releases logical connections while updating physical link parameters. It relies on the Link Manager Protocol (LMP) to carry these control messages.

The Bluetooth Host (Software)

The Bluetooth host is a logical software entity residing in the main system (like a smartphone or PC). It contains the higher protocol layers, including L2CAP, SMP, Attribute Protocol, GAP/GATT (for BLE), and high-level application profiles. The Host and the Controller communicate via the **Host Controller Interface (HCI)**. To minimize hardware costs, the Controller features limited data storage compared to the Host, necessitating specialized data fragmentation functions.

Traffic Flows and L2CAP

Two primary data flows operate between the host and the controller:

- **Synchronous Unframed Traffic:** Dedicated to transporting real-time audio/voice. It requires an established logical connection and utilizes fixed, periodically reserved time slots managed by the master. This ensures low, predictable latency but restricts bandwidth to roughly 64 kbps per channel.
- **Asynchronous Framed Traffic:** Handles general-purpose data (file transfers, commands, signaling). It does not require reserved time slots; the master exchanges asynchronous packets with one slave at a time and utilizes retransmissions to handle errors and ensure reliability. This traffic can be exchanged without a strict logical connection.

The L2CAP Module: The Logical Link Control and Adaptation Protocol (L2CAP) acts as a bridge, adapting the physical link to higher-level protocols and ensuring device interoperability. **Crucially, L2CAP is only involved with asynchronous framed traffic;** synchronous voice traffic completely bypasses it. L2CAP provides connection-oriented channels for reliable data and connectionless channels for broadcast data. Its primary functions are:

- **Multiplexing:** Allowing distinct services (e.g., audio and file transfer) to share a single link without interference.
- **Segmentation and Reassembly:** Breaking down large upper-layer packets (up to 64 kB) into smaller frames for RF transport, and reassembling them at the receiver.
- **Flow Control and QoS:** Enforcing flow rules to prevent receiver overload and negotiating Quality of Service parameters.

Classic Bluetooth Operations

Classic Bluetooth operates in the 2402-2480 MHz Industrial, Scientific, and Medical (ISM) band, which is unregulated and shared with other technologies like IEEE 802.11 Wi-Fi. This shared environment makes Bluetooth susceptible to interference.

To combat interference, Bluetooth defines 79 radio channels (each 1 MHz wide) managed by the master device within a piconet. Devices transmit Information Units (IUs) utilizing a **Frequency Hopping Spread Spectrum (FHSS)** technique. The hopping frequency (f_h) is 1600 Hz, meaning the transmission channel changes 1,600 times per second. Consequently, a single IU transmission

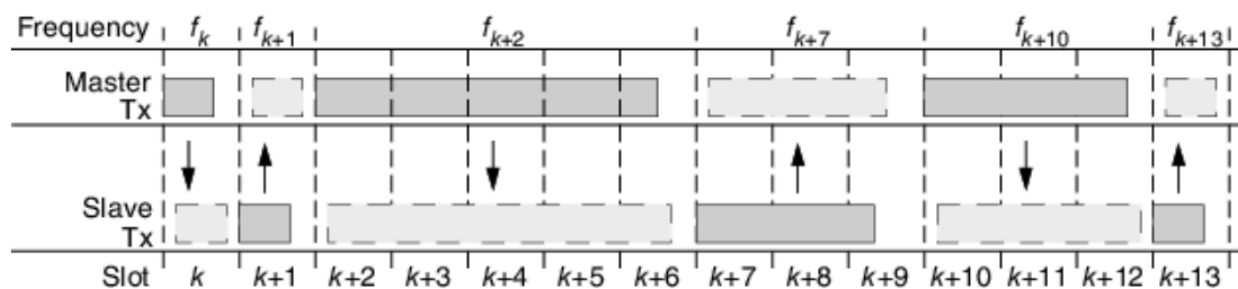
cannot exceed 625 microseconds (1/1600 seconds). The master dictates the specific sequence of frequencies used in the piconet to actively avoid signals emitted by Wi-Fi equipment.

Communication and Piconet Formation

In Classic Bluetooth, the master acts as a server, listening to the channel and initiating connections, while the slave acts as a client scanning the environment. The dynamic formation of a piconet involves several steps:

1. **Discovery:** The master periodically transmits search messages carrying a universal access code across a predetermined sequence of channels.
2. **Response & Activation:** A listening device that receives the search message replies, becomes an active slave in the piconet, and receives a specific address.
3. **Frequency Sharing:** The master informs all active slaves of the specific frequency hopping sequence to be used. (A slave in two piconets receives two separate sequences from two masters) .
4. **Polling:** Communication relies on a strict polling-and-response protocol. The master periodically sends interrogation messages to check if slaves need to transmit. The slave *must* transmit its response in the exact time slot immediately following the master's polling slot.

An Information Unit can occupy a single time slot, or it can span three or five consecutive slots. For multi-slot IUs, the initially selected frequency channel is held for the entire transmission duration; the overall hopping sequence remains unaltered once the transmission concludes.



Example of IU exchange between a master device and a slave device, where both devices transmit IUs of different lengths, occupying 1, 3, or 5 time slots.

Power Classes

Bluetooth defines three power transmission levels based on distance requirements. Class 1 and BLE devices utilize Transmit Power Control (TPC) to dynamically

adjust power, minimizing interference and saving battery; TPC is optional for Classes 2 and 3.

- **Class 1:** [2.5 mW, 100 mW], Range < 100 m. TPC is mandatory.
- **Class 2:** [1 mW, 2.5 mW], Range < 10 m. TPC is optional.
- **Class 3:** < 1 mW, Range 0.1 to 1 m. TPC is optional.
- **BLE:** [10 μ W, 10 mW], Range < 50 m. TPC is mandatory.

Information Unit (IU) Structures

Basic Rate (BR) IU: The standard Bluetooth BR IU achieves symmetric speeds up to 433.9 kbit/s, and asymmetric speeds up to 723.2 kbit/s down and 57.6 kbit/s up. Its structure contains:

- **Access Code (68-72 bits):** Identifies a specific device or indicates a generic search message.
- **Header (54 bits):** Identifies the target receiving device, defining the logical transport channel.
- **Payload (0-2745 bits):** Contains user data, LMP messages, or L2CAP frames.

Extended Data Rate (EDR) IU: EDR increases throughput to 2.1 Mbit/s by dividing the IU into two sections with different modulation schemes.

- **Section 1 (GFSK Modulation):** Contains the Access Code (72 bits) and the Header (54 bits), identical to the BR standard.
- **Guard Time & Sync:** A guard period and a 30-bit synchronization sequence allow devices to recalibrate for the new modulation.
- **Section 2 (DPSK Modulation):** Contains a Payload Header (16 bits, used if multiple logical channels exist on one transport), the Payload (0-8168 bits), and a 6-bit Trailer to explicitly mark the final payload bit.

Bluetooth Low Energy (BLE)

The introduction of Bluetooth Low Energy (LE), commonly referred to as Bluetooth Smart, aimed to fundamentally minimize the energy required for communication between nearby devices. This optimization enables the creation of highly miniaturized devices, such as those used in sensor networks, that can operate continuously for years without the need for battery replacement.

Unlike Classic Bluetooth, BLE is typically utilized strictly for data transfer rather than continuous audio streaming. While BLE shares the same 2.4 GHz ISM band and Frequency Hopping Spread Spectrum (FHSS) mechanism as the BR/EDR standards, it employs a distinct radio channel structure and relies exclusively on GFSK modulation.

BLE Channel Architecture

BLE divides the spectrum into 40 channels, each with a wider bandwidth of 2 MHz (compared to Classic Bluetooth's 79 channels of 1 MHz).

- **Data Channels:** 37 channels are dedicated exclusively to the transmission of data frames between devices.
- **Advertising Channels:** The remaining 3 channels are reserved for signaling operations—specifically, advertising frames used to establish connections.

[Photo: BLE Frequency Channel Diagram] Description: The BLE spectrum ranges from 2402 MHz to 2480 MHz. To mitigate interference from Wi-Fi networks, the three Advertising Channels are strategically placed: Channel 37 at 2402 MHz (the very beginning of the band), Channel 38 at 2426 MHz (in the gap between common Wi-Fi channels), and Channel 39 at 2480 MHz (at the very end of the band). The 37 Data Channels (labeled 0 through 36) are distributed evenly in the remaining frequencies between the advertising channels.

The BLE Link Layer State Machine

The operational core of the BLE link layer is governed by a precise state machine comprising five distinct states:

1. **Standby State:** This is the default, initial state characterized by low power consumption where the link layer is completely inactive. From here, a device can switch into the Scan, Advertising, or Init states.
2. **Advertising State:** A device actively sends advertising frames and responds to scan requests from surrounding devices. While in this state, the device is discoverable. If it receives a connection request from a master device (which is in the Init state), it can transition into the Connection state as a "slave" node.
3. **Init State:** This is the initial state of a node intended to act as a master, actively listening for a specific advertising frame to initiate a connection.
4. **Scan State:** The device actively listens the wireless channel for advertising frames. If scanning stops, it reverts to Standby. Scanning operates in two modes:

- *Passive*: The device simply listens for incoming advertising frames.
- *Active*: The device actively sends out scan requests, prompting other devices to reveal themselves by sending advertising frames.

5. **Connection State**: When a scanning device receives an advertising frame, it can attempt to connect by sending a connection request, transitioning the master node to the Connection state. Concurrently, the advertising device receives this request and transitions to the Connection state as a slave. Within this state, the master periodically sends data frames to the slave. To save energy, the slave node can enter a "sleep state," temporarily ignoring master frames. A device can switch from the Connection state back to the Standby state.

BLE Communication and Data Flow

To maximize battery life, BLE allows a device (the *advertiser*) to autonomously send Information Units (IUs) on one of the three advertising channels without waiting to be polled by a master. Any receiving device (the *scanner*) can request additional information on that exact same channel, and the advertiser will respond there as well.

If a sustained data exchange is necessary, an *initiator* device can use the advertising channel to request a formal logical connection with the advertiser. Once connected, the initiator assumes the master role, and the advertiser becomes the slave. Furthermore, to prevent waking up for irrelevant data, a device's host controller block can maintain a specific list of approved advertisers, scanners, or initiators from which it will accept IUs.

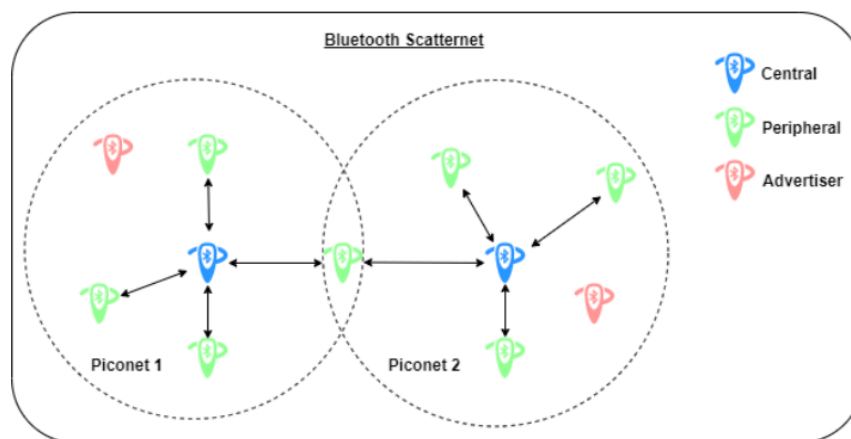
During a connection, devices transmit data frames sequentially. Each frame explicitly dictates three parameters: the sequence number of the current frame (S), the expected sequence number of the next incoming frame (N), and a flag indicating whether more frames will follow (M). If a frame is lost in transit, the transmitting node analyzes the expected sequence number (N) requested in the subsequent incoming frame and retransmits the missing data.

The BLE Information Unit (IU) Format

Despite BLE having a transmission rate of 1 Mbit/s (matching the Classic BR standard), its payload capacity is deliberately constrained to facilitate short, low-energy bursts of data. The BLE IU format consists of:

- **Preamble (8 bits):** Ensures synchronization between the transmitter and receiver.
- **Access Code (32 bits):** Identifies the specific Bluetooth device or flags the packet as a search/inquiry message.
- **Payload Header (16 bits):** Contains the receiving device's identifier to specify the transport channel. If a device uses multiple logical channels, it also contains the specific logical channel identifier.
- **Payload (0 to 296 bits):** Carries L2CAP frames or direct user data, restricted to a maximum of 37 bytes.
- **CRC (24 bits):** A Cyclic Redundancy Check code used to verify payload integrity and detect errors.

BLE Topologies and Bluetooth Mesh



- Example: a smartphone with a point-to-point connection to a smartwatch.

Classic Bluetooth piconets typically support up to 7 active slaves communicating with a single master. Conversely, a BLE piconet operates strictly as a point-to-point link involving exactly two active devices: one central (master) and one peripheral (slave), which share a slot sequence dictated by the master. If a device acts as a master to two separate slaves, it is technically managing two entirely separate piconets, each utilizing its own distinct frequency hopping channel sequence. The total number of active slaves a BLE master can connect to is dictated by hardware limits and application needs, circumventing the hard limit of 7 found in Classic Bluetooth.

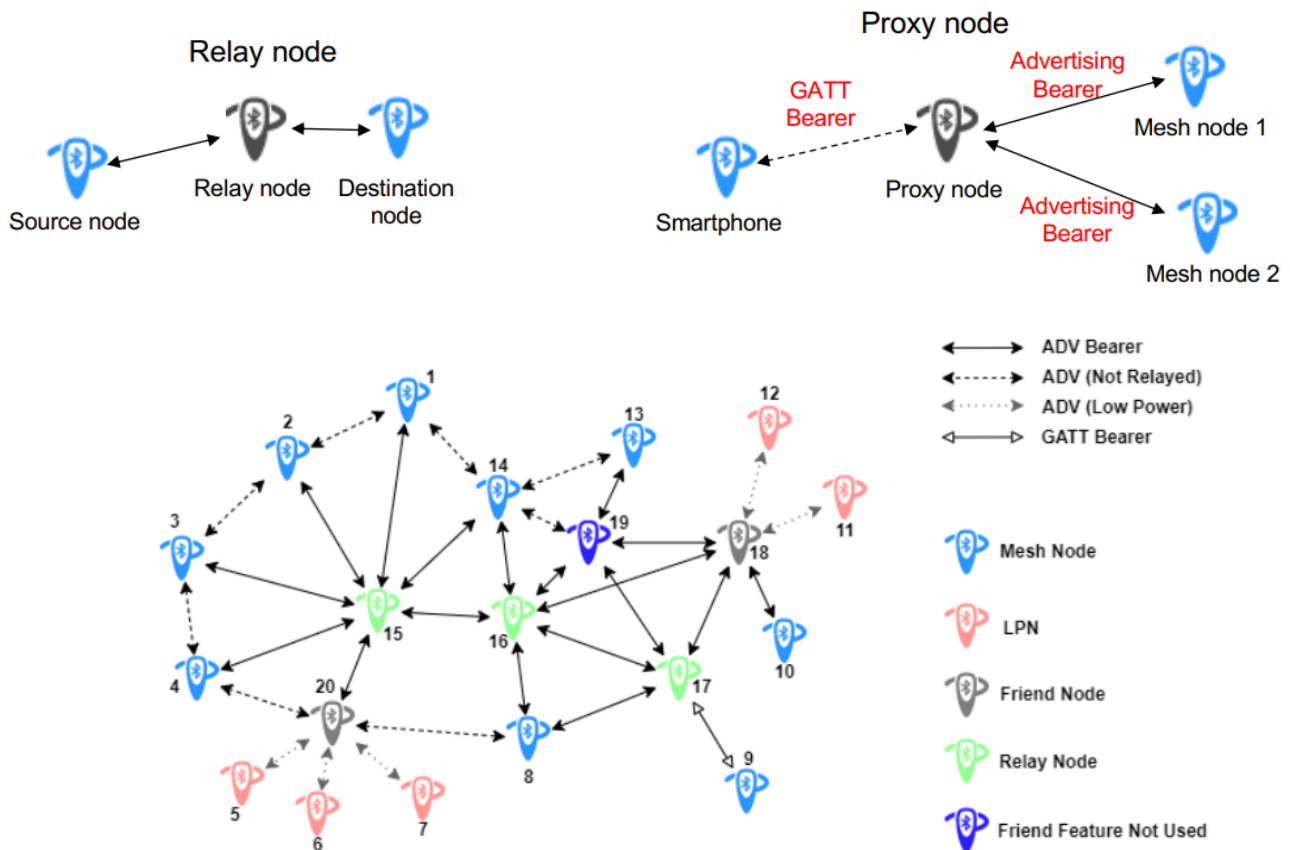
To overcome the range limits of point-to-point and point-to-multipoint topologies, the **Bluetooth Mesh** topology was introduced. Mesh networking enables many-to-many communication, allowing devices to relay data across long distances to reach nodes outside of their immediate radio range. This architecture boasts self-

healing capabilities; if a node fails or disconnects, there is no single point of failure, and the network routes around the missing device to remain operational.

Mesh Node Types and Functions

Devices within a mesh network are classified as "nodes," while external devices are called "unprovisioned devices". Mesh nodes can assume specific specialized roles:

- **Relay Nodes:** Responsible for multihop routing, forwarding received messages deeper into the network toward their destination.
- **Proxy Nodes:** Serve as an intermediary bridge for standard BLE devices (like smartphones) that lack a full Bluetooth Mesh protocol stack. The smartphone communicates with the proxy node using standard GATT (Generic Attribute Profile) operations, and the proxy node translates and broadcasts these messages into the mesh using the Advertising Bearer.
- **Low Power Nodes (LPNs) & Friend Nodes:** Nodes with severe battery constraints (e.g., a temperature sensor sending data only when a threshold is breached) act as LPNs. They form a "friendship" relationship with a strictly grid-powered "Friend Node". The Friend node caches and stores incoming messages addressed to the LPN in a "friend queue". The LPN conserves energy by remaining in a sleep state. Periodically, the LPN wakes up and enters a *receive window* to poll its Friend node, downloads any cached messages, and then immediately returns to sleep for the duration of the *receive delay*. If an LPN fails to poll within a predefined *poll timeout*, the friendship tie is severed.



Network Operations: Provisioning, Publish/Subscribe, and Flooding

To transform an unprovisioned device into a functional mesh node, it must undergo **Provisioning**, a secure process managed by a Provisioner (usually a smartphone):

1. **Beaconing:** The unprovisioned device broadcasts mesh beacons to signal its availability.
2. **Invitation:** The provisioner sends an invite packet, and the device responds with its technical capabilities.
3. **Public Key Exchange:** Both devices securely swap cryptographic public keys.
4. **Authentication:** The device generates a random number for the user to authenticate the connection.
5. **Data Distribution:** A secure session key is generated, and critical network provisioning data is transferred to finalize the node's integration.

Once operational, the mesh relies on a **Publish/Subscribe** messaging system. Nodes do not necessarily address messages to specific MAC addresses; instead, a "Publisher" node (e.g., a light switch) sends messages to a virtual group address

(e.g., "Kitchen"). Any node (e.g., light bulbs) configured to "Subscribe" to that virtual address will process the message, while all others ignore it. This decoupling allows devices to be replaced or added seamlessly without network-wide reconfiguration.

To route these messages, Bluetooth Mesh uses **Managed Flooding**. When a message is published, it is broadcast to all neighboring nodes. However, to prevent network congestion, *only* devices configured as Relay Nodes will retransmit the broadcast message further down the line until it reaches its intended subscribers.

The Evolution of Bluetooth 5.x Standards

- **Bluetooth 5.0 and 5.1:** These specifications doubled the data transfer capacity of version 4.0, pushing throughput to 2 Mbit/s. They optimized the IU payload to carry up to 255 bytes (achieving a max real-world throughput of 1.4 Mbit/s) and allowed advertisers to send data at strict, predefined intervals, granting scanner devices even greater energy savings.
- **Bluetooth 5.2:** Released in December 2019, this update was a massive leap forward, introducing:
 - **LE Audio:** Enabled by the new Low Complexity Communication Codec (LC3), LE Audio provides superior sound quality at roughly half the bitrate (e.g., 160 kbps) of the legacy SBC codec used in Classic Bluetooth, drastically reducing power consumption.
 - **Multistream & Isochronous Channels (ISOC):** BT 5.2 introduced ISOC to enable time-synchronized data distribution. This supports Multistream audio, allowing a phone to send independent, perfectly synced audio streams to multiple receivers (like left and right earbuds) simultaneously. This is managed by creating a Connected Isochronous Stream (CIS) for each connection, and grouping them into a single Connected Isochronous Group (CIG) to ensure perfect timing. A CIG can hold up to 31 CIS links.
 - **Broadcast Audio:** Using Broadcast Isochronous Streams (BIS) grouped into a Broadcast Isochronous Group (BIG), an audio source (like a TV in a public space) can broadcast synchronized audio connectionlessly to an infinite number of receiving peripherals.
 - **EATT & LEPC:** Enhanced Attribute Protocol (EATT) allowed concurrent data transactions, while LE Power Control (LEPC) dynamically managed power to ensure coexistence with Wi-Fi and ZigBee.

- **Bluetooth 5.3:** Focused on iterative improvements rather than speed/range increases. It optimized channel management to reduce interference, enhanced connection stability for wearables, and eliminated unnecessary transmissions to boost device battery autonomy.
- **Bluetooth 5.4:** Specifically designed to bolster large-scale IoT deployments (like sensor networks). It introduced highly advanced periodic advertising and Encrypted Advertising Data (EAD), ensuring that broadcasted sensor data is encrypted and readable only by authorized network nodes.

ZigBee Technology Overview

Developed in 2004, ZigBee is a short-range radio technology specifically designed to establish WPANs tailored for control and monitoring applications. Its core strengths lie in providing reliable, low-cost, low-complexity, and highly secure networking with exceptionally long battery life . It is capable of supporting a massive number of interoperable nodes.

The standard is maintained by the ZigBee Alliance, a non-profit association comprising roughly 150 companies (including industry giants like Mitsubishi, Philips, Samsung, Siemens, and Motorola). While the physical (PHY) and medium access control (MAC) layers are strictly based on the IEEE 802.15.4 standard, the ZigBee Alliance defines and develops the upper layers, including the Network/Security layer and the Application layer.

The "ZigBee" Analogy

The technology's unique name is derived from the "waggle dance" of honeybees. When a swarm of bees needs to relay a message toward the hive, individual bees perform a specific zigzag dance. This dance is sequentially repeated by bees progressively closer to the hive until the message arrives. Similarly, ZigBee devices coordinate in a mesh network where a wireless signal "zigzags" from node to node, ensuring the message reaches its destination efficiently and robustly.

Application Domains

Given its low power consumption, ZigBee is ubiquitous in monitoring and automation:

- **Building and Home Automation:** Includes HVAC (Heating, Ventilation, and Air-conditioning) control and smart home ecosystems.

- **Healthcare and Fitness:** Used for medical monitoring, personal health care, and fitness tracking.
- **Industrial & Environmental:** Encompasses industrial monitoring, agricultural sensors, vehicular tracking, and water level management.
- **Smart Energy & Lighting:** Utilized for home energy monitoring and the Light Link standard for controlling LED lighting.
- **Consumer Electronics & Services:** Includes RF4CE (Radio Frequency for Consumer Electronics) remote controls, wireless restaurant ordering systems, and intelligent traffic light control.

ZigBee Protocol Architecture and Modulation

The ZigBee protocol stack is layered. At the bottom, the IEEE 802.15.4 standard dictates the PHY and MAC layers. Above this, the ZigBee Platform Stack introduces the **Network and Security Layer**, which provides critical mechanisms for configuring the network, managing security, and handling routing. This layer actively calculates routing paths when topologies change, discovers neighboring nodes, and updates routing tables when new devices join. The top consists of the Application Support layer and the specific Application layer (which can be ZigBee-defined or vendor-specific).

Frequencies, Data Rates, and Modulation

ZigBee operates on unlicensed Industrial, Scientific, and Medical (ISM) radio bands. The standard defines specific parameters for different global regions:

- **Global (2.4 GHz):** Offers a data rate of 250 kbps across 16 available channels.
- **Americas (915-921 MHz):** Offers a data rate of 40 kbps across 10 available channels.
- **Europe (868 MHz):** Offers a data rate of 20 kbps on a single channel.

To ensure robust communication, ZigBee utilizes **Direct Sequence Spread Spectrum (DSSS)** modulation. DSSS intentionally spreads the signal across the frequency domain, resulting in a wider bandwidth that is highly resistant to narrow-band interference. Channel access is governed by the **CSMA/CA** (Carrier-Sense Multiple Access with Collision Avoidance) protocol, which allows multiple nodes to access the channel at different times, effectively avoiding collisions.

Transmission distances generally vary from 10 to 100 meters based on environmental conditions, though outdoor line-of-sight paths can achieve ranges up to 1000 meters. The standard restricts transmission power to a maximum of -3 dBm (0.5 W) and requires a minimum receiver sensitivity of -92 dBm at 900 MHz, and -85 dBm at 2.4 GHz.

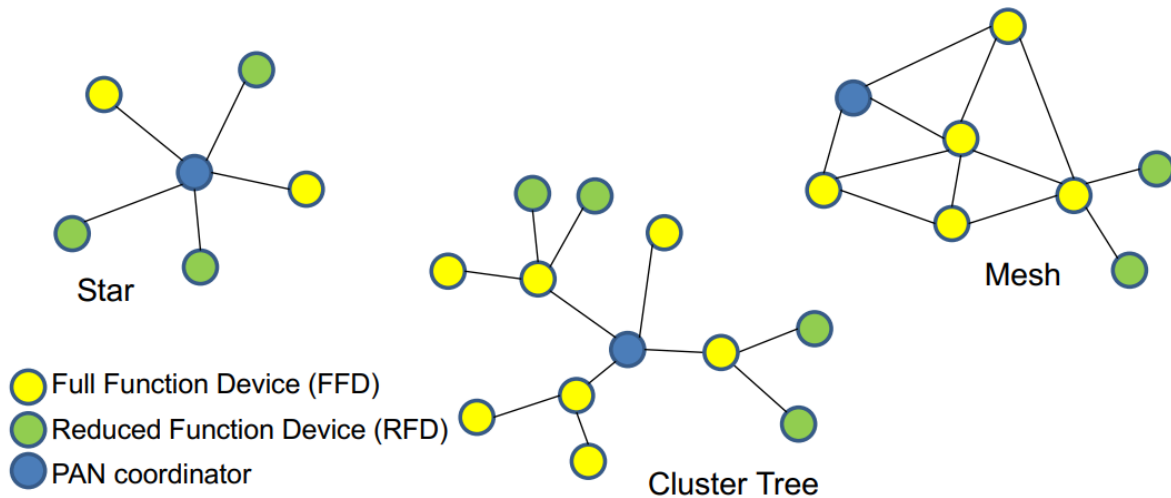
Node Types and Network Topologies

ZigBee networks are constructed using three specific hardware/software classifications:

1. **Reduced Function Device (RFD):** These are highly simplified, battery-powered nodes with limited memory. They are restricted to a star topology and can *only* communicate directly with an FFD.
2. **Full Function Device (FFD):** These robust devices are typically powered by the electrical grid. They can operate in any network topology, communicate with both RFDs and other FFDs, and serve as routers to forward data across the network.
3. **PAN Coordinator:** This is a specialized, mandatory FFD that acts as the network's root. It initializes the network, manages active nodes, and securely stores the network's topological information.

Using these nodes, ZigBee supports several network topologies:

- **Star Topology:** All peripheral devices (RFDs and FFDs) communicate exclusively with the central PAN coordinator. To form a star network, an FFD broadcasts beacon requests searching for a coordinator. If present, the coordinator responds, and the FFD stores the network info. Devices join or leave the network using formal Association/Disassociation request messages.
- **Cluster-Tree Topology:** This hierarchical structure evolves from a peer-to-peer layout. New nodes send a "network join request" to an existing FFD. If accepted, the new node becomes a "child" situated exactly one hop further from the coordinator than its parent FFD. Routing in a cluster tree is strict; messages follow a singular predetermined path up and down the established branches.
- **Mesh Topology:** A peer-to-peer structure where multiple varied paths exist between any two nodes. FFDs act as routers, forwarding messages dynamically. This provides extreme reliability; if a node fails, the network reroutes the data.



Addressing and AODV Routing

ZigBee assigns 16-bit short addresses, allowing a massive theoretical capacity of approximately 65,000 nodes per network. In a hierarchical setup, the coordinator allocates blocks of addresses to directly connected routers, which sequentially subdivide and assign those addresses to their child nodes.

In a Mesh topology, routing relies on a simplified version of the **AODV (Ad Hoc On-Demand Distance Vector)** algorithm. AODV is a *reactive* protocol. Unlike proactive protocols that map the entire network constantly, AODV networks remain entirely dormant regarding routing until a specific connection request is made.

- **Route Discovery:** When data needs to be sent, the source node broadcasts a **Route Request (RREQ)** packet. Each RREQ includes a Time To Live (TTL) parameter to limit infinite retransmissions. If a node receives no response, a backoff mechanism triggers, retrying the request at increasing intervals up to a set maximum.
- **Route Reply:** When the RREQ reaches a node that knows the destination, that node sends a **Route Reply (RREP)** back along the temporary path to the source. The nodes compare the lengths of the different paths and select the most efficient route.
- **Route Maintenance:** Local routing tables store these efficient connections. If a link breaks or a node becomes unreachable, a **Route Error (RERR)** message is generated and propagated to warn the network to seek alternative paths.

Coexistence: ZigBee, Wi-Fi, and Bluetooth

Because ZigBee, Bluetooth, and standard Wi-Fi (IEEE 802.11b/g) all operate within the heavily congested 2.4 GHz ISM band, careful channel management is required to mitigate interference.

[Photo: Frequency Coexistence between IEEE 802.11b/g, IEEE 802.15.4, and Bluetooth] Description: This three-part diagram illustrates the RF spectrum overlap. (a) Displays the 16 ZigBee (IEEE 802.15.4) channels, numbered 11 to 26, spanning 2405 MHz to 2480 MHz. (b/c) Displays the wide Wi-Fi channels (22 MHz bandwidth each). In North America, the three non-overlapping Wi-Fi channels are 1, 6, and 11; in Europe, they are 1, 7, and 13. The diagram highlights that ZigBee channels 15, 20, 25, and 26 are strategically placed in the narrow "guard band" spaces exactly between the wide non-overlapping Wi-Fi channels to minimize interference.

Wi-Fi 802.11b/g features center frequencies defined by the equation $f_C = 2412 + 5 \times (k - 1)(MHz)$ (where k is the channel number from 1-11 in NA, or 1-13 in EU) . Because Wi-Fi channels are 22 MHz wide and spaced only 5 MHz apart, they heavily overlap. To survive, a ZigBee network deployment should ideally utilize channels **15, 20, 25, or 26**. These specific channels fall in the gaps between the main Wi-Fi channels, experiencing significantly less interference.

Regarding Bluetooth: A ZigBee signal has a bandwidth of 2 MHz. Because Classic Bluetooth rapidly hops across 79 individual 1 MHz channels, an active Bluetooth connection will inevitably hop directly onto the active ZigBee frequency. However, because Bluetooth is constantly moving, the statistical probability of a collision occurring on that specific 2 MHz overlap is roughly $3/79$, or approximately 4%.

Practical Network Scenarios and Calculations

To synthesize the operational parameters of both Bluetooth Mesh and ZigBee, consider the following practical mathematical scenarios:

Scenario 1: Bluetooth Publish/Subscribe Overhead

The Setup: A smart building contains 50 light bulbs and 5 control switches. Each switch publishes commands to a unique "Zone" virtual address, and exactly 10 bulbs subscribe to each Zone. **The Problem:** If all 5 switches simultaneously publish an "ON" command (1 message per switch), and the network's relay nodes

are configured for a retransmission count of 3, what is the total number of physical message transmissions? **The Calculation:**

- Original Messages generated: $5 \text{ switches} \times 1 \text{ message} = 5 \text{ original messages}$.
- Total Transmissions: Each original message is transmitted once, plus 3 subsequent retransmissions by the relay nodes.
- $5 \times (1 \text{ original} + 3 \text{ retransmissions}) = 20 \text{ total transmissions}$.

Scenario 2: BLE Friend Node Capacity

The Setup: A grid-powered Friend Node provides caching for 5 Low Power Nodes (LPNs). The Friend node possesses a tiny 1000-byte storage buffer. **The Problem:** Each LPN receives an average of 4 messages per hour, with each message sized at 20 bytes. How long will it take for the Friend node's buffer to overflow if the LPNs fail to wake up and poll? **The Calculation:**

- Total messages entering the node per hour: $5 \text{ LPNs} \times 4 \text{ msgs/hr} = 20 \text{ msgs/hr}$.
- Total data entering per hour: $20 \text{ msgs/hr} \times 20 \text{ bytes/msg} = 400 \text{ bytes/hr}$.
- Buffer Lifespan: $\frac{1000 \text{ bytes capacity}}{400 \text{ bytes/hr}} = 2.5 \text{ hours}$.

Scenario 3: Audio Codec Efficiency (BT Classic vs. LE Audio)

The Setup: A smartphone streams stereo audio for exactly 2 hours (7200 seconds). **The Problem:** Calculate the total data transmitted using the legacy SBC codec (345 kbps) versus the new LE Audio LC3 codec (160 kbps), and find the percentage of data reduction. **The Calculation:**

- SBC Data: $345 \text{ kbps} \times 7200 \text{ s} = 2,484,000 \text{ kbits}$. Converting to Megabytes ($\div 8000$): $\approx 310.5 \text{ MB}$.
- LC3 Data: $160 \text{ kbps} \times 7200 \text{ s} = 1,152,000 \text{ kbits}$. Converting to Megabytes ($\div 8000$): $\approx 144 \text{ MB}$.
- Data Reduction: Total megabytes saved is $310.5 - 144 = 166.5 \text{ MB}$. The percentage reduction is $\frac{166.5}{310.5} \times 100 \approx 53.6\%$.

Scenario 4: ZigBee Network Latency, Throughput, and Energy

The Setup: An industrial ZigBee mesh operates at 250 kbps in the 2.4 GHz band. The payload data packet size is 80 bytes, and the ACK packet is 15 bytes. The propagation time between any two nodes is 0.5 ms, and the internal

processing time at each node is 2 ms. The Short Interframe Space (SIF) is $192 \mu\text{s}$ (0.192 ms). Transmit power is 100 mW (0.1 W). **Calculations:**

1. **Total Latency for a 5-hop route:** A 5-hop route involves 5 intermediate router jumps to reach the final 6th node (the destination).

- Propagation Time: $5 \text{ hops} \times 0.5 \text{ ms/hop} = 2.5 \text{ ms}$.
- Processing Time: $6 \text{ total nodes} \times 2 \text{ ms/node} = 12 \text{ ms}$.
- Total Latency: $2.5 + 12 = 14.5 \text{ ms}$.

2. **Total Physical Transmission Time (Data + ACK):**

- Data packet (80 bytes = 640 bits): $\frac{640 \text{ bits}}{250,000 \text{ bits/s}} = 0.00256 \text{ s} = 2.56 \text{ ms}$.
- ACK packet (15 bytes = 120 bits): $\frac{120 \text{ bits}}{250,000 \text{ bits/s}} = 0.00048 \text{ s} = 0.48 \text{ ms}$.
- Total Time: $2.56 \text{ ms (Data)} + 0.192 \text{ ms (SIF)} + 0.48 \text{ ms (ACK)} = 3.232 \text{ ms}$.

3. **Maximum Theoretical Throughput:**

- Packets per second: $\frac{1 \text{ second}}{0.003232 \text{ s/packet}} = 309 \text{ packets/s}$.
- Throughput: $309 \text{ pkts/s} \times 640 \text{ data bits/pkt} = 197,760 \text{ bps} \approx 198 \text{ kbps}$.
- Network Efficiency: $\frac{198 \text{ kbps actual}}{250 \text{ kbps theoretical}} = 79.2\%$. If 10 nodes transmit simultaneously, they share the channel, resulting in an effective throughput of 19.8 kbps per node.

4. **Energy Consumption and Congestion:**

- Energy per transmission ($E = P \times t$):
 $0.1 \text{ W} \times 0.003232 \text{ s} = 0.0003232 \text{ J} = 0.3232 \text{ mJ}$.
- Daily Consumption (assuming 50 transmissions/day):
 $0.3232 \text{ mJ} \times 50 = 16.16 \text{ mJ/day} = 0.01616 \text{ J/day}$. Powered by a standard 10,000 Joule AA battery, the node would last theoretically
 $\frac{10,000}{0.01616} \approx 618,811 \text{ days} \approx 1.695 \text{ years}$.
- Network Congestion: If a network holds 32 sensors, each firing once every 30 seconds, that equals 64 transmissions per minute. The total time the channel is occupied per minute is
 $64 \times 3.232 \text{ ms} = 206.848 \text{ ms} = 0.2068 \text{ seconds}$. This yields a channel utilization of $\frac{0.2068}{60} = 0.345\%$. Because it is far below 100%, the channel is entirely uncongested.

CHAPTER 4

1. Introduction to IoT Systems

In an Internet of Things (IoT) system, there are three essential components: devices, a communication channel, and an application. These components continuously interact with the external physical world to gather and process data. Devices primarily consist of sensors, controllers, and actuators. They are typically embedded devices with highly limited computational capacity, a design choice made specifically to minimize overall energy consumption. The communication channel serves as the critical medium that enables the propagation of signals between these field devices and higher-level applications.

Structurally, physical assets in the real world are subject to sensing and actuation by these devices. The devices subsequently transmit data through the communication channel to an application layer, which is responsible for analytics. Finally, this application layer interfaces bidirectionally with broader enterprise processes to integrate the data into business operations.

2. IoT Network Typologies: WPAN vs. LPWAN

There are two primary types of IoT networks designed to facilitate communication: Wireless Personal Area Networks (WPAN) and Low-Power Wide Area Networks (LPWAN). The fundamental difference between these two technologies lies in the trade-off between network coverage and data transmission rate.

WPAN provides higher transmission rates, typically on the order of Mbps, but is limited to supporting short communication distances in the range of hundreds of meters. Conversely, LPWAN is associated with very low transmission rates, reaching only up to 50 kbps, but it can operate over extremely long distances spanning several kilometers.

When mapping technologies by throughput and range:

- **WPAN Technologies:** Protocols like RFID/NFC, BLE, and IEEE 802.15.4 occupy the low-range, low-to-medium throughput spectrum. Wi-Fi variants (IEEE 802.11a/b/g/n/ac/ah) offer significantly higher throughput at slightly longer WPAN ranges.
- **LPWAN:** Operates uniquely with very high range but lower throughput.
- **Mobile Technologies:** IP-based cellular networks like 3G, 4G, and 5G offer both high throughput and high range.

Both WPAN and LPWAN technologies reside in end devices that must communicate with IoT gateways, which ultimately provide access to applications via the broader Internet. The network segment connecting the remote devices to

the gateway is referred to as the access network, whereas the segment extending from the gateway out to the application is called the core network or backbone.

Network Connectivity and Gateway Roles

In WPAN, connectivity generally relies on a multi-hop architecture, where a device communicates with the gateway by relaying data through intermediate peer devices. This mesh connectivity approach allows WPAN to effectively operate over longer aggregate distances and increases coverage in the access network; however, it comes at the steep cost of additional latency introduced by the packet forwarding operations of those intermediate nodes. Furthermore, WPAN gateways are fully IP-compliant across the board.

In LPWAN, connectivity is strictly one-hop, featuring direct star-topology communications between a single device and the gateway. Unlike WPAN, the LPWAN gateway supports the IP protocol on only one of its interfaces (the core network side). Gateways in both architectures act as crucial network translators, performing the following operations:

- Converting physical layer frames (e.g., translating between a wired Ethernet link and a wireless IEEE 802.15.4 link).
- Converting network layer datagrams.
- Converting transport layer segments (e.g., forwarding traffic between HTTP and CoAP protocols).

The Need for LPWAN

While the traditional Internet prioritizes high data rates and Quality of Service for mobile users, cyber-physical communication systems have vastly different requirements. Many IoT applications demand networking capabilities specifically for remote support and control of very small devices equipped with sensors and actuators, making high data rates completely unnecessary. Therefore, a massive Machine-to-Machine (M2M) ecosystem requires technologies that support long-distance communications for low-cost devices constrained by battery limitations.

Comparing different network paradigms highlights why LPWAN fills a unique void:

- **WSN (Wireless Sensor Networks):** Aim for low data rates, low costs, and very high energy efficiency (transmit power $< 1\text{mW}$), but provide very short coverage ranges ($\sim 1\text{-}3$ meters).

- **WLAN:** Offers medium costs and very high data rates with short-range coverage (~10-100 meters, transmit power ~80mW), but fundamentally lacks support for energy efficiency.
- **Cellular Networks:** Designed for voice applications and high data rate services, these feature long ranges (~1-5 kilometers) but come with high complexity, infrastructure costs, and low energy efficiency (transmit power ~500mW).
- **LPWAN:** Features very high coverage of several kilometers (~5-20 km), very low costs, and very high energy efficiency (transmit power ~20mW), completely trading off data rate (which is very low) for these benefits.

Ultimately, WPAN technologies for IoT (like IEEE 802.15.4 and BLE) are power-efficient but suffer from low throughput and highly limited coverage. LPWAN technologies complement WPAN by drastically improving the transmission range while maintaining energy consumption at minimal levels, enabling battery lifetimes of several years. LPWAN devices generally transmit very small packets just a few times an hour over these vast distances. While LPWANs offer exceptional coverage—often exceeding 10 km in rural areas—low communication costs, and high energy efficiency, they are best suited for communications with fixed nodes or those with limited mobility. Because of this static nature, LPWAN core networks are highly simplified, avoiding complex mobility management or handover functionalities. Consequently, LPWANs are entirely unsuitable for mission-critical applications that demand ultra-low latencies (on the order of 1 ms) and high reliability.

3. The LoRa Protocol Stack

Long Range (LoRa) is the generic terminology for a specific protocol stack designed to provide LPWAN functionality to devices, prioritizing extreme battery energy savings extending beyond 10 years. The architecture of the LoRa stack is minimal, formed by a small subset of standard network layers encompassing only the physical layer, link layer, and application layer. This streamlined stack functions effectively as long as LoRa devices are communicating exclusively with one another. Notably, LoRa does not natively support the IP protocol, meaning there is no need for traditional network or transport layers within its localized stack.

The stack features a physical layer utilizing LoRa modulation across ISM bands, a LoRaWAN link layer divided into MAC options (Classes A, B, and C) alongside LoRa MAC, and an overarching application layer. Security is implemented natively at both the link and application layers.

The Physical Layer

At the physical layer, LoRa utilizes transmissions within various Industrial, Scientific, and Medical (ISM) bands, specifically the 433 MHz band and the 915/868 MHz band. The core of LoRa modulation relies on Chirp Spread Spectrum (CSS), a highly power-efficient Spread Spectrum mechanism. CSS enables exceptionally long-distance communications, providing coverage exceeding 10 km at very low transmission rates. While practical coverage estimates heavily depend on physical obstructions, a single LoRa base station can theoretically cover hundreds of square kilometers.

Network capacity is dictated by a strict trade-off between coverage, speed, and the number of devices associated with a given gateway. For instance, providing a range of 8 km at a data rate of 250 bps allows a gateway to reliably support up to 50 devices. By reducing the range to 2.5 km and increasing the speed to 5 kbps, the exact same gateway could support up to 700 devices.

Data rates are fundamentally tied to the specific ISM band in use, as different bands mathematically support varying numbers of subchannels. In Europe, the 868 MHz band supports 10 subchannels, allowing for transmission rates between 250 bps and 50 Kbps. In the US, the 915 MHz band supports more than 64 subchannels, yielding rates between 980 bps and 21.9 Kbps. Additionally, subchannel bandwidth varies heavily by region: uplink bandwidth ranges between 125 and 500 KHz for both bands, whereas downlink bandwidth is fixed at 125 KHz for the 868 MHz band and 500 KHz for the 915 MHz band.

The Link Layer

The LoRaWAN block serves as the link layer, governing channel access algorithms and defining the overall network architecture. LoRaWAN explicitly avoids the complex mesh architectures and data aggregation typical of WPANs—which drain the battery life and transmission capacity of intermediate nodes—in favor of a simpler star topology. LoRa devices communicate directly with gateways without any intermediate hops, prioritizing extended coverage over high transmission rates. To introduce redundancy into this system, a single end device is typically associated with multiple gateways simultaneously. The core LoRaWAN MAC layer operates based on the ALOHA protocol.

Gateways operate with dual interfaces: a LoRa interface to receive packets from the airwaves, and an IP interface to forward those packets to a centralized backend server. The backend server is then entirely responsible for verifying packet security

and systematically eliminating any redundant copies received from multiple listening gateways.

LoRa Device Classes

LoRa devices are categorized into three distinct classes based on power consumption, downlink latency, and their overall degree of interaction with gateways:

- **Class A:** Supported universally by all LoRa devices, it boasts the lowest power consumption, making it the most energy-efficient class for maximizing battery life. Gateways can process 6LoWPAN packets as soon as uplink traffic is received, but downlinks back to the device are strictly limited to two short receive windows occurring immediately after an uplink transmission.
- **Class B:** Builds upon Class A by incorporating extra, scheduled receive slots that are actively synchronized via gateway beacons. This scheduling allows gateways to send downlink packets at predictable times, drastically reducing latency compared to Class A, though it results in medium overall power consumption.
- **Class C:** The most responsive class, characterized by continuous listening; the device only closes its receive window when it is actively transmitting. Because the gateway can send downlink packets at any time as soon as traffic is received, latency is minimized, but it severely penalizes battery life due to constant high power consumption.

Latency Calculations in LoRa (Exercise)

To practically evaluate the latency trade-offs between classes, consider a LoRa network where a single device is located at a distance $d = 10$ km from a gateway. Assume the data rate is $R = 1$ Kbps, the frame length is $L = 13$ bytes, and the only delays present are transmission and propagation. The operational scenario requires the gateway to send a frame, followed by the device sending a responding frame after a processing delay of 56 ms.

The fundamental delays are calculated as follows:

- **Transmission Delay:** $D_t = \frac{8 \times L}{R} = \frac{8 \times 13}{1000} = 104$ ms
- **Propagation Delay:** $D_p = \frac{d}{c} = \frac{10 \times 10^3}{3 \times 10^8} = 33.34$ μ s

In **Class C**, both the gateway and the device can transmit frames immediately as their receive windows are open. Therefore, the total delay heavily simplifies to the

sum of two transmissions (downlink + uplink):

$$D_C = 2 \times (D_t + D_p) \approx 2D_t = 208 \text{ ms}$$

Conversely, in **Class A**, the gateway is artificially constrained; it can only transmit its downlink frame in a receive window that opens in response to a frame initially sent by the device. This requirement enforces the 56 ms waiting period, bringing the total delay to:

$$D_A = D_C + 56 \text{ ms} = 264 \text{ ms}$$

This mathematical proof confirms that while Class A extends battery life through restricted listening, it inherently results in increased communication latency compared to the always-listening Class C.

4. The SigFox Protocol Stack

SigFox represents an alternative LPWAN protocol stack, fundamentally distinct from LoRa because it operates entirely over a single commercial network (also named SigFox). However, the hardware ecosystem remains open, with multiple independent manufacturers producing compatible SigFox chips. Originally designed strictly for one-way uplink traffic (where sensors simply propagate readouts to the network), the stack was subsequently upgraded to accommodate limited downlink transmissions.

Like LoRa, SigFox devices prioritize very low power consumption and extremely low transmission rates to support extended battery lifespans. A critical distinction is that the SigFox protocol stack includes absolutely no native encryption mechanisms; instead, it relies completely on the application layer to encrypt data before transmission. The protocol structure maps an application frame down through the MAC (link) layer to the physical ISM band.

The Physical Layer

The SigFox physical layer operates within the 915/868 MHz ISM bands. For uplink transmissions, each band is subdivided into 333 ultra-narrow band (UNB) subchannels, each just 100 Hz wide, utilizing Differential Binary Phase Shift Keying (DBPSK) modulation. Downlink subchannels are significantly wider at 600 Hz each and employ Gaussian Frequency Shift Keying (GFSK) modulation. The 868 MHz band utilizes a transmission power of 14 dBm with a receiver sensitivity of -120 dBm, while the 915 MHz band operates at 22 dBm transmission power and -142 dBm receiver sensitivity.

To combat fading and improve channel diversity in the uplink, SigFox implements Frequency Hopping Spread Spectrum (FHSS) across 3 of its 333 available subchannels. This frequency hopping is intelligently combined with a random delay window occurring between 500 and 525 ms, a technique known as Random Frequency and Time Division Multiple Access (RFTDMA).

Transmission Rates and Constraints

SigFox is heavily engineered for the transmission of highly infrequent, small data bursts, and network transmission rates are actively estimated based on the total volume of connected devices. The maximum payload size is severely restricted: a mere 12 bytes for an uplink message and 8 bytes for a downlink message.

To guarantee battery longevity, rigid hardware-level constraints are enforced: a device is strictly prohibited from transmitting more than 6 messages per hour, capping maximum daily throughput at 140 uplink messages and just 4 downlink messages. Consequently, the sustained continuous transmission rates are extraordinarily low:

- **Uplink Rate:** $R_{up} = \frac{N_{up} \times L_{up}}{T} = \frac{140 \times 8 \times 12}{60 \times 60 \times 24} = 0.15 \text{ bps}$
- **Downlink Rate:** $R_{down} = \frac{N_{down} \times L_{down}}{T} = \frac{4 \times 8 \times 8}{60 \times 60 \times 24} = 0.003 \text{ bps}$

The Link Layer and Redundancy

At the Link layer, SigFox shares LoRa's reliance on the ALOHA MAC protocol and similarly features end devices that are entirely unsynchronized with the broader network. Transmission mechanics closely mirror LoRa's Class A configuration. Upon receiving an uplink message from a device, the base station (functioning as an IoT gateway) is granted a precise 20-second window to formulate and transmit a response back. Concurrently, the end device will wait up to 25 seconds to receive this 4-byte downlink payload.

Because the network operates in a strict "fire-and-forget" mode—meaning receivers never transmit acknowledgments (ACKs)—SigFox ensures delivery success through brute-force redundancy. Every payload is automatically copied and transmitted three times. These three copies are spread across three distinct, random frequencies and are staggered by three different time delays. Visually, this is represented as frames appearing at differing times t_1 , t_2 , and t_3 on disparate frequencies f_1 , f_2 , and f_3 . The critical 20-second downlink transmission window for the base station opens only after the device has successfully transmitted the final of these three redundant frames.

SigFox Frame Formats

SigFox frames are uniquely identified using an ID field embedded within the frame, negating the need for complex network addressing mechanisms since communication generally flows from massive clusters of devices to a central base station. The ID manages both routing and authentication. All frames begin with a Preamble (PREAMBLE) for physical synchronization, followed by a Synchronization (SYNC) field dictating the frame type.

- **Upstream (UL) Frame:** Contains a PREAMBLE (32 bits), SYNC (16 bits) for frame synchronization, Device ID (32 bits), a variable PAYLOAD (0-96 bits), AUTH (8 bits) for authentication, and a Frame Checksum (FCS) (16 bits) for error detection.
- **Downstream (DL) Frame:** Contains a PREAMBLE, SYNC (13 bits), FLAGS (2 bits), FCS (8 bits), AUTH (8 bits), Error Code (EC) (8 bits), and a variable PAYLOAD (0-64 bits).

Network Topology

The topology of a SigFox network is a massive star configuration where up to 1 million devices can seamlessly interact with a single base station. The number of supported devices scales heavily alongside the volume of messages transmitted in the network. Base stations straddle the divide between the SigFox Access Network and the IP Core Network, serving as the gateways that interface constrained end devices with powerful backend servers. While the device-to-gateway segment relies exclusively on specialized SigFox physical and link layer mechanisms, the gateway-to-server backbone leverages traditional IP networking.

In this highly redundant star topology, devices blindly transmit their messages simultaneously to multiple overlapping base stations. Therefore, the centralized backend server bears the heavy responsibility of filtering and removing all duplicate messages before storing the data. Finally, end-user applications interact with the network not by talking to devices directly, but by utilizing SigFox cloud service APIs to retrieve the deduplicated data stored on these backend servers. When an application needs to send a downlink message back to a device, the backend server calculates and selects the specific base station exhibiting the best connectivity and physical position to handle the transmission.

5. The Publish/Subscribe Model

The Publish/Subscribe (Pub/Sub) model consists of an event-based architecture specifically designed to provide asynchronous observations, perfectly aligning with the strict requirements of IoT ecosystems. It fundamentally differs from the traditional Request/Response model.

Request/Response (REST) vs. Publish/Subscribe

The **Request/Response model** (commonly known as the client/server model) is based on direct, synchronous interactions between applications and devices. In an IoT context, the end device typically operates as the server, while the application operates as the client. The client (application) requests data from the sensor or sends actuation commands via polling. The server (sensor/actuator) responds by transmitting the requested readouts or actuation notifications. This synchronous polling model dictates that for every single request generated, there must be a direct response, forcing devices to be awake and available to receive queries.

Conversely, the **Publish/Subscribe paradigm** relies on an asynchronous observation model without continuous polling. It introduces one or more central entities known as **brokers**. Brokers queue and deliver messages, providing services through the collection, storage, and forwarding of events between end nodes.

- First, the application **subscribes** to a specific topic of interest supported by a device.
- Second, the device **publishes** messages by transmitting them to the broker when an event occurs.
- Finally, the broker stores these messages and forwards them via **notifications** to all applications that previously subscribed.

This mechanism significantly improves network utilization and helps preserve device battery life, as devices only transmit when they have new data. However, it introduces a structural vulnerability: the central broker can become a single point of failure.

Core Publish/Subscribe Operations

To facilitate this architecture, end nodes (which can act as publishers, subscribers, or both) utilize the following six core operations:

1. **Advertise(t)**: Sent by a publisher to inform the broker about a topic t that will represent future events to be published. Brokers then broadcast this information to all other end nodes.

2. **Unadvertise(t):** Sent by a publisher to remove a previously active advertisement related to topic t .
3. **Subscribe(t):** Sent by an end node (application) to subscribe to a specific topic t .
4. **Unsubscribe(t):** Sent by an end node to cancel/replace an active subscription to a specific topic t .
5. **Publish(e):** Sent by a publisher to forward a specific event e to the broker.
6. **Notify(e):** Sent by the broker in response to receiving a published event e . This is broadcast sequentially to all subscribers associated with topic t .

Architectural Models and Performance

Pub/Sub architectures can be either centralized or distributed (decentralized):

- **Centralized Model:** Based on a single broker with point-to-point connections from publishers to the broker, and from the broker to subscribers. The broker manages internal queues to store incoming events before distribution. If the event arrival rate exceeds the queue's processing rate, the queue fills up, causing new events to be dropped (packet loss). Therefore, the single broker can become a critical bottleneck due to congestion.
- **Decentralized Model:** Prevents the congestion of the centralized model by removing the single broker and replacing it with a network of cooperating brokers. This provides redundancy and scales the network's overall functionality.

When comparing standard REST (Request/Response) architectures against Pub/Sub:

- **Energy Consumption:** Both architectures consume a similar amount of energy, assuming the physical distance between the client/server is comparable to the distance between the devices/applications and the broker.
- **Packet Loss:** The REST architecture relies on single-hop communications, whereas Pub/Sub inherently relies on multi-hop communications (Device $\rightarrow$ Broker $\rightarrow$ Application). Consequently, Pub/Sub mathematically experiences higher packet loss.
- **Latency:** Similarly, the multi-hop paths required by the broker system in Pub/Sub are responsible for higher overall latency compared to direct REST communications.

Practical Latency Calculations: Pub/Sub vs REST

To fully understand the latency differences, consider a lossless network where the only delay is transmission delay. We wish to calculate the time to transmit a packet of size $L = 200$ bytes.

Scenario 1: Pub/Sub Delay

Data is published from a sensor (at $R_{sensor} = 100$ Kbps) to a Gateway, then to a Broker, and finally to the Application. Assume the Gateway-to-Broker and Broker-to-Application links operate at $R_{GW} = R_{broker} = 1$ Gbps.

The packet is fully received and retransmitted at each hop (Gateway and Broker). The total delay D is:

$$D = L \left(\frac{1}{R_{sensor}} + \frac{1}{R_{GW}} + \frac{1}{R_{broker}} \right)$$

$$D = 200 \times 8 \left(\frac{10^{-3}}{100} + \frac{2 \times 10^{-9}}{1} \right) = 16 \text{ ms}$$

(Note: If all links were restricted to 100 Kbps, the delay would simply be $200 \times 8 \times \frac{3 \times 10^{-3}}{100} = 48 \text{ ms}$.)

Scenario 2: REST (Request/Response) Delay

In a REST setup, an Application must first send a Request packet ($L_{req} = 100$ bytes) through the Broker and Gateway to the Sensor. The Sensor takes 0.5 seconds (500 ms) to process the event, then sends a Response packet ($L_{res} = 200$ bytes) back through the same chain.

Assuming the same 1 Gbps core speeds and 100 Kbps edge speed:

- **Request Delay:** $D_{req} = 100 \times 8 \times \left(2 \times 10^{-9} + \frac{10^{-3}}{100} \right) = 8 \text{ ms}$
- **Response Delay:** $D_{resp} = 200 \times 8 \times \left(2 \times 10^{-9} + \frac{10^{-3}}{100} \right) = 16 \text{ ms}$
- **Total Delay:** $D = D_{req} + D_{resp} + 500 \text{ ms} = 524 \text{ ms}$

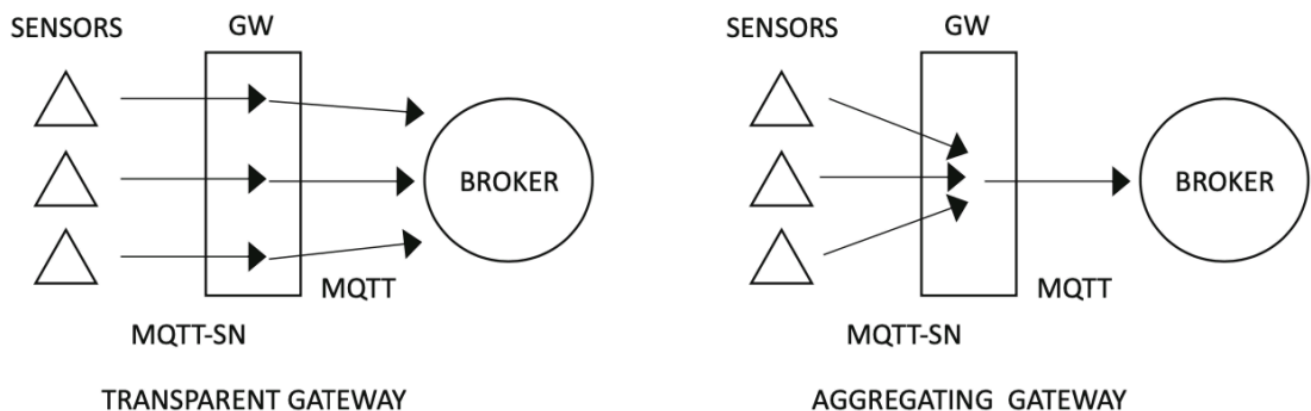
(If all links were restricted to 100 Kbps, $D_{req} = 24 \text{ ms}$, $D_{resp} = 48 \text{ ms}$, resulting in a total delay of 572 ms.)__

6. MQTT (Message Queuing Telemetry Transport)

Developed by IBM in 1999 specifically for low-bandwidth devices, MQTT is a broker-based publish/subscribe protocol. Due to its simplicity, low overhead, and

suitability for scenarios characterized by low transmission rates, high latency, and limited computational/storage capacity, it is one of the preferred protocols for IoT networks.

While MQTT heavily incorporates the Pub/Sub model, it outsources certain features; for example, topic advertising is handled through other out-of-band mechanisms. MQTT natively runs over TCP, providing an initial layer of reliable transport.



MQTT-SN (Sensor Networks)

Because standard MQTT relies on TCP—which can be heavy for highly constrained sensors—the protocol was extended in 2013 to **MQTT-SN**. This extension is optimized for IoT sensor networks and operates over connectionless UDP.

MQTT-SN requires an intermediate MQTT-SN Gateway to translate events into traditional MQTT messages for broker interaction. This gateway operates in two modes:

1. **Transparent Gateway:** Each individual MQTT-SN event from a sensor is translated directly into a single MQTT event.
2. **Aggregating Gateway:** Multiple distinct MQTT-SN events from various sensors are combined and forwarded as a single, consolidated MQTT event to save bandwidth.

MQTT Message Format and Types

To remain ideal for constrained IoT devices, MQTT messages are kept extremely small. The format consists of:

1. **Fixed Header (24 bits / 3 bytes):** Present in every message.

2. **Variable Size Header:** Optional, heavily dependent on the specific message type.
3. **Payload:** The actual data being transmitted.

The **Fixed Header** is meticulously structured:

- **Byte 1:**
 - **Message Type (4 bits):** Dictates the purpose of the message (e.g., PUBLISH, CONNECT).
 - **DUP (1 bit):** The Duplicate flag. Set to 1 for retransmissions in higher QoS levels.
 - **QoS (2 bits):** Defines the Quality of Service level (0, 1, or 2).
 - **RET (1 bit):** The Retain flag. Used by the publisher to instruct the broker to store the published message and automatically push it to any *future* subscribers.
- **Bytes 2 & 3:** Contain the **Remaining Length** field (16 bits), dictating the size of the variable header and payload.

Primary MQTT Message Types (0-15):

- **Connection Management:** To establish a TCP connection, a client sends `CONNECT` (1). The broker replies with `CONNACK` (2). To sever the connection safely, the client sends `DISCONNECT` (14).
- **Keep-Alive:** If no data is flowing, endpoints periodically send a `PINGREQ` (12) to renew the connection. The broker replies with `PINGRESP` (13). If a ping goes unanswered, the connection is considered terminated.
- **Publishing:** `PUBLISH` (3) is used to send events. Depending on the QoS level, this is acknowledged by `PUBACK` (4), `PUBREC` (5), `PUBREL` (6), and `PUBCOMP` (7).
- **Subscriptions:** Clients request data via `SUBSCRIBE` (8) and are confirmed via `SUBACK` (9). They stop receiving data via `UNSUBSCRIBE` (10) and `UNSUBACK` (11). (Types 0 and 15 are reserved).

Quality of Service (QoS) Levels

Although TCP provides baseline reliability, network instability in IoT can cause sudden disconnections. MQTT solves this by introducing three distinct QoS levels that define delivery guarantees:

QoS 0: "At Most Once" (Fire and Forget)

The publisher sends events on a best-effort basis. The `PUBLISH` message is sent exactly once, with no acknowledgment required from the broker. If transmission errors occur, the message simply vanishes and is never retransmitted.

- *Flow:* Publisher sends `PUBLISH (QoS=0)` → Broker receives and publishes to subscribers.

QoS 1: "At Least Once"

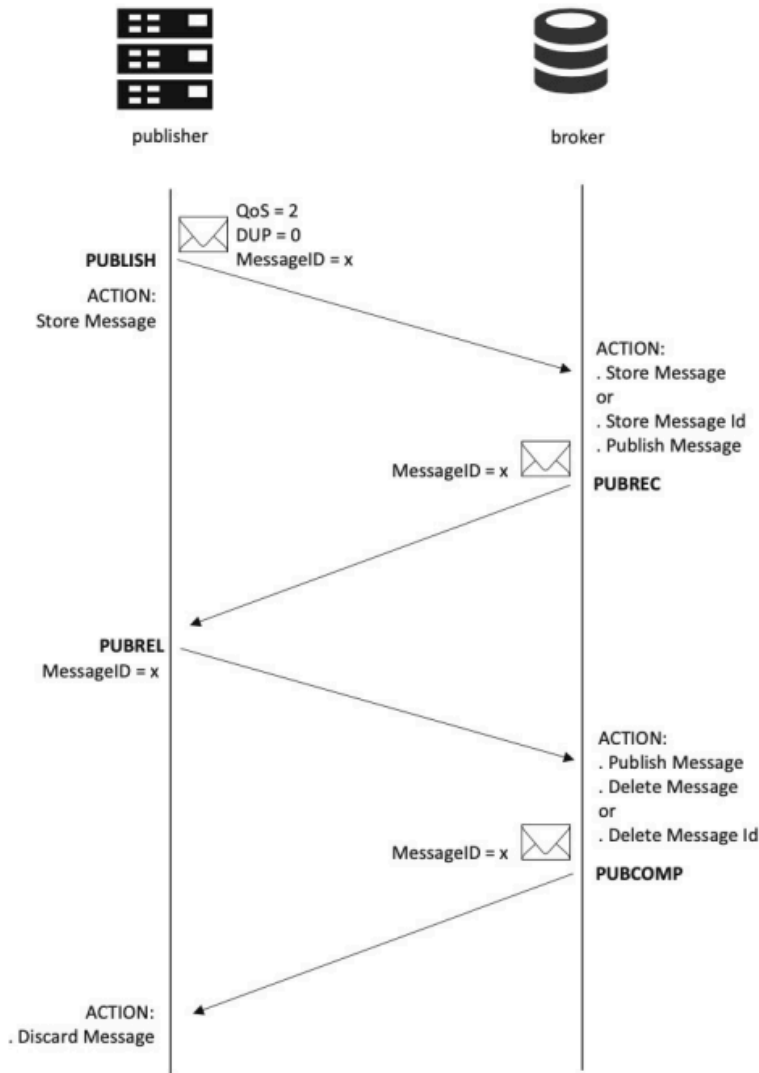
Guarantees that an event arrives at the broker at least once, meaning duplicate deliveries are possible.

- *Flow:* The publisher sends a `PUBLISH` message featuring a unique `MessageID = x`, with the `DUP` flag initially set to 0.
- Upon receipt, the broker stores the message, publishes it to subscribers, deletes it from its queue, and replies with a `PUBACK` to the publisher.
- *Error Handling:* If the publisher does not receive the `PUBACK`, it initiates a retransmission of the exact same message, but changes the flag to `DUP = 1`.

QoS 2: "Exactly Once"

The highest level of reliability. It guarantees delivery while strictly ensuring that duplicate messages are completely avoided. This requires a complex, four-way double-transaction handshake, causing additional delay and extra network overhead.

- *Flow:* The publisher sends a `PUBLISH` message (`MessageID = x`, `DUP = 0`).
- The broker receives it, stores it (or stores its ID), and replies with a `PUBREC` (Publish Received) message.
- The publisher receives the `PUBREC`, safely discards its own local copy of the message, and sends a `PUBREL` (Publish Release) message back to the broker.
- Upon receiving `PUBREL`, the broker finally publishes the message to subscribers, deletes the message/ID from its memory, and replies with a `PUBCOMP` (Publish Complete).



MQTT Transmission Rate Exercise

To see the overhead cost of higher QoS levels, consider a sensor generating MQTT messages over an IEEE 802.15.4 link (rate = 250 kbps). The average packet size is 200 bytes. The sensor must wait for one transaction to fully complete before starting the next. (Ignoring propagation delays).

- **QoS 0 Rate:** The sensor just sends PUBLISH messages continuously.

$$R_{QoS0} = \frac{250 \times 10^3 \text{ bps}}{8 \text{ bits} \times 200 \text{ bytes}} = 156.25 \text{ messages/second}$$

- **QoS 1 Rate:** The sensor sends a PUBLISH and must wait for a PUBACK (2-way trip).

$$R_{QoS1} = \frac{250 \times 10^3}{2 \times (8 \times 200)} = 78.125 \text{ messages/second}$$

- **QoS 2 Rate:** The sensor sends PUBLISH, waits for PUBREC, sends PUBREL, waits for PUBCOMP (4-way trip).

$$R_{QoS2} = \frac{250 \times 10^3}{4 \times (8 \times 200)} = 39.062 \text{ messages/second}$$

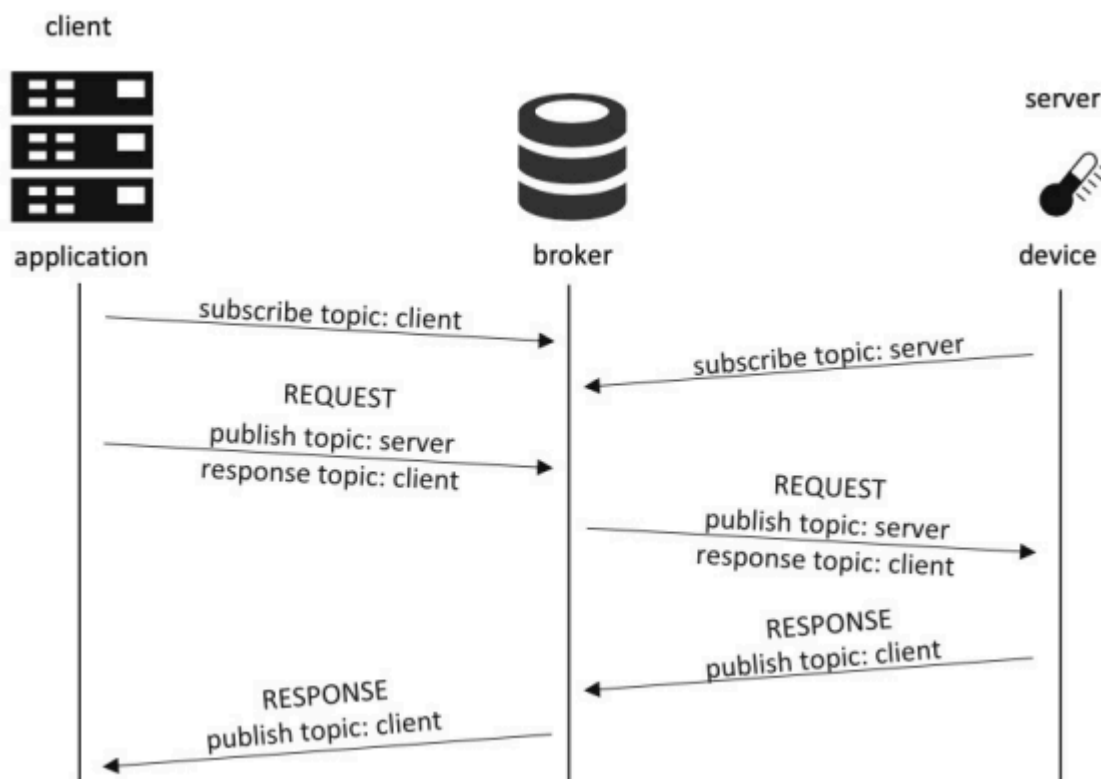
Conclusion: If the underlying TCP connection is already highly reliable, it is highly preferable to use QoS 0 sessions to maximize the data transmission rate.

MQTT v5: Request/Response Support

While inherently a Pub/Sub protocol, MQTT v5 introduced formal support for Request/Response transmissions, allowing it to emulate REST-like architectures while utilizing the broker to coordinate messages.

To achieve this bidirectionality, both clients (applications) and servers (devices) behave simultaneously as subscribers and publishers:

1. The **Application (Client)** subscribes to a private *Client Topic*.
2. The **Device (Server)** subscribes to public *Server Topics*.
3. When the Application wants data, it publishes a **REQUEST** to the *Server Topic*. Crucially, this request includes a "response topic" field pointing to its private *Client Topic*.
4. The Broker forwards this **REQUEST** to the Device.
5. The Device processes the request and publishes a **RESPONSE** directly to the *Client Topic* specified in the initial message.
6. The Broker routes this **RESPONSE** back to the Application.



While effective, compared to native REST protocols (like HTTP or CoAP), MQTT's emulation of Request/Response introduces additional latency because every single message must be inherently routed and forwarded through the intermediary broker.

7. CoAP (Constrained Application Protocol)

While HTTP is the undisputed standard for the traditional Internet, it was never designed for the unique constraints of the Internet of Things. Enter **CoAP (Constrained Application Protocol)**, a protocol explicitly created for data transmission between sensors within Low-power and Lossy Networks (LLNs). Standardized by the IETF in RFC 7252 in 2014, CoAP is highly prevalent in applications such as smart energy, smart grids, industrial automation and control systems, and environmental monitoring.

CoAP vs. HTTP and the Role of UDP

The most fundamental difference between CoAP and HTTP is the underlying transport layer. While HTTP relies on TCP (a connection-oriented protocol that guarantees delivery), CoAP is built entirely on top of **UDP (User Datagram Protocol)**.

Because UDP is connectionless, CoAP avoids the heavy overhead of establishing and maintaining TCP handshakes. This makes CoAP perfectly suited not only for standard unicast scenarios but also for multicast and broadcast applications—capabilities that are vital in many IoT environments (e.g., sending a single command to turn off all smart lights in a room simultaneously). Furthermore, UDP intrinsically improves network latency because it lacks a native retransmission mechanism; it does not pause the flow of traffic to recover lost packets, which avoids introducing arbitrary delays.

Network Architecture and Proxying

CoAP is a RESTful protocol deliberately designed to map seamlessly onto HTTP functionality. This allows for a hybrid IoT infrastructure: CoAP is used strictly in the constrained *access network* (sensor to gateway), while HTTP is used in the powerful *core network* (gateway to application).

In this architecture, the IoT Gateway acts as a highly specialized **proxy**. When an application needs data, it sends a standard HTTP GET request via the core network (carried over TCP, IPv4, and a physical medium like IEEE 802.11 Wi-Fi).

Upon receiving this HTTP request, the proxy gateway maps it directly into a CoAP GET message. This translated CoAP message is then sent into the access network (carried over UDP, IPv6, 6LoWPAN, and a medium like IEEE 802.15.4).

At the sensor (which acts as a CoAP server), the request triggers a readout. The sensor replies with a CoAP 2.05 Content response message. When this reaches the gateway, it is translated back into an HTTP 200 OK response for the application. Because both CoAP and HTTP are inherently stateless protocols, this complex proxy mapping is also performed statelessly, requiring minimal memory overhead on the gateway. Finally, CoAP uses a highly compressed binary encoding format (unlike HTTP's text-based headers) to radically reduce the transmission rate over the air.

8. CoAP Architecture and Message Types

To compensate for UDP's lack of reliability, CoAP utilizes a unique two-layer internal architecture:

1. **The MESSAGE Sublayer (Lower):** Interfaces directly with UDP. It is responsible for handling optional retransmissions (when confirmable mode is active) and ensuring message deduplication.
2. **The REQUEST/RESPONSE Sublayer (Upper):** Interfaces with the application data. It is responsible for parsing, constructing, and managing the RESTful binary messages.

The Four Message Types

CoAP defines four distinct message types at the MESSAGE sublayer to handle different reliability requirements:

- **Confirmable (CON):** Used when reliability is mandatory. The sender requires an explicit acknowledgment from the receiver.
- **Non-confirmable (NON):** Operates on a "fire-and-forget" approach. Used for frequent, non-critical telemetry data where speed and battery preservation outweigh the need for guaranteed delivery.
- **Acknowledgment (ACK):** Sent explicitly in response to a Confirmable (CON) message to confirm receipt.
- **Reset (RST):** Sent by a receiver to indicate that it has received a message (either CON or NON) but cannot process it, signaling a critical error.

9. Transaction Models and Sessions

A single request/response interaction—identified by a shared Message ID—is referred to as a **CoAP Transaction**.

Confirmable vs. Unreliable Transactions

In a **Confirmable Transaction**, the client sends a CON request tagged with a specific 16-bit message ID (e.g., 0x8c56). The device receives it and replies with an ACK notification bearing the exact same message ID. If the client does not receive this ACK, the MESSAGE sublayer automatically attempts to retransmit the CON message three more times, exponentially increasing the timeout duration with each attempt.

In an **Unreliable Transaction**, the client simply sends a NON request (e.g., 0x8c57). The sensor processes it but does not issue an ACK.

Piggybacked vs. Separated Responses

When a client sends a request that requires data to be returned (like a GET request for a temperature reading), the timing of the sensor's response dictates how the transaction unfolds.

The Piggybacked Response:

If the sensor has the temperature readout immediately available, it "piggybacks" the data directly onto the required ACK message.

- *Flow:* Client sends CON GET → Sensor instantly replies with ACK 2.05 Content (containing the temperature). If the data doesn't exist, the sensor instead replies with ACK 4.04 Not Found .

The Separated Response:

If the sensor must wake up a physical component or poll data, it will not have the readout instantly available. To prevent the client from timing out and retransmitting, the transaction is split.

- *Flow:* Client sends CON GET → Sensor immediately replies with an *empty* ACK (acknowledging receipt of the command).
- Later, once the temperature is polled, the Sensor acts as a client and sends a brand new CON 2.05 Content message containing the data.
- The original Client receives it and replies with an *empty* ACK .

Tokens vs. Message IDs

During separated responses, how does the client know that the delayed data belongs to its original request? This introduces the difference between a Message ID and a Token.

- **Message ID:** A 16-bit identifier strictly used to track a single *transaction* (an initial message and its immediate ACK).
- **Token:** A variable-length identifier used to track a logical *session* across multiple transactions. In the Separated Response model, the original `CON GET` and the delayed `CON 2.05 Content` will have entirely different Message IDs, but they will perfectly share the same Token (e.g., `0x21`).

10. CoAP Message Format and Response Codes

CoAP uses a highly optimized sequence of binary fields to eliminate overhead while maintaining HTTP compatibility. Every CoAP message starts with a fixed-length 32-bit (4-byte) header, structured as follows:

1. **VER (Version - 2 bits):** Currently fixed at `1`. Other values are reserved for future protocol versions.
2. **T (Type - 2 bits):** Specifies the message type: `0` (CON), `1` (NON), `2` (ACK), or `3` (RST).
3. **TKL (Token Length - 4 bits):** Specifies the exact length in bytes of the Token field that follows the header. It ranges from 0 to 8 bytes. (Lengths greater than 9 bytes are strictly reserved for special applications).
4. **CODE (8 bits):** Indicates if the message is a request or a response.
5. **MESSAGE ID (16 bits):** Used to detect duplicates and match ACKs to CONs.

Immediately following this 4-byte fixed header is the **Token** (if `TKL > 0`), a sequence of **Options** (which act like HTTP headers for URI paths, content types, etc.), and finally the actual **Payload** data.

Decoding the CODE Field

The 8-bit CODE field is formatted as `a.bb`, where `a` is a 3-bit class digit (0 to 7) and `bb` is a 5-bit detail digit (00 to 31).

- If $a = 0$, the message is a **Request**. The `bb` field defines the method: `01` (GET), `02` (POST), `03` (PUT), `04` (DELETE).
- If $a > 0$, the message is a **Response**. The classes map directly to HTTP status codes:

- $a = 2$: Success (e.g., 2.05 Content)
- $a = 3$: Redirection
- $a = 4$: Client Error (e.g., 4.04 Not Found)
- $a = 5$: Server Error

11. CoAP Latency Calculation (Exercise)

To understand CoAP overhead, consider an end-to-end Non-confirmable scenario. An application requests a readout from a sensor via a Gateway.

The setup parameters are:

- Access Network (Sensor to GW): IEEE 802.15.4/6LoWPAN at 250 Kbps
- Core Network (GW to App): Ethernet/IPv4 at 1 Gbps
- The only delay is transmission delay (ignore propagation).
- Packet sizes: UDP Header = 8 bytes, CoAP GET Request = 20 bytes, CoAP 2.05 Content = 10 bytes, IPv4 Header = 20 bytes, 6LoWPAN Header = 10 bytes, Ethernet Header = 16 bytes, IEEE 802.15.4 Header = 10 bytes.

Step 1: Calculate Request Delay (T_1)

The request travels from App $\rightarrow$ GW (Ethernet), then GW $\rightarrow$ Sensor (802.15.4).

Payload at this stage is the GET Request (20 bytes).

$$T_1 = T_{\text{Ethernet}} + T_{802.15.4}$$

$$T_1 = \frac{8 \times (L_{eth} + L_{IPv4} + L_{UDP} + L_{GET})}{10^9} + \frac{8 \times (L_{802.15.4} + L_{6LoWPAN} + L_{UDP} + L_{GET})}{250 \times 10^3}$$

$$T_1 = \frac{8 \times (16 + 20 + 8 + 20)}{10^9} + \frac{8 \times (10 + 10 + 8 + 20)}{250,000}$$

Because 10^9 (1 Gbps) is so large, the Ethernet transmission time is effectively negligible (≈ 0 ms). Thus, T_1 relies purely on the access network:

$$T_1 = 0 + \frac{8 \times 48}{250,000} = 1.536 \text{ ms} \approx 1.54 \text{ ms}$$

Step 2: Calculate Response Delay (T_2)

The response travels from Sensor $\rightarrow$ GW (802.15.4), then GW $\rightarrow$ App (Ethernet).

Payload at this stage is the 2.05 Content (10 bytes).

$$T_2 = \frac{8 \times (L_{802.15.4} + L_{6LoWPAN} + L_{UDP} + L_{2.05Content})}{250 \times 10^3} + \frac{8 \times (L_{eth} + L_{IPv4} + L_{UDP} + L_{2.05Content})}{10^9}$$

$$T_2 = \frac{8 \times (10 + 10 + 8 + 10)}{250,000} + \approx 0$$

$$T_2 = \frac{8 \times 38}{250,000} = 1.216 \text{ ms} \approx 1.22 \text{ ms}$$

Step 3: Total Latency

The total time required for the application to receive the sensor readout is the sum of the request and response delays:

$$T = T_1 + T_2 = 1.54 \text{ ms} + 1.22 \text{ ms} = 2.76 \text{ ms}$$

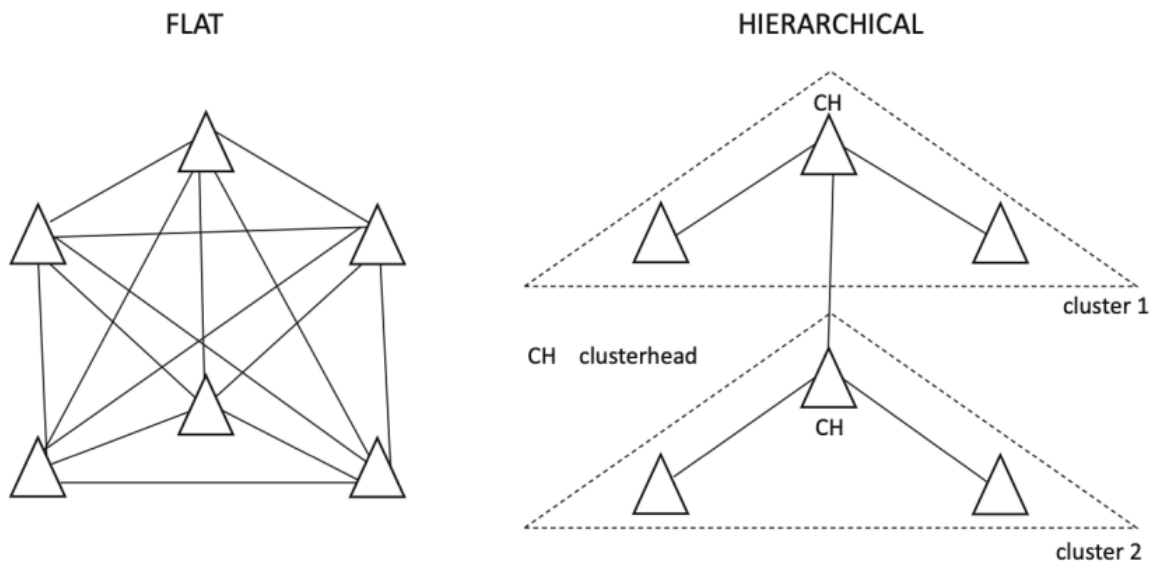
Routing Architecture in the Internet of Things

Routing in the Internet of Things (IoT) fundamentally traces its origins back to the routing mechanisms developed for Wireless Sensor Networks (WSNs). Because of the unique constraints and topologies of sensor networks, traditional routing protocols designed for Mobile Ad Hoc Networks (MANETs) are inherently unsuitable for both WSNs and IoT environments. Instead, routing in these contexts must carefully balance data delivery needs against strict energy constraints.

The general philosophy behind IoT and WSN routing mechanisms is to maximize the overall lifetime of the network. This is typically achieved by distributing routing traffic across as many devices as possible, preventing any single node from prematurely depleting its battery. However, there is an inherent trade-off in network design: lowering power consumption generally makes the routing mechanism less reactive, requiring engineers to find a delicate balance between network reactivity and energy preservation.

Network Topologies: Flat vs. Hierarchical Routing

Routing protocols in WSNs are broadly categorized into two structural topologies: flat and hierarchical.



Flat Routing

In a flat routing architecture, all devices exist on the same structural level and interact directly with one another. There is no single "parent" device responsible for concentrating or aggregating traffic from "child" devices.

- **Advantage:** Flat routing generally offers less delay in data transmission.
- **Disadvantage:** This lack of structure comes at the cost of lower energy efficiency.

Hierarchical Routing and Capillary Networks

Hierarchical routing introduces structural tiers by grouping devices into clusters. Within each cluster, a designated head node (the cluster head, or CH) acts as a parent device.

- Child devices in a hierarchical setup typically do not interact with other standard nodes.
- All traffic generated by child nodes is routed directly to their respective cluster heads.
- The cluster heads are responsible for concentrating and aggregating this traffic, communicating exclusively with other cluster heads to provide end-to-end network connectivity.
- **Advantage:** This method yields higher energy efficiency, achieved primarily through multihop and mesh routing techniques.
- **Disadvantage:** The aggregation and multi-tier communication inherently

introduce greater latency and delay.

The implementation of hierarchical multihop routing creates what is known as a capillary network. In a capillary network, individual devices forward their data messages through intermediate intermediary devices until the data successfully reaches a cluster head. On average, this multihop approach allows individual devices to consume significantly less power by passing traffic to nearby intermediate nodes rather than attempting to broadcast it directly to a distant recipient or application.

Paradigm Shifts: Data-Centric and Location-Based Routing

While traditional networks route based on specific IP addresses, IoT and WSN environments frequently utilize alternative paradigms better suited for sensory data.

Data-Centric Routing

As hierarchical networks grow, aggregated traffic requires extra buffering and retransmissions, which naturally increases network latency. This evolution leads to multi-hop clustering, where multiple cluster heads aggregate data from various devices before passing it along to other cluster heads.

To optimize this, networks utilize data-centric routing—the primary alternative to traditional address-based routing. Instead of targeting a specific IP address, clients send requests to specific geographical areas of the network to obtain specific readouts or data types (e.g., temperature or humidity).

- In response to an initial query, multiple devices act as sources, collecting readouts and forwarding the relevant information back to the receiver.
- For example, temperature, humidity, and pressure readouts are tagged and forwarded by network routers until they reach the specific application designed to process that exact data type.
- Conceptually, this routing technique mirrors the Publish/Subscribe (Pub/Sub) model. Data-centric routing operates at the application layer, where user requests function as "subscriptions" and the sensor responses operate as "notifications".

In a typical data-centric transaction, a remote user at the edge of an IP cloud sends a query targeting a specific type of sensor data. The request propagates

through a gateway and floods the WSN via intermediate forwarding nodes until it reaches the relevant sensors. The sensors holding matching data then route their responses back through the network to the remote user.

Key Benefits of Data-Centric Routing:

- **Traffic Reduction:** It filters out unnecessary data, aggregates responses, and eliminates duplicate packets.
- **Energy Efficiency:** Highly optimized for the strict constraints of sensor networks.
- **Scalability & Flexibility:** Functions seamlessly with massive numbers of nodes and allows new data to become available without requiring complex address reconfiguration.
- **Reliability:** Utilizes multi-path cooperation to ensure data delivery.

Comparison: Traditional vs. Data-Centric Routing

Feature	Traditional Routing (IP-centric)	Data-Centric Routing (Content-centric)
Based on	Node addresses	Content/Data
Goal	Reach a specific node	Obtain information
Communication	Unicast to destination	Queries / Publications
Example	Send to node 192.168.1.10	I want temperature > 30°C
Typical in	Traditional Internet	WSN, IoT, NDN, etc.

Location-Based Routing

Location-based routing serves as an alternative to both data-centric and address-based routing. In this paradigm, a client or device forwards datagrams to a destination based entirely on the destination's geographic or physical coordinates.

- The client and every router along the transmission path use their own coordinates to estimate the optimal direction of packet propagation.
- A router will consistently forward the datagram to the next neighboring router that is physically closest in the direction of the final destination.
- This directional forwarding ensures efficient delivery with minimal energy consumption and can be effectively combined with hierarchical routing to leverage the benefits of traffic aggregation.

The Mechanics of Traffic Aggregation

Traffic aggregation is a critical component of energy-efficient routing, but it must be managed carefully. While it reduces overall traffic, the aggregated data requires high computational power to be processed and compressed by the device.

Furthermore, if throughput increases too much, it can limit channel capacity, leading to network contention and additional latency.

- **Without Aggregation:** Every single node sends its data separately. This floods the network with many individual packets, resulting in high overhead from headers and routing info, higher chances of packet collisions, increased retransmissions, and ultimately, higher energy consumption and lower throughput.
- **With Aggregation:** Intermediate nodes collect data from multiple sources and aggregate it (e.g., calculating averages, sums, min/max values, or detecting events). Only a few aggregated packets are forwarded onward. This results in fewer packets transmitted, less network congestion, fewer collisions, and lower energy consumption, which ironically allows for a higher effective throughput of *useful* data.

Quantitative Example of Aggregation Efficiency: Consider a scenario with 10 source nodes, each generating 10 bytes of data.

- **Without Aggregation:** 10 packets are sent. Including a 40-byte header per packet, the total bytes sent is 500 bytes to deliver 100 bytes of useful data (20% efficiency).
- **With Aggregation:** Only 1 aggregated packet is sent. The total bytes sent is 140 bytes to deliver the same 100 bytes of useful data (71% efficiency). This yields a throughput roughly 3.5 times higher.

Ultimately, while aggregation drastically reduces traffic and energy consumption, it introduces processing costs and potential delay, requiring network architects to navigate the trade-off between delay, energy, and throughput.

Routing Strategies: Proactive, Reactive, and Hybrid

Beyond topology and addressing paradigms, routing mechanisms are classified by their operational strategies, which dictate how they discover and maintain network paths.

1. **Proactive Routing (Table-driven):** * Routes are calculated and maintained in advance, meaning a route to any destination is always immediately available.
 - It relies on nodes maintaining up-to-date routing tables through the periodic, continuous transmission of routing information.
 - **Pros/Cons:** It is highly effective when network traffic is high and low latency is strictly required. However, it suffers from heavy control-traffic overhead because routing updates are transmitted continuously, even when actual data is not being sent.
2. **Reactive Routing (On-demand):** * Routes are discovered dynamically and only when strictly needed.
 - When a source needs to reach a destination, it broadcasts a Route Request (RREQ) that propagates through the network. The destination (or an intermediate node with a valid route) responds with a Route Reply (RREP). The route is then used only until it is no longer needed.
 - **Pros/Cons:** It minimizes the transmission of control traffic, making it highly energy-efficient. However, the initial route-discovery query delays data transmission until routing decisions are finalized.
3. **Hybrid Routing (Zonal/Hybrid):** * Combines the advantages of both proactive and reactive routing across different network sections or zones.
 - Each localized zone maintains its internal routes proactively. However, routes *between* distant zones are discovered reactively on demand.
 - **Pros/Cons:** It is heavily utilized in hierarchical schemes (e.g., proactive within a cluster, reactive between clusters) to vastly reduce control traffic across long distances while keeping local latency low.

WSN Specific Routing Protocols: From Flooding to Negotiation

Routing schemes in Wireless Sensor Networks (WSNs) incorporate a variety of algorithms that integrate aspects of flat and hierarchical architectures, combined with data-centric and location-based mechanisms. The techniques range from simple, brute-force datagram propagation to highly sophisticated, resource-aware negotiation protocols.

1. Flooding: The Simplest Path Discovery

Flooding is a flat routing technique and represents the most basic method for path discovery in data networks. In a flooding scenario, devices operate under a simple rule: forward any received datagram to *all* possible neighboring nodes.

Advantages:

- **Simplicity and Memory Efficiency:** Flooding does not rely on routing tables. Therefore, it requires no extra memory and imposes no additional computational complexity on the device.
- **Resilience:** Its highly dynamic nature makes flooding exceptionally well-suited to networks with rapidly changing topologies.

Disadvantages and Safeguards:

- **Energy-Intensive:** Creating many copies of the same datagram leads to high channel utilization and network contention, which can result in data loss and increased latency. This makes flooding impractical for many low-power contexts.
- **Loop Prevention:** Because a device could theoretically forward the same datagram an unlimited number of times, flooding relies on **hop-count limits** to prevent infinite loops. As a datagram travels through the network, its hop counter decreases by 1 each time it is forwarded. When the counter reaches zero, the datagram is discarded by the receiving device.

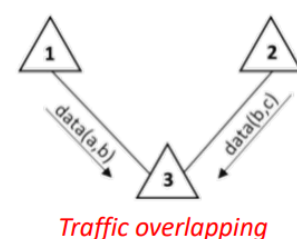
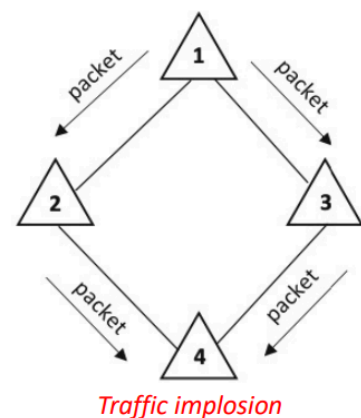
The Inefficiencies of Flooding

Flooding suffers from two major energy inefficiencies:

1. **Traffic Implosion:** This occurs when the exact same datagram, generated by a source device, arrives as multiple copies at a destination device through different paths. This wastes energy, bandwidth, and channel capacity.

Description: A diamond-shaped network of four devices (1, 2, 3, 4). Device 1 generates a packet and forwards it to both Device 2 and Device 3. Subsequently, both Device 2 and Device 3 forward their copies of the exact same packet to Device 4. Device 4 receives two identical copies, demonstrating implosion.

2. **Traffic Overlapping:** This occurs when redundant data, rather than identical packets, is transmitted across multiple datagrams that arrive at the destination simultaneously.



<i>Description: Three devices forming a 'V' shape. Device 1 transmits a datagram containing data associated with assets 'a' and 'b' ($Data(a,b)$) to Device 3. Simultaneously, Device 2 transmits a datagram containing data associated with assets 'b' and 'c' ($Data(b,c)$) to Device 3. Device 3 receives two redundant copies of the data for asset 'b', wasting channel capacity.</i>	
---	--

Quantitative Delay in Flooding:

Consider a scenario to find the delay of a single datagram traversing a specific path. Given a network topology where the transmission/propagation of a single datagram takes 1 second per hop, how long does it take to go from a starting device 'A' to a destination 'B'? In a flooding protocol, the datagram simply takes the shortest available path. If the minimum distance between A and B is 3 hops, the delay will simply be 3 seconds.

2. Gossiping: Randomized Forwarding

Gossiping is a variation of flooding (and thus, also a flat routing algorithm), but it is designed to mitigate the extreme energy waste of total network flooding.

Instead of broadcasting a datagram to all neighbors, intermediary devices in a gossiping protocol forward a received datagram to a **randomly selected single neighboring node**. Like flooding, gossiping datagrams include hop counters to limit how many times they can be forwarded.

- **Advantage:** Limits the number of transmissions associated with each hop, significantly reducing immediate power consumption.
- **Disadvantage:** While flooding guarantees the datagram takes the most efficient (shortest) path, the random nature of gossiping can result in a highly inefficient, meandering path. Longer paths imply greater latency compared with flooding. Furthermore, while gossiping saves energy per hop, an excessively long random path can ultimately result in higher total energy consumption than the shortest randomly selected path.

Delay Example in a Ring Topology:

Consider a WSN with a ring topology of 5 devices where packets are transmitted uniformly to provide full connectivity. Assume the transmission rate is 250 Kbps, the packet size is 200 bytes, and collisions/delays (except transmission delay) are ignored. We want to find the expected transmission delay in a flooding scenario.

First, we calculate the transmission delay of a packet on a single hop (D_h):

$$D_h = \frac{8 \times 200 \times 10^{-3}}{250} = 6.4ms$$

Each device sends to $N = 4$ other devices in the 5-node ring. Therefore, the minimum delay for the two nearest neighboring devices is D_h , while the delay for the other two devices on the far side of the ring is $2D_h$. The expected delay is averaged as:

$$D_{flooding} = \frac{2 \times D_h + 2 \times 2 \times D_h}{N} = 9.6ms$$

3. SPIN: Sensor Protocols for Information via Negotiation

To solve the blind inefficiencies of flooding and gossiping, networks utilize **SPIN**—a data-centric, flat routing mechanism that operates on data negotiation and resource adaptation.

Instead of blindly forwarding raw data, SPIN requires devices to advertise **metadata** (data that describes the actual data) before any datagrams are forwarded. Because metadata is significantly smaller than the data it describes, transmission rates are lower, and less energy is consumed.

Core Mechanisms of SPIN:

- **Data Negotiation:** Data is sent *only* to devices that are truly interested in using or forwarding it. This completely eliminates the redundant data transmissions that cause traffic implosion and overlapping.
- **Resource Adaptation:** Devices monitor their current energy levels. If energy drops too low, a device can autonomously reduce or entirely eliminate its forwarding of metadata and actual traffic, effectively controlling and extending the network's lifetime.

The 3-Step SPIN Transaction:

SPIN routing relies on three specific message types:

1. **Advertising (ADV):** A device advertises the metadata describing the type of data it has.
2. **Request (REQ):** If a neighboring device is interested in the advertised data, it replies with a REQ message.
3. **Data (DATA):** Upon receiving the REQ, the original transmitter sends a DATA message containing the actual, full-sized information. The receiving node then begins advertising its own ADV to its neighbors, continuing the chain.

4. Advanced SPIN Variations

The SPIN protocol has evolved into several specialized versions to handle specific network constraints:

- **SPIN-EC (Energy Conservation):** Takes resource adaptation a step further. A device evaluates its energy threshold *before* engaging in a transaction. It will only send an ADV message if it calculates it has enough energy to process a subsequent REQ and transmit the large DATA message. Conversely, if it receives an ADV, it will only send a REQ if it has enough power to receive and process the incoming DATA.
- **SPIN-BC (Broadcast):** While standard SPIN is a Peer-to-Peer (P2P) mechanism, SPIN-BC is designed for networks sharing a single broadcast channel. When a device sends an ADV, REQ, or DATA message, *all* devices in the coverage area receive it. This exploits redundancy. Timing becomes critical: a device will intentionally delay sending a REQ message to see if a neighbor requests it first (letting the neighbor spend the energy). The delay duration is tied to available energy; nodes with more energy wait less time before transmitting a REQ, effectively volunteering to bear the communication cost.
- **SPIN-RL (Reliable):** Designed for lossy networks prone to packet loss. It adds reliability in two ways:
 1. It triggers retransmissions if a DATA message fails to arrive after a REQ has been sent.
 2. It sends periodic ADV messages, which receiving devices track to ensure they aren't processing redundant requests.
 - *Note on vulnerability:* A common flaw across all SPIN versions is that if a critical intermediate device fails to propagate ADV messages, the entire end-to-end network connectivity can collapse.

Directed Diffusion: Data-Centric Multipath Routing

Directed Diffusion is a highly efficient flat, data-centric routing mechanism. While SPIN uses metadata negotiation to save energy, Directed Diffusion focuses on aggregating responses and propagating datagrams in a multipath mode by exchanging messages between neighboring devices.

It provides routing via a Publish/Subscribe (Pub/Sub) scheme. An application or device acts as a "querier," periodically broadcasting a query that represents a specific *interest*. It hopes that some device in the network will have the matching sensor data and serve the request.

Message exchange in Directed Diffusion relies on four primary components:

1. **Interests:** Define the nature of the requested data (the query).
2. **Gradients:** Specify the spatial direction (the path back to the requester) and the required data rate.
3. **Data messages:** The actual sensor readouts matching the interest.
4. **Reinforcements:** A mechanism that allows a device to assign higher priority to certain, more reliable paths in a multipath scenario.

How Interests and Caches Work

An interest is presented as an attribute-value pair. For example, a sink device (Device 1) might want temperature readouts from a specific area, transmitted every second for one hour.

Example Interest Structure:

- **Type:** Temperature (Temperature readouts)
- **Interval:** 1 s (Report events every second)
- **Duration:** 3600 s (Report for an hour)
- **Field:** [(41.40, 2.17), (41.41, 2.19)] (Report from sensors in this specific coordinate area)

As this interest is flooded through the network (e.g., from sink Device 1 to neighbors 2 and 3, and then to 4, 5, and 6), it is stored by the intermediate devices in a **cache**. The interest is forwarded unchanged, except the "sink" field is modified at each hop to point to the immediate neighbor that forwarded it.

Every cache entry includes:

- **A timestamp:** Indicating when the interest was last received.
- **The gradient:** Pointing to the neighbor that transmitted the interest, specifying the direction and rate at which readouts should be transmitted.
- **The duration:** Indicating the lifetime of the interest (after which it expires and is deleted).

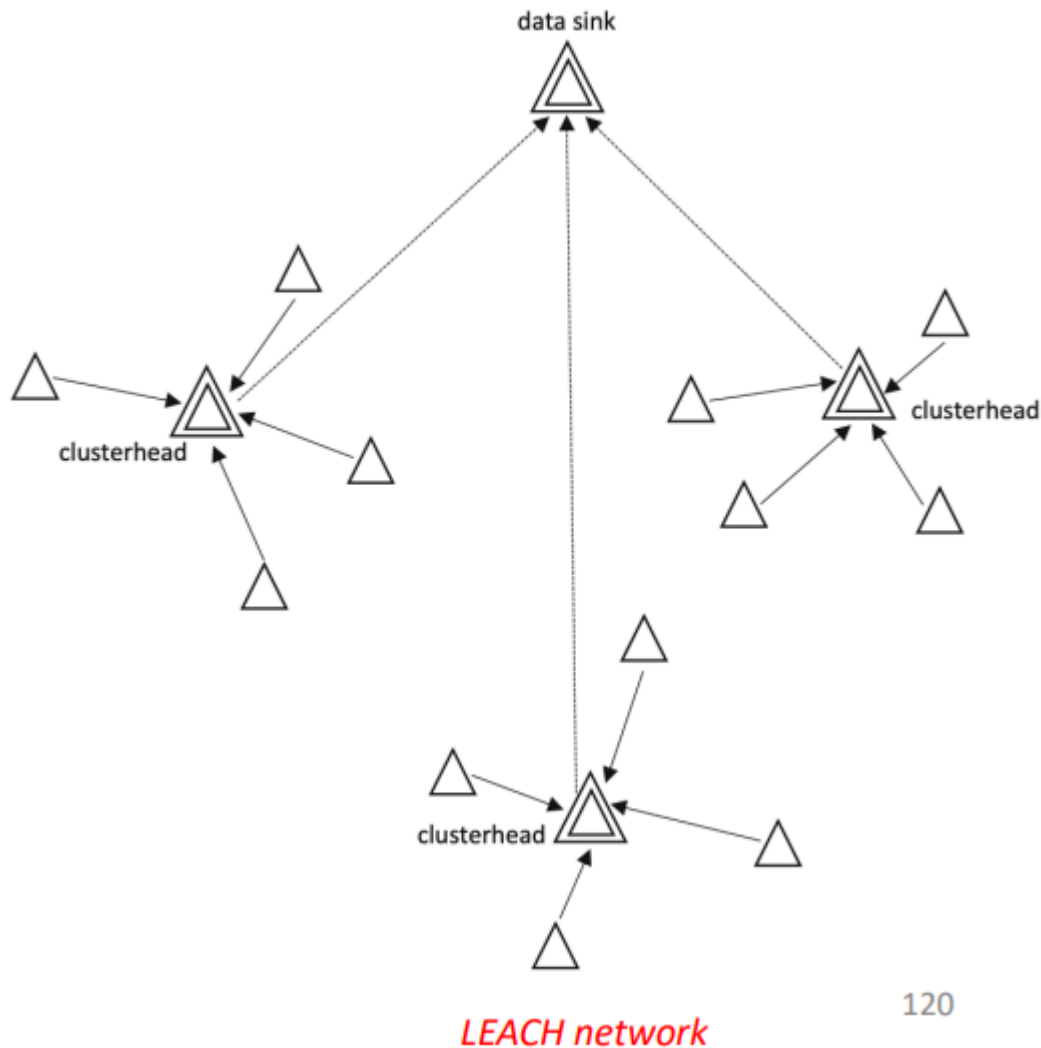
The Reinforcement Phase

When a sensor eventually transmits data that matches the interest, intermediate devices forward the data back to the sink following the established gradients.

Because the network utilizes multipath routing, the sink will likely receive the same data from multiple paths.

The sink can then **reinforce** the best path (e.g., the path with the lowest latency or least packet loss) by re-transmitting the original interest along that specific route, but requesting a higher data rate. Other, slower paths might be allowed to expire or remain at a low baseline data rate. Through continuous updates, a single interest can maintain multiple gradients associated with different data rates and expiration times.

Hierarchical Routing: The LEACH Protocol



LEACH (Low Energy Adaptive Clustering Hierarchy) is a prominent hierarchical routing algorithm that groups nodes into clusters and designates a Cluster Head (CH) to aggregate and transmit data to the final application (the sink).

By leveraging hierarchical routing and data aggregation, LEACH significantly extends the network's lifetime. To conserve energy and limit overhead traffic, CHs actively eliminate any redundant data before forwarding the aggregated packet to the sink.

To manage interference and access, LEACH structures its communication physically:

- **TDMA (Time Division Multiple Access):** Used for communication *inside* the cluster. Devices are assigned specific time slots to transmit to the CH.
- **CDMA (Code Division Multiple Access):** Used for communication *between* devices inside and outside a cluster to prevent cross-cluster interference.

The Round-Based Scheme

Because acting as a CH consumes significantly more energy than being a regular child device, LEACH implements a dynamic, **round-based scheme** to rotate the CH role. This distributes the energy burden across all devices in the network. Each round is divided into two distinct phases:

1. Cluster Setup Phase

This phase is intentionally kept brief to minimize latency and control overhead. The selection of the CH is completely distributed, requiring very little inter-device communication.

Given N devices, a device n decides whether to become a CH based on a threshold $T(n)$. The threshold is calculated as:

$$T(n) = \begin{cases} \frac{P}{1 - P(r \bmod (\frac{1}{P}))} & \text{if } n \in G \\ 0 & \text{otherwise} \end{cases}$$

- $r \geq 0$ is the current round number.
- P is the desired probability (e.g., 20% or $1/5$) that any given device will be a CH in a round.
- G is the set of devices that have *not* been selected as CHs in the last $1/P$ rounds.

Each device generates a uniformly distributed random number $v \in (0, 1)$. **If $v < T(n)$, the device becomes a CH.** Otherwise, it remains a regular node. This ensures that every device acts as a CH exactly once every $1/P$ rounds. Once CHs announce themselves, regular devices estimate their distance to the available CHs (based on received signal strength), pair with the closest one, and receive their TDMA schedules.

2. Steady State Phase

This phase is much longer (though limited to prevent total CH battery depletion). It consists of multiple TDMA frames. Each child device transmits its data only once per frame during its assigned time slot, allowing the device to sleep when it is not its turn.

XLEACH: Fixing the Energy Blindspot

A major flaw in standard LEACH is that the CH selection process does not look at actual battery life. If a device has almost no power left but hasn't been a CH recently, it will still be forced into the high-energy CH role and die.

Quantitative Problem: A WSN has five sensors (A, B, C, D, E) with battery levels $P_A = 94\%$, $P_B = 97\%$, $P_C = 99\%$, $P_D = 91\%$, and $P_E = 4\%$. In the last 4 rounds, A, B, C, and D have been CHs. With a CH probability of $P = 1/5$, which sensor will be selected next under standard LEACH?

Solution: Because standard LEACH ignores energy levels, Node E is the only one remaining in set G . It will be forced to be the CH, likely causing it to die instantly.

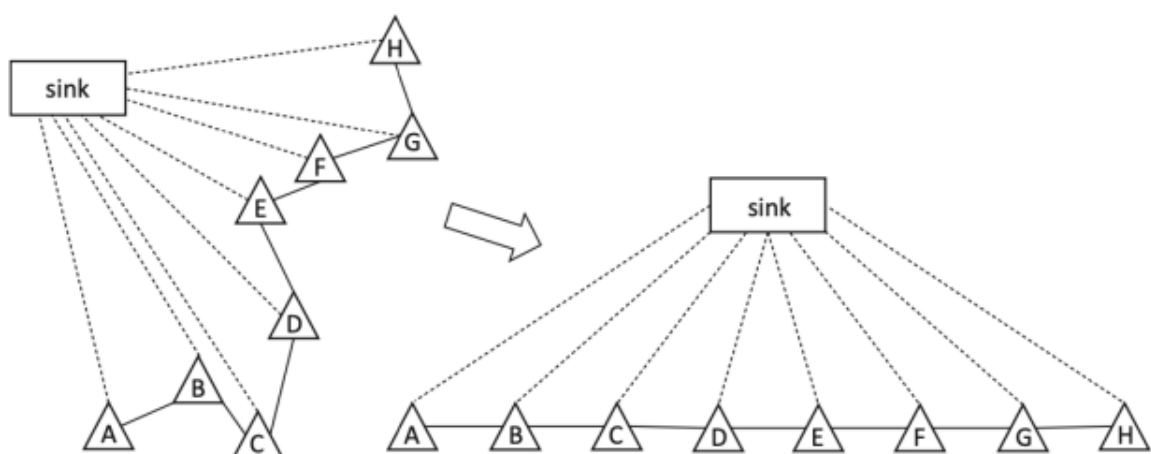
XLEACH (eXtended LEACH) solves this by factoring current battery levels into the threshold equation. The new threshold relies on the ratio of current energy to maximum initial energy:

$$T(n) = \frac{P}{1 - P(r \bmod (\frac{1}{P}))} \left[\frac{E_{n,current}}{E_{n,max}} + (r_n \bmod \frac{1}{P}) \left(1 - \frac{E_{n,current}}{E_{n,max}} \right) \right]$$

- $E_{n,current}$ is the current energy level.
- $E_{n,max}$ is the initial total energy level.
- r_n tracks the number of consecutive rounds the device has *not* been a CH.

When r_n reaches $1/P$, it triggers a reset back to the standard, non-energy-dependent threshold to ensure the node eventually takes its turn if absolutely necessary.

PEGASIS: Power-Efficient Gathering in Sensor Information Systems



PEGASIS is a hierarchical routing protocol that pushes aggregation to the extreme by ditching standard clusters in favor of a **chain structure**.

Instead of many child nodes transmitting over a distance to a central CH, PEGASIS requires a device to know the geographic position of its neighbors. It controls its transmit power to strictly communicate only with its 1-hop nearest neighbor, drastically improving the Signal-to-Noise Ratio (SNR) and saving energy.

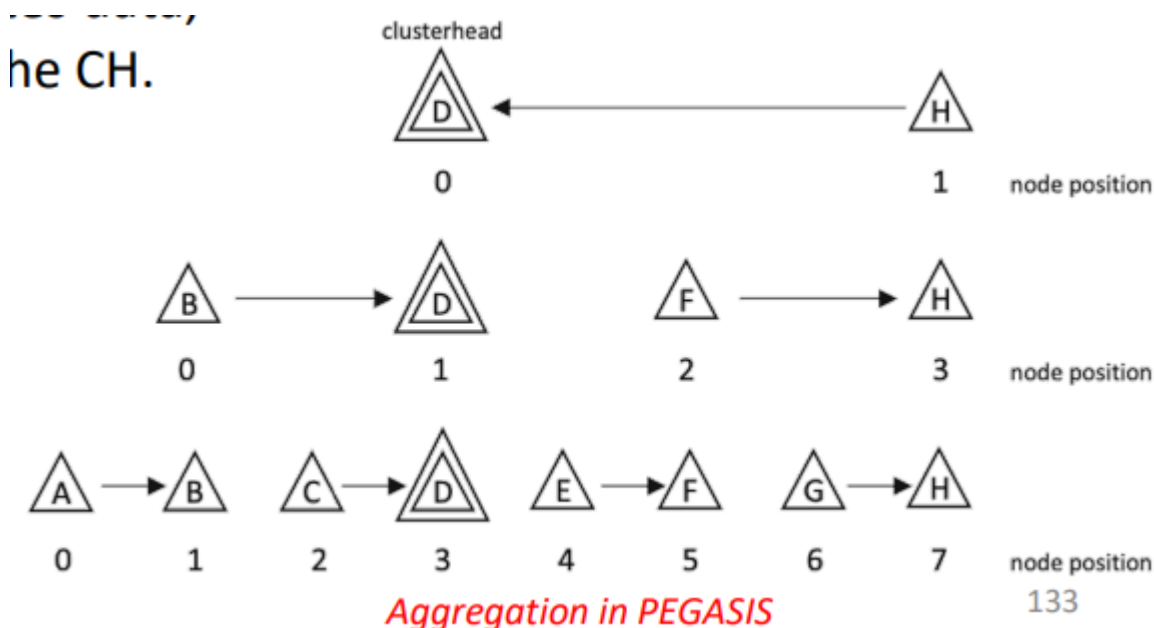
Chain Formation:

The chain is built using a greedy algorithm. Node A finds its closest neighbor, Node B. Node B then finds *its* closest unassigned neighbor, Node C, and so on, linking all the devices together.

Sequential CH Rotation:

Like LEACH, PEGASIS uses a round-based system to distribute the energy load. However, the CH role simply moves sequentially down the chain. If a chain has 8 devices, each device will be the final CH once every 8 rounds. The application/sink initiates the round by sending a special beacon.

Aggregation in PEGASIS



Data aggregation happens at almost every step along the chain, not just at the CH. PEGASIS can perform simultaneous data aggregation across multiple devices.

Consider an 8-device chain labeled A through H (occupying positions 0 through 7). Assume Node D (position 3) is currently the CH:

- **Phase 1:** All devices in *even* positions (A/0, C/2, E/4, G/6) transmit their data to their neighbors in *odd* positions (B/1, D/3, F/5, H/7).
 - *Result:* Node B holds A+B; CH D holds C+D; F holds E+F; H holds G+H.
- **Phase 2:** The active devices are now re-indexed (B is now 0, D is 1, F is 2, H is 3). Again, the new *even* positions (B and F) transmit to the new *odd* positions (CH D and H).
 - *Result:* CH D holds data from A, B, C, D. Node H holds data from E, F, G, H.
- **Phase 3:** Only CH D and Node H remain active. H forwards its aggregated data block to CH D. CH D then transmits the completely aggregated packet to the sink.

Performance Metrics of the Chain:

- **Parallel Aggregation:** If transmit power is strictly limited to immediate neighbors, interference is non-existent, allowing simultaneous transmissions. The number of aggregation stages required is approximately $\lceil \log_2 N \rceil$.
- **Sequential Aggregation:** If interference between devices prevents simultaneous transmission, the nodes must pass the data sequentially down the chain. In this case, the number of stages increases to $N - 1$.

Summary of WSN Routing Protocols

The following table categorizes the core behaviors, complexities, and efficiencies of the routing mechanisms designed for WSNs and IoT environments:

Mechanism	Hierarchical	Data-centric	Complexity	Latency	Energy consumption
Flooding	No	No	Low	Low	High
Gossiping	No	No	Medium	High	Medium
SPIN	No	Yes	Medium	High	Medium
Directed diffusion	No	Yes	Medium	Medium	Low
LEACH	Yes	No	High	Medium	Low
PEGASIS	Yes	No	Medium	Low	Low